Clase 1 · IPv4: arquitectura, direccionamiento, ruteo, fragmentación e ICMP

Oct 4, 2026 · @Joaquin

Apunte para leer a la par de las diapositivas "Protocolo IP – Arquitectura" (Marrone, Robles). Sigue el mismo orden que el PDF y explica cada concepto desde su porqué, incluidas las cosas que la clase da por sabidas.

1. Contexto: el modelo TCP/IP y dónde vive IP

IP es la capa que permite que dos máquinas que **no comparten un cable** se hablen. Todo lo demás de la clase (direcciones, máscaras, ruteo, fragmentación, ICMP) existe para que eso funcione.

El modelo en capas

El modelo TCP/IP divide el problema de comunicar dos programas en capas. Cada capa resuelve una parte, le ofrece un servicio a la de arriba y usa el servicio de la de abajo. La idea clave es el **encapsulamiento**: cada capa toma lo que le da la de arriba, le agrega su propia cabecera (header) y se lo pasa a la de abajo, sin mirar ni entender lo que hay adentro.

Capa	Qué resuelve	Unidad de datos (PDU)	Dirección que usa	Ejemplos
Aplicación	Lo que el usuario quiere hacer	Mensaje	Nombres (DNS)	HTTP, DNS, DHCP
Transporte	Comunicar **procesos** (programas)	Segmento (TCP) / datagrama (UDP)	Puerto	TCP, UDP, QUIC
Red (Internet)	Llevar paquetes entre **hosts** de redes distintas	Datagrama o paquete	Dirección IP	IP, ICMP
Enlace	Llevar tramas entre **vecinos** del mismo enlace	Trama (frame)	Dirección MAC	Ethernet, 802.11 (WiFi)
Física	Poner bits en el medio	Bits	—	Cobre, fibra, radio

Una forma útil de pensarlo: la capa de enlace sabe dar **un salto**; la capa de red encadena muchos saltos para llegar de punta a punta; la de transporte, ya en los extremos, decide a qué programa le toca el dato.

Nivel de enlace (lo que la clase da por sabido)

La capa de enlace conecta nodos **adyacentes**: dos equipos que están en el mismo enlace físico o en la misma LAN. Su unidad es la **trama**. Hace detección de errores (por ejemplo, el CRC de Ethernet), puede retransmitir, hacer control de flujo y resolver el **acceso al medio** (quién habla cuando varios comparten el canal).

Lo importante para esta materia: un mensaje que va de un origen a un destino lejano **atraviesa muchos enlaces distintos**, y cada uno puede ser de una tecnología diferente (Ethernet en tu casa, fibra del ISP, WiFi del otro lado). La trama se arma y se desarma en cada salto; lo que viaja de punta a punta, intacto en lo esencial, es el **paquete IP** que va adentro.

El esquema de IP dentro de TCP/IP

La diapositiva del "Esquema de IP" muestra a IP en el centro, con TCP y UDP arriba, Ethernet abajo y varios **protocolos auxiliares ("helpers")** alrededor:

- **ICMP**: reporta errores y permite diagnósticos (ping, traceroute). Viaja dentro de IP. Se ve al final de esta clase.
- **IGMP**: permite que un host le avise al router que quiere recibir tráfico de un grupo multicast. Viaja dentro de IP.
- **ARP**: traduce una dirección IP a la dirección MAC del vecino, porque la trama necesita una MAC para salir. Viaja directamente dentro de la trama, no dentro de IP. Es la clase 3.

IP "necesita ayuda" porque por sí solo no sabe avisar errores (de eso se encarga ICMP) ni sabe qué MAC poner en la trama (de eso se encarga ARP).

2. Nivel de red: qué se le pide y qué da IP

La capa de red tiene que hacer que cualquier host le pueda mandar un mensaje a cualquier otro, aunque estén en redes distintas y de distinta tecnología. IP lo hace con el servicio más simple posible: **mejor esfuerzo, sin conexión**.

Los servicios que la clase enumera, explicados

- **Independiente de la tecnología de subred.** A TCP no le importa si abajo hay Ethernet, WiFi o fibra. IP es la "capa común" que tapa esas diferencias.

- **Transporte aislado de la cantidad, tipo y topología de las subredes.** El host que envía no necesita conocer el mapa de Internet: le da el paquete a IP y IP se arregla.
- **Direccionamiento uniforme.** Un solo esquema de direcciones (las IP) que vale igual en cualquier LAN o WAN del mundo. Las MAC no sirven para esto porque solo tienen sentido dentro de un enlace.
- **Transportar mensajes de un emisor a un receptor, incluso en redes distintas.** Es la razón de ser de la capa.
- **Determinar el camino (path).** Alguien tiene que decidir por dónde va cada paquete. Eso es el **ruteo** (sección 8).

Con conexión vs sin conexión (circuitos virtuales vs datagramas)

Una red puede funcionar de dos maneras:

	Circuitos virtuales (con conexión)	Datagramas (sin conexión)
Antes de mandar datos	Se establece un camino y cada router lo registra	No se establece nada
Qué lleva cada paquete	Un identificador corto de circuito	La dirección completa de destino
Estado en los routers	Sí, por cada conexión	No, solo la tabla de ruteo
Camino	Todos los paquetes siguen el mismo	Cada paquete se rutea por separado y pueden tomar caminos distintos
Si se cae un router	Se caen las conexiones que pasaban por él	Los paquetes siguientes se desvían por otro lado
Ejemplos	Red telefónica, X.25, Frame Relay, ATM, MPLS	**IP**

IP eligió datagramas porque es más simple y más robusto: los routers no guardan nada de cada conversación. Consecuencia directa: los paquetes de una misma comunicación pueden llegar **desordenados**, **duplicados** o **no llegar**.

IP en concreto

- Definido en la **RFC 791** (1981), Internet Standard **STD 5**. Una RFC es el documento oficial que define un protocolo de Internet; la clase las cita para que sepas dónde está la definición "de verdad".

- **Best-effort (mejor esfuerzo):** IP intenta entregar, pero **no promete nada**. No hay control de errores sobre los datos, ni secuenciamiento (orden), ni confiabilidad (reenvío de lo perdido). Si un router se queda sin memoria, descarta el paquete y no avisa necesariamente a nadie.
- **No orientado a conexión:** cada paquete es independiente.
- **PDU:** datagrama o paquete (se usan como sinónimos).

¿Y entonces quién hace que la web funcione sin perder datos? **TCP**, en los extremos. Es el principio "end-to-end" de Internet: la red es tonta y rápida, la inteligencia está en los hosts.

Las funciones de IP que nombra la clase

1. **Direccionamiento:** darle a cada interfaz un identificador global (sección 4).
2. **Ruteo / forwarding:** decidir hacia dónde va cada paquete y mandarlo (sección 8).
3. **Multiplexación / demultiplexación de protocolos superiores:** varios protocolos (TCP, UDP, ICMP) usan IP al mismo tiempo. El emisor marca en el campo *Protocol* de quién es el contenido (multiplexar); el receptor lee ese campo para entregarlo al protocolo correcto (demultiplexar).
4. **Fragmentación:** partir un paquete que no entra en un enlace (sección 9).

3. El datagrama IP, campo por campo

Un datagrama IP es una **cabecera de 20 a 60 bytes** seguida de los **datos** (payload) del protocolo que transporta. La cabecera se describe en "palabras" de 32 bits (4 bytes), que es como la leen los routers; por eso la clase habla de "primera palabra", "segunda palabra", etc.

La cabecera, palabra por palabra

Palabra (bytes)	Bits 0–3	Bits 4–7	Bits 8–15	Bits 16–18	Bits 19–31
1 (bytes 0–3)	Version	H Len	TOS	Total Length (16 bits, ocupa 16–31)	
2 (bytes 4–7)	Identification (16 bits, ocupa 0–15)			Flags (3)	Fragment Offset (13)
3 (bytes 8–11)	TTL (8 bits, ocupa 0–7)		Protocol	Header Checksum (16 bits, ocupa 16–31)	
4 (bytes 12–15)	Source IP Address (32 bits)				

Palabra (bytes)	Bits 0–3	Bits 4–7	Bits 8–15	Bits 16–18	Bits 19–31
5 (bytes 16–19)	Destination IP Address (32 bits)				
6 a 15 (opcional)	Options + Padding (0 a 40 bytes)				

Las 5 palabras fijas dan los **20 bytes mínimos**. Con opciones se llega hasta 15 palabras = **60 bytes** (el límite sale del campo H Len, que tiene 4 bits).

Primera palabra

- **Version (4 bits):** vale 4 en IPv4. Es lo primero que lee el receptor para saber cómo interpretar el resto.
- **H Len, Header Length (4 bits):** largo de la cabecera **en palabras de 32 bits**, no en bytes. Como 4 bits van de 0 a 15, el máximo es 15 × 4 = 60 bytes. Sin opciones vale 5 (5 × 4 = 20 bytes). Hace falta porque las opciones son de largo variable y el receptor necesita saber dónde termina la cabecera y empiezan los datos.
- **TOS, Type of Service (8 bits):** pensado para pedir prioridad, bajo retardo, alto throughput o alta confiabilidad. En la práctica hoy se reinterpreta como DSCP/ECN (calidad de servicio), pero para la materia alcanza con saber que expresa "cómo quiero que me traten".
- **Total Length (16 bits):** largo total del datagrama **en bytes**, cabecera más datos. Con 16 bits el máximo es $2^{16} - 1$ = **65.535 bytes**. Datos = Total Length − H Len × 4.

Segunda palabra: fragmentación

Estos tres campos solo se usan cuando un datagrama se parte en pedazos (sección 9).

- **Identification (16 bits):** número que pone el emisor a cada datagrama. Todos los fragmentos de un mismo datagrama llevan **el mismo** Identification; así el destino sabe qué pedazos van juntos.
- **Flags (3 bits):**
  - **X:** reservado, siempre 0.
  - **D, Don't Fragment:** 0 = se puede fragmentar; 1 = prohibido fragmentar. Si un router necesitaría fragmentar un paquete con D = 1, lo descarta y avisa con ICMP. Esto se usa para descubrir el Path MTU.
  - **M, More Fragments:** 1 = vienen más fragmentos después de este; 0 = este es el último (o el datagrama no está fragmentado).

- **Fragment Offset (13 bits):** dónde va este pedazo dentro de los datos del datagrama original, medido **en unidades de 8 bytes**. El primer fragmento tiene offset 0. ¿Por qué de a 8? Porque con 13 bits solo se puede contar hasta 8191; multiplicado por 8 se llega a 65.528, que cubre todo el tamaño posible de un datagrama. El precio es que todos los fragmentos menos el último deben tener datos múltiplos de 8.

Tercera palabra

- **TTL, Time To Live (8 bits):** cada router que reenvía el paquete le resta 1. Si llega a 0, el router lo descarta y manda un ICMP *Time Exceeded*. Existe para que un paquete atrapado en un **loop de ruteo** (A manda a B, B manda a A…) no circule para siempre. Traceroute aprovecha esto: manda paquetes con TTL 1, 2, 3… y cada router que los descarta se "delata" con su ICMP.
- **Protocol (8 bits):** qué hay en los datos. **1 = ICMP, 6 = TCP, 17 = UDP.** Es el campo que permite el demultiplexado.
- **Header Checksum (16 bits):** control de errores **solo de la cabecera**, no de los datos. Cada router lo verifica y lo **recalcula**, porque al bajar el TTL la cabecera cambió. Si no coincide, el paquete se descarta. Los datos los protege la capa de transporte (TCP y UDP tienen su propio checksum).

Cuarta y quinta palabra

- **Source IP Address (32 bits):** IP de la interfaz que originó el paquete.
- **Destination IP Address (32 bits):** IP del destino final. Ojo: **no es la del próximo router**. Las IP origen y destino se mantienen de punta a punta (salvo NAT); lo que cambia en cada salto son las MAC de la trama. Esto es central para entender ARP.

Opciones, padding y datos

- **Options (variable):** usadas raramente hoy. Ejemplos de la clase: **Record Route** (cada router anota su IP en el paquete), **Loose Source Routing** (el emisor indica algunos routers por los que el paquete debe pasar) y **Strict Source Routing** (el emisor indica el camino exacto). Las de source routing suelen estar bloqueadas por seguridad.
- **Padding:** ceros de relleno para que la cabecera termine justo en un múltiplo de 32 bits, porque H Len cuenta palabras enteras.
- **Data/Payload:** el segmento TCP, datagrama UDP o mensaje ICMP. Siempre una cantidad entera de bytes. Cabecera + datos ⩽ 65.535 bytes.

4. Direcciones IP

Una dirección IP identifica **una interfaz** (un punto de conexión a una red), no una computadora. Tiene 32 bits divididos en una parte de **red** y una parte de **host**, y esa división es la que hace posible rutear.

Qué identifica y por qué eso importa

- Una **interfaz** es la conexión entre un equipo y un enlace (en Linux: eth0, eth1, wlan0…). Un router conecta varias redes, así que tiene **una IP por interfaz**. Un host **multi-homed** (con varias placas de red) también tiene varias IP.
- **Dirección lógica vs física:** la MAC (48 bits) viene grabada en la placa y solo sirve dentro del enlace; por eso se la llama "física". La IP la asigna un administrador o DHCP y depende de **en qué red estás**; por eso es "lógica". Si llevás tu notebook a otra red, la MAC es la misma pero la IP cambia.
- **Por qué jerárquica:** con $2^{32} \approx 4.300$ millones de direcciones, ningún router podría tener una entrada por host. Al agruparlas en redes, un router solo necesita saber "para la red X, mandá por acá". Es la misma idea que un código postal. El costo es que se pierden direcciones (las de red y broadcast de cada red, y los bloques que sobran), y por eso el número real de direcciones usables es bastante menor que 2^{32}.
- **Notación decimal con puntos:** cada uno de los 4 bytes se escribe en decimal (0 a 255). Ejemplo: 163.10.45.77 = 10100011.00001010.00101101.01001101.

Quién asigna las direcciones

- **Globales (públicas):** únicas en toda Internet, así que tiene que haber una autoridad central. Antes era InterNIC; hoy la **IANA** (que funciona bajo **ICANN**) reparte bloques grandes a los **RIR** (Regional Internet Registries), y estos a ISPs y organizaciones. Para América Latina y parte del Caribe el RIR es **LACNIC**. Otros: ARIN (Norteamérica), RIPE NCC (Europa), APNIC (Asia-Pacífico), AFRINIC (África).
- **Privadas:** cualquiera las puede usar dentro de su red, sin pedir permiso, porque **no salen a Internet** (más abajo).

Direccionamiento por clases (classful, RFC 790, 1981)

Originalmente la división red/host era fija y se deducía de los **primeros bits** de la dirección. Eso define las clases:

Clase	Primeros bits	Rango del primer octeto	Bytes de red / host	Máscara por defecto	Redes posibles	Hosts por red	Uso
A	0	1 a 126 (127 reservado)	1 / 3	255.0.0.0 (/8)	126	$2^{24} - 2 \approx$ 16 M	Redes enormes

Clase	Primeros bits	Rango del primer octeto	Bytes de red / host	Máscara por defecto	Redes posibles	Hosts por red	Uso
B	10	128 a 191	2 / 2	255.255.0.0 (/16)	2^{14} = 16.384	$2^{16} - 2$ = 65.534	Redes medianas
C	110	192 a 223	3 / 1	255.255.255.0 (/24)	$2^{21} \approx 2$ M	$2^{8} - 2$ = 254	Redes chicas
D	1110	224 a 239	—	—	—	—	Multicast
E	1111	240 a 255	—	—	—	—	Experimental

Cómo deducir el rango sin memorizarlo: clase A empieza con 0, así que el primer octeto va de 00000000 a 01111111 (0 a 127). Clase B empieza con 10: de 10000000 a 10111111 (128 a 191). Clase C con 110: de 11000000 a 11011111 (192 a 223). Y así.

El problema de las clases: el salto entre tamaños es brutal (254, 65.534 o 16 millones de hosts). Una empresa con 300 hosts necesitaba una clase B y desperdiciaba más de 65.000 direcciones. Subnetting, VLSM y CIDR (secciones 5 a 7) son la historia de cómo se arregló eso. Hoy las clases ya no se usan para rutear, pero el vocabulario sigue ("una clase C" = un /24).

Tipos de direcciones según a quién van

- **Unicast:** a una sola interfaz. Es lo normal. Ej.: 172.16.4.21.
- **Broadcast:** a todos los hosts de una red. La parte de host va toda en 1. Ej.: 172.16.255.255 es el broadcast de la red 172.16.0.0/16.
- **Multicast:** a un **grupo** de hosts que se suscribieron, en una o varias redes. Son las de clase D: 224.0.0.0 a 239.255.255.255. Sirve para mandar lo mismo a muchos sin repetirlo (streaming, protocolos de ruteo como OSPF).
- **Anycast:** no es un rango aparte sino una técnica: **la misma dirección unicast** se configura en varios servidores en distintos lugares, y el ruteo hace que cada paquete llegue al más "cercano". Así funcionan los servidores raíz de DNS.

Direcciones especiales

Dirección	Qué es	Para qué sirve
127.0.0.0/8 (normalmente 127.0.0.1)	**Loopback** o localhost	Hablarle a la propia máquina. Lo que se manda acá nunca sale a la red. Es toda una clase A reservada, aunque se use casi siempre la .1

Dirección	Qué es	Para qué sirve
Parte de host toda en 0 (ej. 172.16.0.0, 192.168.10.0)	**Dirección de red**	Nombra a la red entera. Es la que aparece en las tablas de ruteo. No se asigna a ninguna interfaz
Parte de host toda en 1 (ej. 172.16.255.255, 192.168.1.255)	**Directed broadcast**	Broadcast a todos los hosts de **esa** red, que puede ser remota. No se asigna a ninguna interfaz
255.255.255.255	**Limited broadcast**	Broadcast a todos los de **mi** red local. Los routers nunca lo reenvían. Lo usa DHCP cuando el host todavía no sabe ni en qué red está
0.0.0.0	**Este host**	IP origen de un host que todavía no tiene dirección (DHCP Discover). En una tabla de ruteo, 0.0.0.0/0 significa "cualquier destino" (ruta default)

De acá sale una regla que se usa todo el tiempo: **en cada red hay dos direcciones que no se pueden asignar** (la de red y la de broadcast). Por eso los hosts útiles son $2^n - 2$.

Direcciones privadas (RFC 1918)

Son rangos reservados para usar **dentro** de una organización (intranets). Cualquiera las puede repetir, así que no son únicas y los routers de acceso a Internet las **filtran** (no las dejan salir ni entrar). Para que un host con IP privada navegue, hace falta **NAT** (clase 5), que la traduce a una pública.

Bloque	Equivale a	En notación CIDR
10.0.0.0 a 10.255.255.255	1 red clase A	10.0.0.0/8
172.16.0.0 a 172.31.255.255	16 redes clase B	172.16.0.0/12
192.168.0.0 a 192.168.255.255	256 redes clase C	192.168.0.0/16

Ojo con una trampa frecuente: 172.32.0.0 **no** es privada; el rango privado de 172 termina en 172.31.

5. Subnetting fijo

Subnetting es **pedir prestados bits de la parte de host para crear subredes** dentro de una red. La máscara indica dónde quedó el corte entre red y host. En subnetting fijo todas las subredes tienen la misma máscara.

El problema que resuelve

En el ejemplo de la clase hay 4 redes físicas (cuatro LANs separadas por routers). **Cada red física necesita su propia red IP**, porque un router solo sabe separar tráfico entre redes distintas. Con clases, eso significaba gastar 4 clases C enteras (193.168.1.0, .2.0, .3.0 y .4.0), aunque el enlace entre dos routers solo use 2 direcciones. Desperdicio enorme.

Subnetting (RFC 950, 1985) agrega un nivel a la jerarquía: **red → subred → host**. Desde afuera la red sigue viendose como un solo bloque; adentro se reparte.

La máscara de subred

La máscara es un número de 32 bits con **1 en todos los bits de red (incluidos los de subred)** y **0 en los de host**. Los unos siempre van juntos a la izquierda.

Se puede escribir de tres formas, todas equivalentes:

Decimal	Hexadecimal	Prefijo	Bits de host
255.0.0.0	0xFF000000	/8	24
255.224.0.0	0xFFE00000	/11	21
255.255.0.0	0xFFFF0000	/16	16
255.255.255.0	0xFFFFFF00	/24	8
255.255.255.192	0xFFFFFFC0	/26	6
255.255.255.224	0xFFFFFFE0	/27	5
255.255.255.252	0xFFFFFFFC	/30	2

(La diapositiva escribe "255.255.255.255.252" para el /30: es un error de tipeo, sobra un 255.)

Truco para no memorizar: los valores posibles de un octeto de máscara son 128, 192, 224, 240, 248, 252, 254, 255 (ir sumando 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1).

Máscaras por defecto de cada clase: A = /8, B = /16, C = /24.

Cómo saber a qué subred pertenece una IP: el AND lógico

El router (o el host) hace un **AND bit a bit** entre la IP y la máscara. Como la máscara tiene 1 en la parte de red, el AND deja esos bits intactos; como tiene 0 en la parte de host, los pone en 0. El resultado es la dirección de la subred.

Ejemplo: 192.168.10.140 con máscara /27 (255.255.255.224). Solo importa el último octeto:

```
140 = 1000 1100
224 = 1110 0000
AND = 1000 0000 = 128
```

La IP pertenece a la subred 192.168.10.128/27. Su broadcast es la parte de host toda en 1: 1001 1111 = 159, o sea 192.168.10.159. Hosts asignables: .129 a .158.

Así es como un host decide si un destino está **en su misma red** (lo manda directo, haciendo ARP por el destino) o **en otra** (lo manda al default gateway). Esto vuelve en la clase de ARP.

Las fórmulas, explicadas

Llamá **n** a los bits que tomás para subredes y **h** a los bits que quedan para hosts (h = 32 − prefijo).

- **Cantidad de subredes = 2^n.** Con n bits podés escribir 2^n combinaciones distintas.
- **Direcciones por subred = 2^h.** En la notación de la clase: 2^(32 − (m + n)), donde m son los bits de red de la clase original. Es lo mismo: lo que queda para host.
- **Hosts asignables por subred = $2^h - 2$**, porque la primera dirección es la de subred y la última la de broadcast.
- **Cuántos bits necesito para k subredes:** el menor n tal que $2^n \geq k$.
- **Cuántos bits de host necesito para H hosts:** el menor h tal que $2^h - 2 \geq H$.

Ejemplo de la clase: una clase B (m = 16) partida en 256 subredes (n = 8). Subredes: 2^8 = 256. Direcciones por subred: 2^(32 − 24) = 2^8 = 256. Hosts asignables: 256 − 2 = 254. Es decir, "usar una clase B como 256 clases C". (La diapositiva escribe la última fórmula como "$2^n - 2$": en este ejemplo da igual porque n y h valen 8, pero lo correcto en general es restarle 2 a la cantidad de direcciones **de host**.)

También hay una imprecisión en la diapositiva: dice "si tomamos la subred 172.16.4.21/24". 172.16.4.21 es un **host** dentro de la subred 172.16.4.0/24, cuyo broadcast es 172.16.4.255, y la red classful de la que sale es 172.16.0.0/16.

El ejemplo de la clase: 4 redes de 25 hosts

Partiendo de una clase C (193.168.4.0/24) y necesitando 4 subredes, hace falta n = 2 (2^2 = 4). El prefijo pasa de /24 a /26 y quedan h = 6 bits: 2^6 = 64 direcciones, 62 hosts. Alcanza para 25 hosts.

Red	Bits de subred	Dirección de subred	Rango asignable	Broadcast
A	00	193.168.4.0/26	.1 a .62	.63
B	01	193.168.4.64/26	.65 a .126	.127
C	10	193.168.4.128/26	.129 a .190	.191
D	11	193.168.4.192/26	.193 a .254	.255

Las subredes avanzan de a 64 porque 64 es el tamaño del bloque (2^h). Ese "salto" es la forma rápida de listarlas: 256 − 192 = 64.

Pregunta de la clase: **¿cuántos bits para 7 subredes?** 2^2 = 4 no alcanza, 2^3 = 8 sí: **3 bits** (sobra una).

La regla vieja de la "subred cero"

Al principio no se permitía usar ni la **primera** subred (la de bits de subred todos en 0) ni la **última** (todos en 1). ¿Por qué? En la primera, la dirección de subred (193.168.4.0) es igual a la de la red completa; en la última, el broadcast de la subred (193.168.4.255) es igual al broadcast de toda la red. Equipos viejos se confundían.

Eso desperdiciaba mucho (en el ejemplo, 2 de 4 subredes: 50 %). Las RFC posteriores lo permitieron y hoy es lo estándar. **La cátedra usa todas las subredes**, incluida la primera y la última. (La diapositiva escribe "Red D: 192.168.4.192": debería decir 193.168.4.192.)

Nota: las direcciones 193.168.x.x de los ejemplos son **públicas** (asignadas por RIPE a otras organizaciones); la clase las usa solo como ejemplo. No confundir con 192.168.x.x, que es privada.

6. VLSM: subredes de distinto tamaño

VLSM (Variable Length Subnet Mask, RFC 1009, 1987) permite que **cada subred tenga su propia máscara**, así cada una recibe un bloque del tamaño que necesita y no más.

Por qué el subnetting fijo no alcanza

Con los requisitos de la clase (Red A: 100 hosts, Red B: 60, Red C: 25, Red D: 25) y una sola clase C:

- Si dimensionás por la red más grande (100 hosts necesita 7 bits de host, /25), solo te queda 1 bit para subredes: **2 subredes**. No alcanza para 4.
- Si dimensionás por la cantidad de redes (4 redes, 2 bits, /26), cada subred tiene 62 hosts: **no entra la red de 100**.

El subnetting fijo obliga a que todas las subredes sean tan grandes como la más grande. VLSM rompe esa restricción.

La idea de fondo: partir bloques a la mitad, recursivamente

Un bloque /24 se puede partir en dos /25. Cada /25 se puede partir en dos /26, cada /26 en dos /27, y así. VLSM consiste en **usar algunas mitades y seguir partiendo otras**. Pensalo como un árbol binario: cada bit extra de máscara divide el bloque en dos.

La única regla dura: **un bloque de tamaño 2^h tiene que empezar en un múltiplo de 2^h**. Un /27 (32 direcciones) puede empezar en .0, .32, .64, .96… pero nunca en .40. Partir siempre a la mitad garantiza que se cumpla.

El método, paso a paso

1. **Ordenar las redes de mayor a menor** cantidad de hosts. Se empieza por la grande porque necesita un bloque alineado grande; si se asignan primero las chicas, pueden "romper" el espacio y que después no entre la grande.
2. Para cada red, calcular **los bits de host**: el menor h con $2^h - 2 \geqslant$ hosts. El prefijo es 32 − h.
3. Asignarle **el primer bloque libre** de ese tamaño, partiendo a la mitad lo que haga falta.
4. Lo que sobra queda libre para las siguientes redes (o para crecer).
5. No olvidar los **enlaces entre routers**: cada uno es una red con 2 hosts, así que lleva un **/30** (4 direcciones: red, 2 interfaces, broadcast).

El ejemplo de la diapositiva

Red	Hosts pedidos	Bits de host	Prefijo	Direcciones (asignables)	Bloque asignado
A	100	7	/25	128 (126)	193.168.4.0/25 (.0 a .127)
B	60	6	/26	64 (62)	193.168.4.128/26 (.128 a .191)
C	25	5	/27	32 (30)	193.168.4.192/27 (.192 a .223)
D	25	5	/27	32 (30)	193.168.4.224/27 (.224 a .255)

Se usó el /24 completo, sin desperdicio de bloques, y cada red tiene lo justo.

Errores típicos

- **Olvidarse del − 2.** 30 hosts entran justo en un /27 (32 − 2 = 30). 31 hosts ya no: necesitan un /26.
- **Contar la interfaz del router.** El router también ocupa una IP de cada LAN. Si el enunciado dice "25 hosts" sin aclarar, se suele asumir que ya incluye al router; si piden rigor, sumá 1.
- **Bloques desalineados.** Si te queda un /27 libre en .32, no podés meter ahí un /26.
- En los ejemplos resueltos de la cátedra ("Ejemplo VLSM") aparece "se necesitarán 2^7 − 2 = 126 **bits** para hosts": son 126 **hosts**, que requieren 7 bits.

VLSM con enlaces punto a punto (el ejercicio de ruteo)

El ejemplo "Ruteo_Explicación" de la cátedra aplica VLSM a 192.168.10.0/24 con Red A = 100, Red B = 13, Red C = 24, Red D = 30 y dos enlaces entre routers. Queda:

Red	Prefijo	Bloque
Red A (100 hosts)	/25	192.168.10.0/25
Red C (24 hosts)	/27	192.168.10.128/27
Red D (30 hosts)	/27	192.168.10.160/27
Red B (13 hosts)	/28	192.168.10.192/28
Enlace Rtr-A ↔ Rtr-B	/30	192.168.10.208/30
Enlace Rtr-B ↔ Rtr-C	/30	192.168.10.212/30
Libres	/29 y /27	192.168.10.216/29 y 192.168.10.224/27

Es la topología que se usa en la sección de ruteo.

7. CIDR y supernetting

CIDR (Classless Inter-Domain Routing, 1993) **elimina las clases**: desde entonces la máscara no se deduce de los primeros bits, sino que **siempre se indica explícitamente** con el prefijo (/n). Eso permite tanto partir bloques (subnetting/VLSM) como **juntarlos** (supernetting o agregación).

El problema de 1992 (RFC 1338)

Internet crecía y aparecieron dos problemas a la vez:

1. **Agotamiento de direcciones:** las clases A y B ya estaban asignadas en un 50 %. La C apenas en un 2 %, pero un /24 es demasiado chico para casi cualquier organización mediana.
2. **Explosión de las tablas de ruteo:** la solución obvia era darle a cada organización varias clases C. Pero había 2^{21} (unos 2 millones) de clases C posibles, y **cada una sería una entrada aparte** en las tablas de los routers del núcleo de Internet. Los routers no dan abasto (memoria y tiempo de búsqueda). Hoy la tabla global ronda el millón de rutas; el link del blog de APNIC en la diapositiva habla justamente de eso.

La RFC 1338 ya advertía que las direcciones IPv4 se iban a agotar, pero no proponía solución de fondo para eso (la solución de fondo terminó siendo IPv6).

Classful vs classless

- **Classful (hasta 1993):** el router mira el primer octeto, deduce la clase y asume la máscara (/8, /16 o /24). Solo se puede subnetear **hacia adentro** de una clase.
- **Classless (CIDR):** la máscara viaja siempre junto a la red (en las tablas y en los protocolos de ruteo). Un bloque puede ser /22, /13, /29, lo que sea. Las clases desaparecen como concepto de ruteo.

RFCs que lo definen: 1338, 1517, 1518, 1519.

Supernetting / sumarización: el beneficio principal

Supernetting es lo inverso del subnetting: **varias redes contiguas se anuncian como una sola, con un prefijo más corto** (menos bits de red). El beneficio es **achicar las tablas de ruteo**: en vez de 4 entradas, 1.

Ejemplo de la diapositiva: 193.168.0.0/24, 193.168.1.0/24, 193.168.2.0/24 y 193.168.3.0/24 se agregan en **193.168.0.0/22**. (La diapositiva escribe la cuarta como 192.163.3.0: es un error de tipeo, debe ser 193.168.3.0.)

Cómo se verifica, mirando el tercer octeto en binario:

```
0 = 000000|00
1 = 000000|01
2 = 000000|10
3 = 000000|11
```

Los primeros 6 bits del tercer octeto son iguales en las cuatro. Prefijo común: 8 + 8 + 6 = **22 bits**. Las cuatro redes juntas son exactamente el bloque 193.168.0.0/22, ni más ni menos.

Las condiciones para poder sumarizar

1. Las redes tienen que ser **contiguas** y la cantidad tiene que ser **potencia de 2** (2, 4, 8…).
2. El bloque resultante tiene que estar **alineado**: la primera red debe ser múltiplo del tamaño del bloque. 193.168.1.0 a 193.168.4.0 **no** se puede sumarizar en un /22, aunque sean 4 contiguas, porque 1 no es múltiplo de 4.
3. En una tabla de ruteo, además, todas tienen que salir **por el mismo next hop e interfaz** (se ve en la sección siguiente).

Si no se cumple la 2, sumarizar igual "funcionaría" pero estaría abarcando redes que no te pertenecen: un error grave.

8. Ruteo

Cada equipo decide **solo el próximo salto**, nunca el camino completo. Lo decide mirando su **tabla de ruteo**: busca la entrada que corresponde a la IP destino y manda el paquete al vecino que esa entrada indica. El camino completo es la suma de muchas decisiones locales.

Hosts, routers y hosts multi-homed

- **Host:** envía sus propios paquetes consultando su tabla, pero **no reenvía** paquetes que le llegan y no son para él (los descarta).
- **Router (o gateway):** nodo intermedio con varias interfaces. Su trabajo es **reenviar** lo que recibe hacia otra interfaz, según su tabla. Los paquetes dirigidos a él mismo (por ejemplo, un ping a su IP) los procesa como cualquier host.
- **Host multi-homed:** tiene varias interfaces (y varias IP) pero **no rutea** entre ellas.

La diferencia entre un router y un host no es el hardware sino una configuración: **el forwarding**. En los routers viene habilitado por defecto; en los hosts, deshabilitado. Un Linux comun se convierte en router activando `net.ipv4.ip_forward = 1`.

Ruteo vs forwarding

La clase distingue dos palabras que en el habla diaria se mezclan:

	Ruteo (routing)	Forwarding (despacho)
Qué es	Elegir el próximo salto y la interfaz de salida; construir y mantener la tabla	Mover efectivamente un paquete de la interfaz de entrada a la de salida

	Ruteo (routing)	Forwarding (despacho)
Quién lo hace	Routers **y hosts** (un host también elige por dónde mandar)	**Solo routers**
Plano	De **control**	De **datos**
Lo alimentan	Los **protocolos de ruteo** (routing protocols: RIP, OSPF, BGP) o rutas estáticas cargadas a mano	Los **protocolos ruteados** (routed protocols: IP es el que se rutea)
Costo	Ocasional (cuando cambia la red)	Por **cada** paquete: es lo más intensivo

Analogía rápida: el ruteo es armar el mapa de carteles en cada cruce; el forwarding es cada auto doblando según el cartel.

Estructura de una tabla de ruteo

Campo	Qué significa
Red destino	La dirección de red (no de host) a la que aplica la entrada
Máscara o prefijo	Cuántos bits de la IP destino hay que comparar
Next hop (próximo salto)	IP del router vecino al que hay que entregarle el paquete. Si la red está **directamente conectada**, no hay next hop: se entrega al destino final
Interfaz de salida	Por qué placa sale el paquete
Métrica	Cuán "lejos" está la red (en este ejemplo, cantidad de saltos). Sirve para elegir entre dos rutas a la misma red

La tabla de un host es mucho más simple: su propia red (directamente conectada) y una ruta default hacia su router.

Cómo se busca en la tabla (lo que la clase no dice explícitamente)

Para cada entrada, el router hace **IP destino AND máscara** y compara el resultado con la red destino. Si coincide, la entrada "matchea". Si matchean varias, gana la de **prefijo más largo** (longest prefix match), porque es la más específica. La ruta default (0.0.0.0/0) matchea con todo, pero como tiene prefijo 0 es siempre la última opción. Esto es lo que hace que la sumarización y la ruta default funcionen sin pisar a las rutas específicas.

Comandos de la clase (Linux, iproute2)

```
# Ruta a una red puntual
Rtr-A# ip route add 192.168.10.160/27 via 192.168.10.210 dev eth0

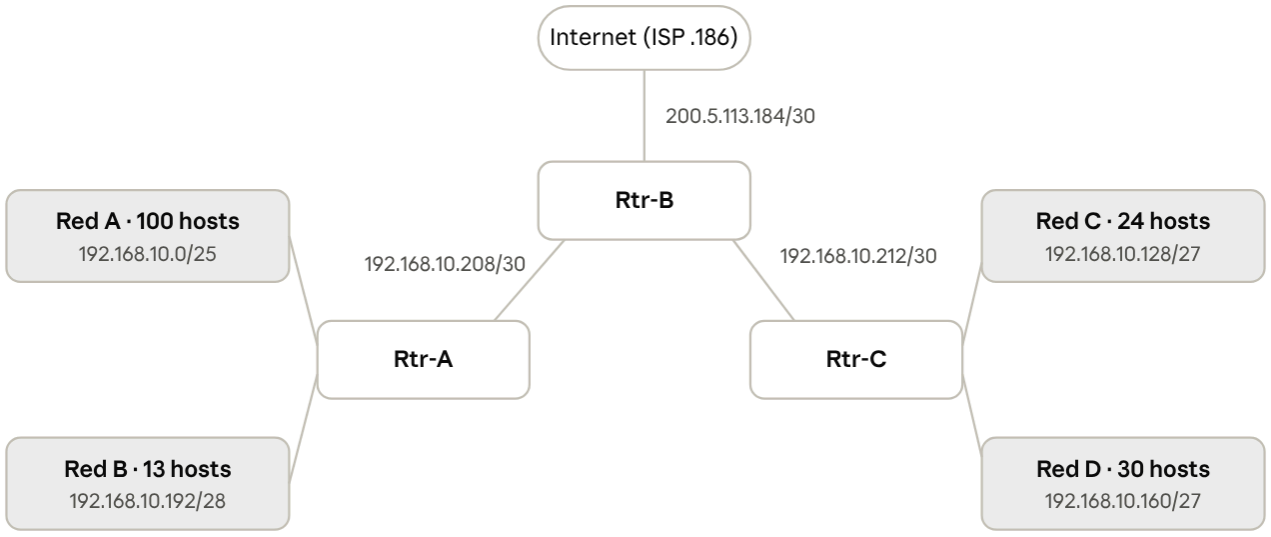
# Ruta default
Rtr-A# ip route add default via 192.168.10.210 dev eth0
```

`via` es el next hop y `dev` la interfaz de salida. Las redes directamente conectadas no se cargan a mano: aparecen solas cuando se le asigna una IP a la interfaz.

La topología del ejemplo

Tres routers en cadena. Rtr-A tiene las LAN A y B; Rtr-B está en el medio y es la salida a Internet; Rtr-C tiene las LAN C y D.

Tres routers en cadena: Rtr-B en el medio es la salida a Internet



topología del ejemplo de ruteo · 3 routers, 4 LANs

Las IP de cada interfaz están en la tabla de abajo.

Red	Bloque	Conectada a
Red A (100 hosts)	192.168.10.0/25	Rtr-A eth2 (.1)
Red B (13 hosts)	192.168.10.192/28	Rtr-A eth1 (.193)
Enlace A–B	192.168.10.208/30	Rtr-A eth0 (.209) ↔ Rtr-B eth0 (.210)
Enlace B–C	192.168.10.212/30	Rtr-B eth2 (.213) ↔ Rtr-C eth0 (.214)
Red C (24 hosts)	192.168.10.128/27	Rtr-C eth1 (.129)

Red	Bloque	Conectada a
Red D (30 hosts)	192.168.10.160/27	Rtr-C eth2 (.161)
Enlace a Internet	200.5.113.184/30	Rtr-B eth1 (.185) ↔ ISP (.186)

Tabla completa de Rtr-A

Red destino	Prefijo	Next hop	Interfaz	Métrica	Por qué
192.168.10.0	/25	—	eth2	0	Red A, directamente conectada
192.168.10.192	/28	—	eth1	0	Red B, directamente conectada
192.168.10.208	/30	—	eth0	0	Enlace con Rtr-B, directamente conectado
192.168.10.212	/30	192.168.10.210	eth0	1	Enlace B–C: hay que pasar por Rtr-B
192.168.10.128	/27	192.168.10.210	eth0	2	Red C: Rtr-B y después Rtr-C
192.168.10.160	/27	192.168.10.210	eth0	2	Red D: igual que Red C
0.0.0.0	/0	192.168.10.210	eth0	—	Todo lo demás (Internet) va a Rtr-B

Atención al next hop: Rtr-A pone **192.168.10.210** (la interfaz de Rtr-B que tiene enfrente), no la IP de Rtr-C, aunque el paquete termine yendo a Rtr-C. **El next hop siempre es un vecino directamente alcanzable.**

Achicar la tabla, paso 1: sumarizar Red C y Red D

Red C (.128/27) y Red D (.160/27) son contiguas, son 2 (potencia de 2), están alineadas (.128 es múltiplo de 64) y salen **por el mismo next hop y la misma interfaz**. Se pueden reemplazar por **192.168.10.128/26**:

```
128 = 10|000000
160 = 10|100000
```

Primeros 2 bits del último octeto iguales: 24 + 2 = /26.

Paso 2: dejar solo la ruta default

Todas las redes no conectadas a Rtr-A (enlace B–C, Red C, Red D e Internet) salen por el mismo lugar: 192.168.10.210 por eth0. Entonces **la ruta default ya las cubre** y la tabla queda en 4 entradas:

Red destino	Prefijo	Next hop	Interfaz
192.168.10.0	/25	—	eth2

Red destino	Prefijo	Next hop	Interfaz
192.168.10.192	/28	—	eth1
192.168.10.208	/30	—	eth0
0.0.0.0	/0	192.168.10.210	eth0

Importante: **solo puede haber una ruta default por tabla**. Si hubiera dos, el router no sabría cuál usar para "todo lo demás".

Las tablas de los otros routers

Rtr-B no puede reemplazar todo por la default, porque su default apunta a Internet y las redes internas están hacia dos lados distintos:

Red destino	Prefijo	Next hop	Interfaz
192.168.10.208	/30	—	eth0
192.168.10.212	/30	—	eth2
200.5.113.184	/30	—	eth1
192.168.10.0	/25	192.168.10.209	eth0
192.168.10.192	/28	192.168.10.209	eth0
192.168.10.128	/26	192.168.10.214	eth2
0.0.0.0	/0	200.5.113.186	eth1

Acá Red C y Red D sí se sumarizan en .128/26. Red A y Red B **no** se pueden sumarizar entre sí: aunque compartan next hop, no forman un bloque contiguo alineado.

Rtr-C (la "tarea" del ejemplo resuelto):

Red destino	Prefijo	Next hop	Interfaz
192.168.10.212	/30	—	eth0
192.168.10.128	/27	—	eth1
192.168.10.160	/27	—	eth2
0.0.0.0	/0	192.168.10.213	eth0

Rtr-C **no puede** sumarizar Red C y Red D, porque las tiene directamente conectadas en **interfaces distintas**: necesita saber que .140 va por eth1 y .165 por eth2. A Rtr-B no le importa esa diferencia porque, para él, las dos van al mismo vecino.

Regla para sumarizar en una tabla de ruteo

Además de las condiciones de CIDR (contiguas, potencia de 2, alineadas), las redes tienen que compartir **next hop e interfaz de salida**. Si comparten la interfaz pero no el next hop (dos routers distintos colgados de la misma LAN), no se sumariza: van entradas separadas.

9. Fragmentación

Cuando un datagrama es más grande que lo que el próximo enlace puede transportar (su **MTU**), el router lo **parte en fragmentos**, cada uno con su propia cabecera IP, y **solo el destino final** los vuelve a juntar.

Por qué hace falta: el MTU

El **MTU (Maximum Transmission Unit)** es el tamaño máximo de datos que una tecnología de enlace acepta dentro de una trama. Ethernet: **1500 bytes**; FDDI: 4500. Es un límite de la capa de enlace, no de IP. Como IP pasa por enlaces de distinta tecnología, un paquete que salió bien de una red con MTU 4500 puede no entrar en la siguiente, de 1500. El MTU mínimo que IPv4 exige que todo enlace soporte es 68 bytes (RFC 791); en IPv6 es 1280.

Ojo: el MTU cuenta el **datagrama IP completo** (cabecera + datos), no la trama Ethernet con sus propios encabezados.

Las reglas

1. Cada fragmento es un **datagrama IP nuevo**: se copia la cabecera original y se modifican Total Length, Flags, Fragment Offset y el checksum. Algunas opciones se copian en todos los fragmentos y otras solo en el primero.
2. Todos los fragmentos llevan el **mismo Identification** que el original.
3. Los datos de cada fragmento, **salvo el último**, tienen que ser **múltiplo de 8 bytes**, porque el offset se mide de a 8.
4. **M = 1** en todos los fragmentos menos el último; **M = 0** en el último.
5. **Offset** = posición de los datos del fragmento dentro de los datos originales, dividida por 8.
6. Si el datagrama tiene **D = 1**, no se fragmenta: se descarta y se avisa con ICMP ("Fragmentation needed").

Receta para calcular

1. Datos a enviar = Total Length original − cabecera (normalmente 20).
2. Datos máximos por fragmento = MTU − 20, **redondeado hacia abajo al múltiplo de 8**.
3. Cantidad de fragmentos = datos a enviar / datos por fragmento, redondeado hacia arriba.
4. El último fragmento lleva lo que sobra.
5. Offset del fragmento k (empezando en 0) = k × (datos por fragmento) / 8.
6. Total Length de cada fragmento = sus datos + 20.

Ejemplo de la diapositiva: 2400 bytes por un enlace de MTU 1000

Datos a enviar: 2400 − 20 = 2380. Datos por fragmento: 1000 − 20 = 980, que no es múltiplo de 8; el múltiplo inferior es **976**. 2380 / 976 = 2,4, así que son **3 fragmentos**.

Fragmento	Datos (bytes)	Total Length	M	D	Offset	Bytes originales que cubre
Original	2380	2400	0	0	0	0 a 2379
1	976	996	1	0	0	0 a 975
2	976	996	1	0	122 (= 976 / 8)	976 a 1951
3	428	448	0	0	244 (= 1952 / 8)	1952 a 2379

Todos con el mismo Identification (0x4428 en la diapositiva). Chequeo: 976 + 976 + 428 = 2380. En el dibujo de la diapositiva los fragmentos aparecen en orden inverso (3, 2, 1) porque están dibujados como saliendo del router hacia la derecha.

Ejemplo de la práctica 3: 1000 bytes por un enlace de MTU 520

Datos a enviar: 1000 − 20 = 980. Datos por fragmento: 520 − 20 = 500, múltiplo de 8 inferior: **496**. 980 / 496 = 1,97, así que son **2 fragmentos**.

Fragmento	Datos	Total Length	M	Offset
1	496	516	1	0
2	484	504	0	62 (= 496 / 8)

Si hubiera un tercero, su offset sería (496 + 496) / 8 = 124. (El ejemplo resuelto de la cátedra escribe 128 en esa última diapositiva: es un error, 992 / 8 = 124.)

Reensamblado

- **Solo en el destino final.** Los routers intermedios nunca reensamblan, porque los fragmentos pueden tomar **caminos distintos** (cada uno es un datagrama independiente) y el router podría no ver todos.
- El destino agrupa por (IP origen, IP destino, Protocol, Identification), ordena por offset y sabe que terminó cuando tiene el fragmento con M = 0 y no hay huecos.
- Hay un **timeout**: si no llegan todos los fragmentos a tiempo, **se descarta el datagrama entero** y se manda un ICMP *Time Exceeded, Reassembly Timeout* (tipo 11, código 1). Perder un fragmento es perder todo el paquete; por eso fragmentar es caro.
- Un fragmento puede volver a fragmentarse si más adelante encuentra un MTU todavía menor. Los offsets se siguen calculando respecto del datagrama **original**, así que el destino no nota la diferencia.

Path MTU

El **Path MTU** es el MTU más chico de todos los enlaces entre origen y destino. Si el origen lo conoce, puede mandar paquetes de ese tamaño y evitar la fragmentación. Se descubre mandando paquetes con **D = 1**: si algún router no puede pasarlo, lo descarta y responde con un ICMP que indica el MTU del enlace; el origen achica y reintenta. Se profundiza en la clase de IPv6, donde los routers directamente no fragmentan.

10. ICMP: el canal de avisos de IP

ICMP (Internet Control Message Protocol, RFC 792, parte de STD 5) es el protocolo con el que los equipos **informan problemas y responden consultas** sobre IP. No arregla nada ni hace a IP confiable: solo da *feedback* para diagnosticar.

Por qué existe

IP es best-effort y descarta paquetes sin dar explicaciones. Sin ICMP, si un paquete muere por TTL, porque la red destino no existe o porque no entra en un enlace, el emisor no se enteraría nunca. ICMP es el "helper" que reporta eso. La clase insiste en que es **informativo**: no toma decisiones; las toma quien recibe el aviso.

Dónde va en la pila

ICMP **viaja dentro de IP** (campo Protocol = **1**), pero **no es un protocolo de transporte**: no lleva datos de aplicaciones ni usa puertos. Se lo considera parte de la capa de red, un compañero de IP que usa a IP como vehículo.

Formato

Todo mensaje ICMP empieza igual:

Campo	Tamaño	Qué indica
Type	8 bits	La categoría del mensaje (ej. 3 = destino inalcanzable)
Code	8 bits	El detalle dentro de esa categoría (ej. 3/3 = puerto inalcanzable)
Checksum	16 bits	Control de errores del mensaje ICMP
Resto	variable	Depende del tipo

Cuando ICMP reporta un error por un datagrama descartado, **incluye la cabecera IP de ese datagrama más al menos sus primeros 8 bytes de datos**. ¿Por qué 8? Porque en TCP y UDP los primeros 8 bytes contienen los **puertos** origen y destino: con eso el host que recibe el error sabe qué aplicación o conexión fue la afectada.

Cuándo NO se genera un ICMP

- Si lo que se descarta es **otro mensaje ICMP de error**. Si no, un error podría generar otro error, y otro, en cadena infinita.
- Por **fragmentos que no sean el primero**: solo el primero tiene la cabecera de transporte con los puertos, y además se evita mandar un error por cada fragmento.
- Por datagramas dirigidos a una **dirección multicast** (o broadcast): un solo paquete podría provocar una avalancha de respuestas de cientos de hosts.

Las dos categorías de mensajes

Mensajes de error (error-reporting): los manda un router o el destino cuando no pudo procesar un paquete.

Mensaje	Type / Code	Cuándo aparece
Destination Unreachable, Net Unreachable	3 / 0	El router no tiene ruta hacia la red destino
Destination Unreachable, Host Unreachable	3 / 1	Se llegó a la red pero el host no responde (por ejemplo, ARP sin respuesta)

Mensaje	Type / Code	Cuándo aparece
Destination Unreachable, Port Unreachable	3 / 3	Llegó al host pero no hay ningún proceso escuchando en ese puerto UDP. Lo genera el **destino**, no un router
Time Exceeded, In Transit	11 / 0	El TTL llegó a 0 en un router. Base de traceroute
Time Exceeded, Reassembly Timeout	11 / 1	El destino no recibió todos los fragmentos a tiempo

Mensajes de consulta (query): vienen **de a pares**, pregunta y respuesta, para obtener información de un equipo.

Mensaje	Type
Echo Request / Echo Reply (lo que usa **ping**)	8 / 0
Timestamp Request / Timestamp Reply	13 / 14

Ping manda un Echo Request; si el destino está vivo y nada lo bloquea, contesta con un Echo Reply, y ping mide cuánto tardó (RTT).

Aparte de los de la diapositiva, conviene conocer uno que aparece después: Destination Unreachable, **Fragmentation needed and DF set** (3 / 4), que es el que permite descubrir el Path MTU.

11. Preguntas para autoevaluarte

Si podés contestarlas razonando (no de memoria), entendiste la clase.

1. ¿Por qué IP no necesita establecer una conexión y qué consecuencias tiene eso para los paquetes de una misma comunicación?
2. ¿Por qué el checksum de IP se recalcula en cada router y por qué no protege los datos?
3. Si H Len vale 7, ¿cuántos bytes de opciones hay?
4. ¿Por qué el Fragment Offset se mide en unidades de 8 bytes y qué restricción impone eso?
5. ¿Por qué en cada red se pierden dos direcciones? ¿Cuáles?
6. Dada 10.1.77.200/21, ¿cuál es la dirección de red, el broadcast y el rango asignable?
7. Necesitás redes de 50, 28, 12 y 2 hosts a partir de 172.16.8.0/24. Asigná los bloques con VLSM.

8. ¿Por qué en VLSM se empieza por la red más grande?
9. ¿Se pueden sumarizar 10.0.4.0/24, 10.0.5.0/24, 10.0.6.0/24 y 10.0.7.0/24? ¿Y 10.0.5.0/24 a 10.0.8.0/24?
10. En la topología del ejemplo, ¿por qué Rtr-B puede sumarizar Red C y Red D pero Rtr-C no?
11. ¿Cuál es el next hop que Rtr-A usa para llegar a Red D, y por qué no es la IP de Rtr-C?
12. Un datagrama de 4000 bytes (cabecera de 20) atraviesa un enlace con MTU 1500. Calculá cantidad de fragmentos, Total Length, M y offset de cada uno.
13. ¿Por qué los routers intermedios no reensamblan los fragmentos?
14. ¿Por qué no se genera un ICMP de error cuando se descarta un ICMP de error?
15. ¿Qué diferencia hay entre 255.255.255.255 y 192.168.1.255 como destino?

Respuestas rápidas para chequear: (3) 7 × 4 − 20 = 8 bytes. (6) red 10.1.72.0, broadcast 10.1.79.255, asignables .72.1 a .79.254. (12) 3 fragmentos de datos 1480, 1480 y 1020; Total Length 1500, 1500 y 1040; M = 1, 1, 0; offset 0, 185, 370.

Clase 2 · IPv6: cabecera, direcciones, NDP, autoconfiguración y transición

Oct 4, 2026 · @Joaquin

Apunte para leer a la par de las diapositivas "IPv6" (Barbieri, Robles). Sigue el orden del PDF y compara todo el tiempo con IPv4 (Clase 1), porque IPv6 se entiende mejor como "qué se arregló y cómo".

1. Por qué IPv6

IPv6 no es "IPv4 con direcciones más largas": es **otro protocolo**, incompatible con IPv4, diseñado para resolver de fondo los problemas que IPv4 parchó con subnetting, CIDR, DHCP y NAT.

Los problemas de IPv4 que motivan el cambio

- **Se acabaron las direcciones.** $2^{32} \approx 4.300$ millones no alcanzan para todas las personas, mucho menos para todos los dispositivos (celulares, IoT). Los RIR ya no tienen bloques libres. NAT estiró la vida de IPv4, pero rompiendo el modelo extremo a extremo (Clase 5).
- **Tablas de ruteo enormes** en el backbone de Internet (la diapositiva dice unas 900.000 rutas). Cada entrada consume memoria y tiempo de búsqueda en los routers del núcleo.
- **Routers congestionados por procesamiento:** en IPv4 cada router recalcula el checksum, puede tener que fragmentar y tiene que interpretar opciones de largo variable.
- **Cosas que no se pensaron desde el principio** y se agregaron después, como pudieron:
  - **Seguridad en capa 3** (IPsec era opcional).
  - **Calidad de servicio (QoS):** distinguir tráfico que necesita bajo retardo.
  - **Autoconfiguración y renumeración fáciles:** en IPv4 hace falta un servidor DHCP.
  - **Movilidad a nivel de red (Mobile IP):** que un equipo cambie de red sin cortar sus conexiones.

¿Y la versión 5?

El campo Version de la cabecera (4 bits) lo asigna la IANA. El número **5 ya estaba usado** por un protocolo experimental, el Internet Stream Protocol versión 2 (ST-II, RFC 1819), pensado para tráfico de tiempo real. Por eso el sucesor de IPv4 es la versión 6. Los otros números se reservaron para otras propuestas de reemplazo de IP que no prosperaron.

Características de IPv6

- **Direcciones de 128 bits:** $2^{128} \approx 3{,}4 \times 10^{38}$ direcciones. No es "4 veces más" que IPv4: es 2^{96} veces más. La escasez deja de existir, y eso permite desperdiciar a propósito (cada LAN recibe un /64 aunque tenga 3 hosts).
- **Cabecera simplificada y de tamaño fijo** (40 bytes), más rápida de procesar.
- **Tablas de ruteo más ordenadas:** la asignación es jerárquica desde el inicio (IANA → RIR → ISP → cliente), lo que permite agregar mucho mejor.
- **Autoconfiguración (plug and play):** un host puede armarse su propia dirección sin servidor.
- **IPsec obligatorio** en la especificación original (hoy recomendado más que obligatorio, pero la clase lo presenta así).
- **Jumbogramas:** datagramas de más de 64 KB, mediante una opción de extensión.
- **Movilidad** y **muchas más direcciones multicast**; desaparece el broadcast.

Los cambios concretos respecto de IPv4

Aspecto	IPv4	IPv6
Direcciones	32 bits	128 bits
Cabecera	20 bytes + opciones (máx. 60), variable	40 bytes fijos
Opciones	Campo Options dentro de la cabecera	No existe; se reemplaza por **cabeceras de extensión**
Fragmentación	La pueden hacer los routers	Solo el **origen**, como cabecera de extensión
Checksum de cabecera	Sí, recalculado en cada salto	**Eliminado**
Identificar flujos	No hay	**Flow Label** (20 bits)
Nombres de campos	TOS, TTL, Protocol	Traffic Class, Hop Limit, Next Header
Broadcast	Sí	No existe (se usa multicast)
Resolver MAC	ARP	Neighbor Discovery (ICMPv6)
Configuración automática	DHCP	SLAAC y/o DHCPv6

2. La cabecera IPv6

La cabecera IPv6 tiene **8 campos y siempre 40 bytes**. Es el doble de larga que la mínima de IPv4, pero solo porque las direcciones ocupan 32 bytes; sin ellas, IPv6 tiene 8 bytes de control contra 12 de IPv4. Menos campos y tamaño fijo = los routers la procesan más rápido.

Los campos

Campo	Bits	Qué hace	Equivalente en IPv4
Version	4	Vale 6	Version
Traffic Class	8	Prioridad / clase de servicio (QoS)	TOS
Flow Label	20	Etiqueta un **flujo**: una secuencia de paquetes que deben tratarse igual (por ejemplo, una llamada de voz). Un router puede reconocer el flujo sin mirar puertos	No existe
Payload Length	16	Largo de lo que viene **después** de la cabecera fija (extensiones + datos), en bytes	Total Length, pero sin contar la cabecera
Next Header	8	Qué viene a continuación: una cabecera de extensión o el protocolo superior (6 = TCP, 17 = UDP, 58 = ICMPv6)	Protocol
Hop Limit	8	Se decrementa en cada router; en 0 se descarta	TTL (el nombre nuevo dice lo que realmente hace: cuenta saltos, no tiempo)
Source Address	128	Dirección origen	Source Address
Destination Address	128	Dirección destino	Destination Address

Qué se eliminó de IPv4 y por qué

La diapositiva "Cambios en la cabecera IPv4" tacha estos campos:

- **Header Length (H Len):** ya no hace falta porque la cabecera mide siempre 40 bytes.

- **Identification, Flags, Fragment Offset:** los routers ya no fragmentan. Si el origen necesita fragmentar, usa una cabecera de extensión de fragmentación (secciones 3 y 10).
- **Header Checksum:** era caro (se recalculaba en cada salto porque cambia el TTL) y redundante: la capa de enlace ya detecta errores en cada salto (CRC de Ethernet) y la capa de transporte verifica los datos de punta a punta. Por eso en IPv6 el **checksum de UDP pasa a ser obligatorio** (en IPv4 era opcional): alguien tiene que proteger las direcciones.
- **Options y Padding:** reemplazados por cabeceras de extensión, que van fuera de la cabecera fija y la mayoría de los routers ni mira.

Resumen de la comparación: de los 13 campos de IPv4 se **conservan** 3 tal cual (Version, Source, Destination), se **renombran** 4 (TOS → Traffic Class, Total Length → Payload Length, TTL → Hop Limit, Protocol → Next Header), se **eliminan** 6 (H Len, Identification, Flags, Fragment Offset, Checksum, Options) y se **agrega** 1 (Flow Label).

3. Cabeceras de extensión

Las cabeceras de extensión son cabeceras opcionales que se **encadenan detrás de la cabecera fija**. Reemplazan al campo Options de IPv4 y hacen que el protocolo sea **extensible** sin tocar la cabecera principal.

Cómo funciona el encadenamiento

Cada cabecera tiene un campo **Next Header** que dice qué viene después. La cabecera fija dice "después viene una de Routing"; la de Routing dice "después viene una de Fragment"; la de Fragment dice "después viene TCP". Es una lista enlazada. Si no hay extensiones, el Next Header de la cabecera fija apunta directo al protocolo superior (TCP, UDP, ICMPv6).

```
[IPv6 fija | NH=Routing] → [Routing | NH=Fragment] → [Fragment | NH=TCP] →
[Segmento TCP]
```

La ventaja frente a las opciones de IPv4

En IPv4 las opciones estaban **dentro** de la cabecera, así que todo router tenía que mirarlas. En IPv6, **en general las procesan solo los extremos**: un router lee la cabecera fija, ve que no hay nada para él y reenvía. Eso acelera el forwarding.

El orden recomendado y quién procesa cada una

Orden	Cabecera	Quién la procesa	Para qué sirve
1	Hop-by-Hop Options	**Cada router** del camino	Opciones que todos deben ver (ej. jumbogramas, Router Alert). Por eso va primera: el router la encuentra enseguida
2	Destination Options (primera aparición)	Los routers listados en la Routing Header	Opciones para los nodos intermedios elegidos
3	Routing	Los routers indicados	Pasar por ciertos nodos (como el source routing de IPv4). La variante **RH0 está desaconsejada** porque permitía ataques de amplificación
4	Fragment	Los extremos	Fragmentación hecha por el origen; contiene Identification, offset y bit M, igual que en IPv4
5	Authentication (AH)	Los extremos	IPsec: autenticidad e integridad
6	Encapsulating Security Payload (ESP)	Los extremos	IPsec: cifrado
7	Destination Options (final)	Solo el destino final	Opciones para el destino

La diapositiva escribe "Encapsulation Secuity": es *Encapsulating Security Payload*.

4. Direcciones IPv6: tipos, alcances y notación

Una dirección IPv6 tiene 128 bits, **no hay clases** ni máscaras en notación decimal: solo un **prefijo** (/n). Y algo nuevo: una interfaz **tiene varias direcciones a la vez**, cada una con distinto alcance y tiempo de vida.

Cómo se llegó a 128 bits

En el diseño hubo dos posturas: direcciones de largo fijo de 64 bits, o de largo variable de hasta 160 bits. El acuerdo fue **128 bits fijos**, con estas ideas:

- **Varias direcciones por interfaz** (link-local + una o más globales + temporales).
- **Distintos alcances (scope):** hasta dónde es válida la dirección (solo el enlace, la organización o toda Internet).
- **Distintos tiempos de vida:** una dirección puede estar *preferred* (se usa para conexiones nuevas) o *deprecated* (sigue funcionando para las existentes pero no se usa para nuevas), y vencer.

Los tres tipos

Tipo	A quién llega	Comentario
Unicast	A una interfaz	Como en IPv4
Anycast	A **una** de un grupo de interfaces, la más cercana según el ruteo	Se toman del mismo espacio que las unicast: no se distinguen por su forma
Multicast	A **todas** las interfaces de un grupo	Prefijo **FF00::/8**

No hay broadcast. Todo lo que IPv4 hacía con broadcast (ARP, DHCP Discover) se hace en IPv6 con multicast a grupos específicos. La ventaja: el broadcast interrumpe a todos los hosts de la LAN, el multicast solo a los suscriptos al grupo.

Alcances de las unicast (resumen, se detallan en la sección 5)

Alcance	Prefijo	Equivalente IPv4
Link-local (solo el enlace)	FE80::/10, usado como FE80::/64	169.254.0.0/16 (APIPA), pero obligatoria
Site-local (desaconsejada)	FEC0::/10	Privadas RFC 1918
Unique Local, ULA	FC00::/7 (en la práctica FD00::/8)	Privadas RFC 1918
IPv4-mapped / IPv4-compatible (esta última desaconsejada)	::FFFF:a.b.c.d / ::a.b.c.d	—
Global	2000::/3	Públicas

Una dirección unicast se divide en **red, [subred] y host**. En la práctica: los primeros 64 bits son el **prefijo** (red + subred) y los últimos 64 son el **identificador de interfaz (IID)**, el equivalente a la parte de host.

```
|        prefijo de red        | subred |      identificador de interfaz
(IID)      |
|   nnnn : nnnn : nnnn         : ssss   :  hhhh : hhhh : hhhh : hhhh
|
|<---------------- 64 bits ------------>|<----------------- 64 bits ---------
----->|
```

Notación

Se escribe en **hexadecimal**, en **8 grupos de 16 bits** (4 dígitos hex cada uno) separados por ":". Dos reglas de abreviación:

1. **Se pueden omitir los ceros a la izquierda** de cada grupo: 0db8 → db8, 0001 → 1, 0076 → 76. Nunca los de la derecha (db80 no es db8).
2. **Una secuencia de grupos todos en cero se reemplaza por "::"**, pero **una sola vez** por dirección. Si se usara dos veces, no se sabría cuántos grupos de ceros hay en cada hueco.

Ejemplos de la clase:

Completa	Abreviada
2001:0db8:1011:0001:36ed:04ff:fe32:0076	2001:db8:1011:1:36ed:4ff:fe32:76
2001:db8:1011:1:0:0:0:1	2001:db8:1011:1::1
2001:db8:0:0:1011:0:0:1	2001:db8:0:0:1011::1 o 2001:db8::1011:0:0:1 (no ambas "::")
2001:db8:0:1011:0:0:0:1	2001:db8:0:1011::1

Cuando hay dos huecos posibles, la convención (RFC 5952) es comprimir el más largo, y si empatan, el primero.

En URLs la dirección va entre corchetes, porque los ":" se confundirían con el puerto: `http://[2001:db8:1011:1::1]:8080`.

No se usa máscara en decimal, solo largo de prefijo: 2001:db8:1234::/48.

5. Direcciones unicast, una por una

Un host IPv6 típico tiene, en la misma interfaz, **una link-local y una o más globales** (ver la salida de `ip -6 addr show` en la sección 12). Cada tipo existe por una razón distinta.

Link-local (FE80::/10, usada como FE80::/64)

- **Alcance: solo el enlace** directamente conectado. Un router **nunca** reenvía un paquete con origen o destino link-local.
- **Obligatoria en toda interfaz multiacceso.** Es la primera que se configura y no depende de nadie: el host la genera solo, sin router ni servidor.
- ¿Para qué sirve si no sale de la LAN? Para todo el **plano de control local**: Neighbor Discovery, la autoconfiguración, hablar con el router. De hecho, la ruta default de un host IPv6 suele apuntar a la **link-local del router** (se ve en la tabla de ruteo de la sección 12: `default via fe80::200:ff:feaa:5`).
- Se arma como **FE80:0:0:0** + **IID de 64 bits**.
- Como todas las interfaces tienen una FE80::/64, si un equipo tiene varias interfaces hay que aclarar por cuál se habla (`fe80::1%eth0`).

El identificador de interfaz (IID) y cómo se genera

El IID son los 64 bits bajos. Hay varias formas de obtenerlo:

1. **EUI-64 derivado de la MAC (EUI-48).** Es el método clásico y el que dibuja la diapositiva "IPv6 sobre LAN":
   a. Se toma la MAC de 48 bits, por ejemplo 00:54:21:CB:97:92.
   b. Se parte al medio y se inserta **FF:FE** en el centro: 00:54:21:**FF:FE**:CB:97:92 (ahora son 64 bits).
   c. Se **invierte el séptimo bit** del primer byte (el bit U/L, universal/local): 00 = 0000 0000 → 0000 00**1**0 = 02.
   d. Resultado: IID 0254:21FF:FECB:9792; link-local **FE80::254:21FF:FECB:9792**.
   - Es un método **desaconsejado** (RFC 8064) por privacidad: la MAC queda visible en la dirección, y como la MAC no cambia, se puede rastrear al dispositivo aunque cambie de red.
   - Se aplica a redes multiacceso con MAC-48: Ethernet, 802.11, Bluetooth, Fibre Channel y tecnologías viejas (FDDI, Token Ring, Token Bus, ATM).
2. **IID opaco y estable (RFC 7217).** Se calcula con un hash de datos como el prefijo y un secreto del equipo: es estable dentro de una red, pero distinto en cada red y no revela la MAC. Es lo recomendado.
3. **Direcciones temporales (RFC 8981).** IID aleatorio que se cambia periódicamente; se usa para conexiones salientes, para que los servidores no puedan seguirte. Es la que aparece como `scope global temporary` en la salida de Linux.
4. **Manual:** típico en routers y servidores (por ejemplo, 2001:db8:1234:3::10, fácil de recordar).

Sea cual sea el método, antes de usar la dirección el host hace **DAD (Duplicate Address Detection)** para verificar que nadie más la tenga (sección 9). El mismo IID puede reutilizarse para armar las otras direcciones unicast (la global, por ejemplo).

Site-local (FEC0::/10), desaconsejada

Pensada como el equivalente a las privadas de IPv4: válida dentro de un "sitio" u organización. Se abandonó (RFC 3879, 2004) porque **nunca quedó claro dónde termina un sitio**, y eso generaba ambigüedad y problemas al unir redes de dos organizaciones que usaban los mismos prefijos.

Unique Local Addresses, ULA (FC00::/7)

Reemplazan a las site-local (RFC 4193).

- El rango FC00::/7 se divide en FC00::/8 (sin definir) y **FD00::/8**, que es la que se usa. El octavo bit (L) en 1 indica "definida localmente".
- Después del FD van **40 bits de Global ID generados (pseudo)aleatoriamente**. Ese es el truco: como cada organización elige un ID al azar entre 2^{40}, es casi imposible que dos coincidan. Si dos empresas se fusionan, sus redes privadas no chocan, que era el problema de site-local (y de 192.168.0.0 en IPv4).
- Alcance: el sitio u organización; no se rutean en Internet.

```
| FD (8 bits) | Global ID aleatorio (40 bits) | Subnet ID (16) | IID (64) |
```

IPv4-mapped y IPv4-compatible

- **IPv4-mapped (::FFFF:a.b.c.d, RFC 4291):** representa una dirección IPv4 dentro del formato IPv6. No viaja por la red; la usan los sistemas operativos para que una aplicación con un socket IPv6 pueda atender también a clientes IPv4. Ej.: ::ffff:10.0.3.10.
- **IPv4-compatible (::a.b.c.d):** era para túneles automáticos viejos. **Desaconsejada.** Ej.: ::10.0.3.10.

Notación mixta: los últimos 32 bits se pueden escribir en decimal con puntos.

Globales (Aggregatable Global Unicast, 2000::/3)

- Equivalen a las **públicas** de IPv4: alcance Internet.
- 2000::/3 significa que los primeros 3 bits son 001; o sea, empiezan con 2 o 3 (2000:: a 3FFF::).
- El **prefijo lo cede el proveedor** (ISP), que a su vez lo recibió de un RIR. Por eso se llaman "agregables": todos los clientes de un ISP comparten el prefijo del ISP, y en Internet se anuncia una sola ruta por ISP.

Estructura típica:

```
| Prefijo global de ruteo (48 bits, del ISP) | Subnet ID (16 bits, tuyo) | IID (64 bits) |
```

Con un /48 una organización tiene 2^{16} = 65.536 subredes /64. Con un /56 (común en hogares), 256.

6. Multicast y direcciones especiales

Como IPv6 no tiene broadcast, el multicast hace todo ese trabajo. Entender **solicited-node** es la clave para entender por qué Neighbor Discovery es más eficiente que ARP.

Formato de una dirección multicast (FF00::/8)

```
| FF (8 bits) | Flags (4) | Scope (4) | Group ID (112 bits) |
```

- **FF:** identifica que es multicast.
- **Flags:** indican si el grupo es **permanente** (asignado por IANA, bien conocido) o **temporal** (creado ad hoc). Los demás bits están reservados.
- **Scope (alcance):** hasta dónde puede viajar el paquete. Es lo que en IPv4 se intentaba controlar con el TTL.

Scope	Alcance	Ejemplo de dirección
1	Nodo / interfaz local (no sale del equipo)	FF01::1
2	Link-local (no sale de la LAN)	FF02::1
5	Site-local	FF05::2
8	Organización	FF08::…
E	Global (toda Internet)	FF0E::…

Leer una dirección: en FF02::1, el "0" son los flags (permanente) y el "2" es el scope (link-local).

Direcciones multicast bien conocidas

Dirección	Grupo	Equivalente IPv4
FF01::1 / FF01::2	Todos los nodos / routers, dentro del propio equipo	—

Dirección	Grupo	Equivalente IPv4
FF02::1	**Todos los nodos** de la LAN. Es lo más parecido a un broadcast	224.0.0.1
FF02::2	**Todos los routers** de la LAN. Acá se manda el Router Solicitation	224.0.0.2
FF02::5	Routers OSPFv3	224.0.0.5
FF02::6	Routers designados OSPFv3 (DR)	224.0.0.6
FF02::8	Routers IS-IS	—
FF02::9	Routers RIP	224.0.0.9
FF02::1:2	**Todos los agentes DHCP** (servidores y relays) de la LAN	255.255.255.255 (DHCP usaba broadcast)
FF05::2	Todos los routers del sitio	—
FF05::1:3	Todos los servidores DHCP del sitio (RFC 3315)	—
FF0X::FB	mDNS (DNS multicast)	224.0.0.251
FF0X::101	NTP	224.0.1.1

La X en FF0X significa "cualquier scope". (La diapositiva lista NTP como FF0X::102; según el registro de IANA, NTP es FF0X::101.)

Solicited-Node Multicast Address (la "SD" de la diapositiva)

Es un grupo multicast **derivado de cada dirección unicast** del host. Se arma así:

```
FF02:0:0:0:0:1:FF  +  últimos 24 bits de la dirección unicast
```

Ejemplo: para 2001:db8:1234:3::10, los últimos 24 bits son 00:00:10, así que la solicited-node es **FF02::1:FF00:10**. Para fe80::200:ff:feaa:7 → **FF02::1:FFAA:7**.

Por cada dirección unicast o anycast que tiene, el host **se une (join)** a su solicited-node correspondiente.

Por qué existe: en IPv4, ARP pregunta "¿quién tiene 192.168.1.5?" por broadcast, y **todos** los hosts de la LAN tienen que recibir el paquete, procesarlo y descartarlo. En IPv6, la pregunta va a la solicited-node de la IP buscada. Como solo los hosts cuyas direcciones terminan en esos mismos 24 bits están en ese grupo (normalmente, uno solo), la placa de red de los demás ni siquiera le pasa el paquete al sistema operativo. Se evita el *flooding* de la LAN.

Cómo un multicast IP se convierte en MAC destino

Para que la placa de red filtre por hardware, cada grupo multicast IP se mapea a una MAC multicast:

	Prefijo MAC	Qué se copia de la IP
IPv4	01:00:5E (rango 01:00:5E:00:00:00 a 01:00:5E:7F:FF:FF)	Los **23 bits menos significativos**
IPv6	**33:33** (rango 33:33:00:00:00:00 a 33:33:FF:FF:FF:FF)	Los **32 bits menos significativos**

Ejemplos: FF02::1 → MAC 33:33:00:00:00:01. FF02::1:FF00:10 → 33:33:FF:00:00:10. En la salida de `/proc/net/dev_mcast` de la sección 12 se ve exactamente esto (333300000001, 3333ff000010…).

Direcciones especiales

Dirección	Qué es	Equivalente IPv4
::/128 (todo en cero)	**Sin especificar**: origen de un host que todavía no tiene dirección (por ejemplo, durante DAD)	0.0.0.0
::/0	Cualquier destino: la **ruta default**	0.0.0.0/0
::1/128	**Loopback** / localhost. Una sola dirección, no un /8 entero	127.0.0.1
2001:db8::/32	Reservado para **documentación** y ejemplos (por eso todos los ejemplos de la clase usan 2001:db8)	192.0.2.0/24
3FFE::/16	Del **6Bone**, la red experimental de IPv6. Devueltas a IANA en 2006	—

La diapositiva escribe "Any: ::0/0" mezclando dos cosas: la dirección sin especificar es ::/128 y ::/0 es la ruta default.

7. ICMPv6 y el nuevo plano de control

En IPv6, **ICMPv6 absorbe funciones que en IPv4 hacían tres protocolos separados**: ICMP (errores y ping), ARP (resolver MAC) e IGMP (grupos multicast). Por eso la clase dice que "cambia el plano de control".

La pila comparada (diapositiva "IPv6 Stack")

Función	IPv4	IPv6
Errores y diagnóstico (ping, unreachable, time exceeded)	ICMP	ICMPv6
Resolver IP → MAC	**ARP** (va directo sobre Ethernet, fuera de IP)	**Neighbor Discovery** (mensajes ICMPv6, dentro de IPv6)
Suscribirse a grupos multicast	**IGMP**	**MLD** (Multicast Listener Discovery, mensajes ICMPv6)
Descubrir el router y la configuración de la red	DHCP (o configuración manual)	**Router Discovery** de NDP (+ DHCPv6 opcional)
Redirigir a un host hacia un router mejor	ICMP Redirect	Redirect de NDP

Una consecuencia importante: como todo esto viaja dentro de ICMPv6, **no se puede bloquear ICMPv6 en un firewall** como a veces se hace con ICMP en IPv4. Si se bloquea, IPv6 deja de funcionar (no hay resolución de vecinos ni autoconfiguración).

ICMPv6 en concreto

- Definido en la **RFC 4443**. Next Header = **58**.
- Es **parte fundamental** de la pila IPv6, no un accesorio.
- Tres usos: mensajes de **control**, **informativos** (Echo Request/Reply = ping6) y de **error** (Destination Unreachable, **Packet Too Big**, Time Exceeded, Parameter Problem).
- "Packet Too Big" es nuevo y central: como los routers no fragmentan, es la forma de avisarle al origen que achique (sección 10).
- Además lo usan **MLD** (reemplazo de IGMP) y **NDP** (reemplazo de ARP, Router Discovery y Redirect).

8. Neighbor Discovery (NDP)

Neighbor Discovery (RFC 4861) es el protocolo con el que un host IPv6 **descubre a sus vecinos y a sus routers**. Reemplaza a ARP, al Router Discovery y al Redirect de IPv4, y todo viaja como mensajes ICMPv6.

Qué tiene en común con ARP

- Trabaja junto con Ethernet u otras capas de enlace multiacceso (802.11, Bluetooth, Token Ring, FDDI).
- **Mapea direcciones lógicas (IPv6) a físicas** (MAC EUI-48 o EUI-64).
- Es **dinámico**: aprende solo, sin configuración. También se pueden cargar entradas estáticas.
- Guarda lo aprendido en una tabla, la **neighbor cache** (`ip -6 neigh show`), equivalente a la tabla ARP.

Lo que cambia respecto de ARP

	ARP (IPv4)	NDP (IPv6)
Va encapsulado en	Directamente en la trama Ethernet	IPv6 → ICMPv6
La pregunta va a	Broadcast (FF:FF:FF:FF:FF:FF): todos la procesan	Multicast **solicited-node**: casi nadie más la procesa
Seguridad	Ninguna posible (no es IP)	Se le puede aplicar seguridad de IP (SEND)
Funciones	Solo resolver MAC	Resolver MAC, encontrar routers, prefijos, DAD, detectar vecinos caídos, redirect

Los 5 mensajes

Mensaje	Quién lo manda	A quién	Para qué
Neighbor Solicitation (NS)	Un host	Solicited-node de la IP buscada (o unicast para verificar)	"¿Quién tiene esta IP? Decime tu MAC." También se usa para DAD
Neighbor Advertisement (NA)	El dueño de la IP	Unicast al que preguntó (o FF02::1 si es un anuncio espontáneo)	"Soy yo, mi MAC es esta"
Router Solicitation (RS)	Un host	FF02::2 (todos los routers)	"¿Hay algún router? Mandame la configuración"

Mensaje	Quién lo manda	A quién	Para qué
Router Advertisement (RA)	Un router	FF02::1 (todos) periódicamente, o en respuesta a un RS	"Acá hay un router, el prefijo es tal, usá tal configuración"
Redirect	Un router	Un host	"Para ese destino hay un router mejor en tu LAN, mandáselo a él"

El ejemplo de la clase: n7 hace ping a n8

n7 (2001:db8:1234:3::10, MAC 00:00:00:aa:00:07) quiere hacer `ping6 2001:db8:1234:3::11`. n8 está en la misma red (mismo /64), así que n7 lo manda directo, pero no conoce su MAC: en la neighbor cache solo tiene al router (fe80::200:ff:feaa:5). Igual que con ARP, tiene que preguntar antes de poder armar la trama.

n7 resuelve la MAC de n8 con dos mensajes ICMPv6 antes del ping



Neighbor Discovery entre n7 y n8 · 4 mensajes

Paso a paso:

1. **NS:** n7 manda un Neighbor Solicitation. IP destino: la solicited-node de ::11, **FF02::1:FF00:11**. MAC destino: **33:33:FF:00:00:11** (los últimos 32 bits de esa multicast). Adentro, el campo *Target Address* = 2001:db8:1234:3::11, y una opción con la MAC de n7 (*Source Link-Layer Address*). Es "multicast en L2 y L3", como dice la diapositiva.
2. Solo n8 está suscripto a ese grupo, así que solo n8 procesa el mensaje. Como el NS trae la MAC de n7, **n8 ya la anota en su cache**.

3. **NA:** n8 responde con un Neighbor Advertisement **unicast** a n7, con su MAC en la opción *Target Link-Layer Address*. Lleva flags: **S** (solicited: es respuesta a un NS), **O** (override: actualizar la entrada existente) y **R** (si el que responde es router).
4. n7 anota la MAC de n8 y ahora sí manda el Echo Request. n8 contesta el Echo Reply sin tener que preguntar nada.

La neighbor cache y sus estados

En la salida `fe80::200:ff:feaa:5 dev eth0 lladdr 00:00:00:aa:00:05 router STALE` aparece un estado. NDP no solo aprende MAC: también verifica que los vecinos sigan vivos (**NUD, Neighbor Unreachability Detection**). Los estados principales son INCOMPLETE (pregunté y espero respuesta), REACHABLE (confirmado hace poco), **STALE** (la entrada sirve pero hace rato que no se confirma; se re-verifica la próxima vez que se use), DELAY y PROBE (verificándolo). "router" indica que ese vecino anunció ser router (flag R).

9. Autoconfiguración: SLAAC, DAD y DHCPv6

Un host IPv6 puede **armarse solo su dirección global** combinando el prefijo que anuncia el router con su propio IID. Eso es **SLAAC** (Stateless Address Autoconfiguration). "Stateless" significa que **nadie lleva registro** de qué dirección tiene cada host, a diferencia de un servidor DHCP que guarda una tabla de asignaciones (stateful).

El proceso completo, paso a paso

Es lo que se ve en la captura de la clase cuando n7 baja y sube la interfaz (`ifconfig eth0 down; ifconfig eth0 up`) y, unos segundos después, ya puede hacer ping a otra red.

1. **Link-local.** El host genera su IID y arma FE80::IID. Todavía no la usa: está en estado *tentative*.
2. **DAD sobre la link-local.** Manda un **NS** preguntando por su propia dirección, con origen **::** (todavía no tiene ninguna) y destino la solicited-node de esa dirección. Si alguien responde con un NA, la dirección está duplicada y no se usa. Si nadie responde en un tiempo, la dirección es válida. Es la misma idea que el ARP Probe de IPv4.
3. **Router Solicitation.** Para no esperar el próximo anuncio periódico, el host manda un **RS** a **FF02::2** (todos los routers) desde su link-local.
4. **Router Advertisement.** El router responde con un **RA** (a FF02::1 o al host). El RA trae:
   - Uno o más **prefijos** (por ejemplo, 2001:db8:1234:3::/64), cada uno con sus flags y tiempos de vida.
   - Que el que lo manda es un router, y por cuánto tiempo usarlo como default (*router lifetime*).

- Los flags **M** y **O** (abajo).
- Opcionalmente, servidores **DNS** (opción RDNSS, RFC 6106) y el MTU del enlace.

5. **Dirección global.** El host arma prefijo + IID (por EUI-64, RFC 7217 o temporal) y vuelve a hacer **DAD**.
6. **Default gateway.** El host toma como ruta default la **dirección link-local del router** que mandó el RA. Por eso en la tabla de ruteo de n7 aparece `default via fe80::200:ff:feaa:5 ... proto kernel ... expires 24sec`: la aprendió del RA y vence si no se renueva.

Resultado: sin servidor, sin configuración manual, el host tiene link-local, global, gateway y (si el RA lo trae) DNS. Las diapositivas "Router Discovery: aprendizaje de su propia configuración" muestran justamente a n7 pensando "quiero salir de mi red pero solo conozco mi link-local".

Los flags de la información de prefijo: L, A, R

Flag	Nombre	Qué indica
L	On-link	El prefijo está en este enlace: las direcciones con ese prefijo se alcanzan **directo** (con ND), sin pasar por el router. Si no está en 1, no se asume que sea off-link: simplemente no se afirma nada
A	Autonomous	El host **puede usar este prefijo para autoconfigurarse** una dirección global con SLAAC
R	Router address	El campo de prefijo contiene además la dirección completa del router. La clase lo vincula con NUD (saber que ese vecino es un router)

Cada prefijo trae dos tiempos: **valid lifetime** (hasta cuándo la dirección existe) y **preferred lifetime** (hasta cuándo se usa para conexiones nuevas). Entre uno y otro la dirección está *deprecated*. En la salida de Linux de la clase: `valid_lft 86396sec preferred_lft 14396sec`. Esto es lo que hace fácil **renumerar** una red: el router anuncia el prefijo viejo con lifetime 0 y uno nuevo, y los hosts migran solos.

SLAAC vs DHCPv6: los flags M y O del RA

La configuración manual sigue siendo posible (routers, servidores). Para la automática hay dos mecanismos que se combinan:

- **SLAAC:** stateless, lo da el RA.

- **DHCPv6:** como DHCP de IPv4, con servidor. Puede ser **stateful** (el servidor asigna y registra direcciones) o **stateless** (solo da parámetros extra, como DNS o NTP). Usa UDP (puertos 546 cliente, 547 servidor) y el cliente busca servidores en **FF02::1:2**.

El router le dice al host qué usar con dos bits del RA (RFC 4861):

M (Managed)	O (Other)	Qué hace el host
0	0	Solo **SLAAC**. Si hay un servidor DHCPv6, no lo usa
0	1	Dirección por **SLAAC** + **otros parámetros** (DNS, etc.) por DHCPv6 stateless. En el daemon del router (radvd) es `AdvOtherConfigFlag`
1	cualquiera	Dirección por **DHCPv6 stateful**. En radvd es `AdvManagedFlag`

Un detalle que la clase marca con "salvo router": **DHCPv6 no entrega el default gateway**. Aun con M = 1, el host aprende el router por el RA. Por eso "DHCPv6 solo" igual **requiere RA**.

Las combinaciones que nombra la clase:

- **SLAAC solo:** hoy alcanza para la configuración básica de Internet (prefijo, router y DNS; también MTU).
- **SLAAC solo, pero el equipo no soporta la opción DNS del RA:** necesita DHCPv6 para el DNS.
- **SLAAC + DHCPv6:** configuración básica por SLAAC y parámetros extra por DHCPv6.
- **DHCPv6 solo:** necesita igual el RA para el gateway.

10. PMTU Discovery

En IPv6 **los routers nunca fragmentan**. Si un paquete no entra en el próximo enlace, el router lo descarta y le manda al origen un **ICMPv6 Packet Too Big** con el MTU de ese enlace. El origen achica y, si hace falta, fragmenta él mismo.

Conceptos

- **Link MTU:** el MTU de un enlace en particular. Depende de la capa 2 (Ethernet: 1500).
- **Path MTU (PMTU):** el **menor** link MTU de todos los enlaces del camino entre origen y destino. Es el paquete más grande que puede ir de punta a punta sin fragmentarse.

- **MTU mínimo garantizado:** IPv6 exige que todo enlace soporte al menos **1280 bytes** (RFC 2460, hoy RFC 8200). En IPv4 el mínimo es 68 bytes (RFC 791). Si un enlace físico no llega a 1280, la capa de enlace tiene que fragmentar y reensamblar por debajo de IPv6, de forma transparente.

Por qué sacarle la fragmentación a los routers

Fragmentar en cada router es caro (copiar cabeceras, recalcular), y perder un solo fragmento hace perder todo el datagrama. Pasando la responsabilidad al origen, los routers solo reenvían o descartan: más simple y más rápido. Es coherente con el resto del diseño (cabecera fija, sin checksum).

Cómo funciona (RFC 1981)

1. El origen asume PMTU = MTU de su propio enlace y manda paquetes de ese tamaño.
2. Si un router en el camino tiene un enlace de salida más chico, descarta el paquete y responde **ICMPv6 Packet Too Big** indicando su MTU.
3. El origen baja su estimación de PMTU a ese valor y reenvía.
4. Se repite hasta que los paquetes lleguen. Desde ahí, si el origen tiene que mandar algo más grande, **fragmenta de extremo a extremo** con la cabecera de extensión Fragment.

Atajo posible: mandar siempre paquetes de 1280 bytes y no descubrir nada. Funciona en cualquier camino, pero **no es óptimo**: en caminos con MTU 1500 se desaprovecha capacidad.

El ejemplo de la clase

n6 (2001:db8:1234:2::10) le manda a n7 (2001:db8:1234:3::10). El camino atraviesa enlaces de MTU **1500** (la LAN de n6), **1350** (entre n2 y n1) y **1280** (la LAN de n7). El PMTU es 1280. Con paquetes grandes, primero n2 responde Packet Too Big (1350), después n3 responde Packet Too Big (1280), y ahí n6 ya manda paquetes que llegan. La captura `ipv6-frag-doble` de la diapositiva muestra ese doble ajuste.

En IPv4 existe el mismo mecanismo (con el bit DF = 1 y el ICMP "Fragmentation needed"), pero es opcional: los routers IPv4 todavía pueden fragmentar.

11. Transición y convivencia IPv4 / IPv6

Como IPv4 e IPv6 **no son compatibles** (un host solo IPv6 no puede hablar con uno solo IPv4), no hay un "día D" para cambiar. Van a convivir durante mucho tiempo, y para eso hay tres familias de técnicas: **doble pila**, **túneles** y **traducción**, además de combinaciones entre ellas.

Doble pila (dual stack)

- Cada equipo corre **IPv4 e IPv6 a la vez**, con direcciones de los dos. No se elimina IPv4.
- Hoy la mayoría de los sistemas operativos lo soporta por defecto.
- ¿Quién elige cuál usar? **La aplicación** (o la biblioteca de red), según lo que devuelva el DNS: un registro **A** da la dirección IPv4 y un **AAAA** ("quad A") la IPv6.
- **Happy Eyeballs (RFC 6555):** si el destino tiene las dos, el cliente intenta IPv6 y casi en paralelo IPv4, y se queda con la que conecte primero. Así un IPv6 roto no le hace esperar al usuario.
- Es la técnica preferida, pero no ahorra direcciones IPv4: cada equipo sigue necesitando una.

Túneles

Sirven para cruzar una zona que **solo tiene IPv4**: el paquete IPv6 entero se mete **como datos dentro de un paquete IPv4** (encapsulamiento), cruza la zona IPv4 y del otro lado se desarma. Los extremos del túnel (routers o hosts) necesitan doble pila.

Manuales (alguien configura los dos extremos a mano):

- **6in4 / SIT (Simple Internet Transition), IP-IP:** IPv6 directo dentro de IPv4 (protocolo 41). Punto a punto.
- **GRE (Generic Routing Encapsulation):** encapsulación genérica, protocolo IP **47**. Punto a punto entre host y router o entre routers; típico en intranets. La captura de la clase muestra un paquete IPv6 dentro de GRE dentro de IPv4.
- **Tunnel brokers:** servicios con interfaz web que te arman el otro extremo del túnel 6in4 (por ejemplo, tunnelbroker.net de Hurricane Electric).

Automáticos (el extremo remoto se deduce de la propia dirección):

- **6to4:** arma un prefijo IPv6 a partir de la IPv4 pública. **No funciona detrás de NAT** (necesita IPv4 pública).
- **ISATAP:** para conectar hosts dentro de una intranet IPv4. Tampoco atraviesa NAT.
- **Teredo:** encapsula IPv6 en **UDP** sobre IPv4, justamente para poder **atravesar NAT** (el NAT sabe traducir UDP; no sabe qué hacer con el protocolo 41).

Traducción

La clase solo la nombra: convertir paquetes IPv6 en IPv4 y viceversa (por ejemplo, NAT64 junto con DNS64), para que un host **solo IPv6** pueda hablar con un servidor **solo IPv4**.

Estado del despliegue (estadísticas de Google)

- 2023: algunos países ya superaban el 60 % de tráfico IPv6 (Francia, Alemania, India).

- 2025: Francia 85,41 %, India 75,18 %, Alemania 74,33 %, EE.UU. 55,07 %, Brasil 51,01 %.
- A nivel mundial la adopción crece de forma sostenida, pero IPv4 sigue presente: por eso la convivencia es la regla.

12. Subnetting en IPv6 y comandos de Linux

En IPv6 **cada LAN lleva un /64**, siempre. No se calcula el tamaño según los hosts como en VLSM: sobran direcciones, y SLAAC necesita exactamente 64 bits de IID. El subnetting se hace con los bits que hay **entre el prefijo recibido y el /64**.

El ejemplo de la clase: 4 subredes

Caso 1: te dan 2001:db8:1234::/48. Entre /48 y /64 hay **16 bits** (el cuarto grupo), así que podés tener 2^{16} = 65.536 subredes. Simplemente numerás el cuarto grupo:

Red	Prefijo
0	2001:db8:1234:0000::/64 = 2001:db8:1234::/64
1	2001:db8:1234:0001::/64 = 2001:db8:1234:1::/64
2	2001:db8:1234:0002::/64 = 2001:db8:1234:2::/64
3	2001:db8:1234:0003::/64 = 2001:db8:1234:3::/64

¿Y si lo resuelvo con solo 2 bits, como en IPv4? Tomo los 2 primeros bits del cuarto grupo y obtengo cuatro /50:

Red	Cuarto grupo en binario	Prefijo
0	**00**00 0000 0000 0000	2001:db8:1234::/50
1	**01**00 0000 0000 0000	2001:db8:1234:4000::/50
2	**10**00 0000 0000 0000	2001:db8:1234:8000::/50
3	**11**00 0000 0000 0000	2001:db8:1234:c000::/50

(La diapositiva escribe estos prefijos como "2001:db8:1234:40::/50", ":80::" y ":C0::": es un error. Con los dos bits altos del cuarto grupo, el valor hexadecimal de ese grupo es 4000, 8000 y C000. "40" sería 0040.)

Esto está mal como diseño de LAN por tres motivos que da la clase:

- **No escala:** si mañana necesitás una quinta red, ya usaste todo.

- **SLAAC no funciona con /50:** la autoconfiguración necesita un prefijo /64.
- Solo tiene sentido si **después vas a subdividir** cada /50 (por ejemplo, un /50 por edificio y adentro /64 por piso).

Caso 2: te dan un /56 (2001:db8:1234::/56). Entre /56 y /64 hay **8 bits**, el último byte del cuarto grupo: $2^8 = 256$ subredes, de la forma 2001:db8:1234:00**XX**::/64. Las cuatro primeras son 2001:db8:1234:0::/64, :1::/64, :2::/64 y :3::/64. (Una de las diapositivas escribe el /56 como 2001:db8:1234:1234::/56; ese prefijo no está bien formado, porque el /56 corta en la mitad del cuarto grupo y el "34" final quedaría en la parte de subred. La otra diapositiva usa 2001:db8:1234::/56, que es la correcta.)

El ejemplo de VLSM de la práctica 3 llega a la misma conclusión: con 2001:db8:1::/48 y tres departamentos, no vale la pena hacer VLSM; se le da un /64 a cada uno (2001:db8:1:1::/64, :2::/64, :3::/64), y cada /64 tiene $2^{64} \approx 1{,}8 \times 10^{19}$ direcciones.

Comandos de Linux de la clase

Comando	Qué muestra o hace
`ip -6 addr show` (o `ifconfig eth0`)	Direcciones IPv6 de cada interfaz, con su scope y lifetimes
`ip -6 addr add 2001:db8:1234:3::10/64 dev eth0`	Agrega una dirección manual
`ip -6 neigh show`	Neighbor cache (el "arp -a" de IPv6)
`ip -6 route show` o `netstat -nr -A inet6`	Tabla de ruteo IPv6
`ip -6 maddr show` o `netstat -g -A inet6 -n`	Grupos multicast a los que está unida cada interfaz
`cat /proc/net/igmp6`	Lo mismo, desde el kernel
`cat /proc/net/dev_mcast`	Las **MAC** multicast que la placa acepta
`ping6 <dir>`	Ping sobre ICMPv6

Cómo leer las salidas de la clase

La interfaz eth0 de n7 tiene, al mismo tiempo:

Dirección	Scope	De dónde sale
fe80::200:ff:feaa:7/64	link	Link-local con IID EUI-64 de la MAC 00:00:00:aa:00:07 (00 con el bit U/L invertido = 02 → "200"; FF:FE en el medio)
2001:db8:1234:3:200:ff:feaa:7/64	global, dynamic	SLAAC: prefijo del RA + el mismo IID EUI-64. Tiene lifetimes
2001:db8:1234:3:f8b0:a208:ca54:4ef5/64	global, temporary	SLAAC con IID temporal aleatorio (RFC 8981)
2001:db8:1234:3::10/64	global	Configurada a mano. valid_lft forever
::1/128 (en lo)	host	Loopback

Y los grupos multicast de eth0: **ff02::1** (todos los nodos) y una solicited-node por cada unicast: **ff02::1:ff54:4ef5** (de la temporal), **ff02::1:ffaa:7** (de la link-local y la EUI-64, que comparten los últimos 24 bits; por eso dice `users 2`) y **ff02::1:ff00:10** (de ::10). En `/proc/net/dev_mcast` aparecen sus MAC: 33:33:00:00:00:01, 33:33:ff:54:4e:f5, 33:33:ff:aa:00:07, 33:33:ff:00:00:10, y 01:00:5e:00:00:01 (el all-hosts de IPv4, porque tiene doble pila).

En la tabla de ruteo: `2001:db8:1234:3::/64 dev eth0` es la red directamente conectada, `fe80::/64 dev eth0` la link-local, y hay dos defaults (una manual vía 2001:db8:1234:3::1 y otra aprendida del RA vía fe80::200:ff:feaa:5, que vence). También aparecen las direcciones compat (::10.0.3.10, scope Compat) y mapped (::ffff:10.0.3.10) agregadas a mano como demostración.

13. Preguntas para autoevaluarte

1. ¿Por qué IPv6 eliminó el checksum de cabecera y qué tuvo que cambiar en UDP como consecuencia?
2. ¿Por qué la cabecera IPv6 no necesita un campo Header Length?
3. ¿Qué ventaja tienen las cabeceras de extensión frente a las opciones de IPv4? ¿Por qué Hop-by-Hop va primera?
4. Abreviá 2001:0db8:0000:0000:0001:0000:0000:0abc de la forma correcta.
5. Calculá la link-local EUI-64 de una interfaz con MAC 3C:22:FB:10:AB:CD.
6. ¿Cuál es la solicited-node de 2001:db8::1:2:3c4d:5e6f? ¿Y su MAC multicast?
7. ¿Por qué preguntar por la solicited-node es mejor que el broadcast de ARP?

8. ¿Por qué se abandonó site-local y qué resuelven los 40 bits aleatorios de las ULA?
9. Describí el proceso de SLAAC desde que se enciende la interfaz hasta que el host puede salir a Internet. ¿En qué dos momentos hace DAD?
10. Un RA llega con M = 0 y O = 1. ¿De dónde saca el host su dirección global, su gateway y su DNS?
11. ¿Por qué DHCPv6 solo no alcanza para que un host salga de su red?
12. Un paquete IPv6 de 1500 bytes llega a un router cuyo enlace de salida tiene MTU 1400. ¿Qué hace el router? ¿Y el origen?
13. ¿Por qué Teredo atraviesa NAT y 6to4 no?
14. Te asignan 2001:db8:abcd::/48. Listá las primeras tres subredes para LANs. ¿Por qué no conviene hacer /50?

Respuestas rápidas: (4) 2001:db8::1:0:0:abc. (5) FE80::3E22:FBFF:FE10:ABCD (3C = 0011 1100, invertir el séptimo bit da 0011 1110 = 3E). (6) los últimos 24 bits son 4d:5e:6f → **FF02::1:FF4D:5E6F**, MAC 33:33:FF:4D:5E:6F. (14) 2001:db8:abcd:0::/64, :1::/64, :2::/64.

Clase 3 · ARP: de la dirección IP a la dirección MAC

Oct 4, 2026 · @Joaquin

Apunte para leer a la par de las diapositivas "ARP – Address Resolution Protocol" (Robles, Barbieri) y de los ejemplos resueltos de ARP. La idea central de toda la clase: **las IP viajan de punta a punta; las MAC cambian en cada salto**.

1. El problema: un paquete necesita dos direcciones

Para que un paquete IP salga por un cable Ethernet hay que meterlo en una **trama**, y la trama necesita una **MAC destino**. El host sabe la IP destino (se la dio la aplicación o el DNS), pero no la MAC. **ARP es el que averigua esa MAC.**

Por qué no alcanza con una sola dirección

	Dirección IP	Dirección MAC
Capa	Red (3)	Enlace (2)
Tamaño	32 bits	48 bits
Tipo	Lógica: depende de en qué red estás	Física: viene grabada en la placa
Alcance	Toda la Internet (de punta a punta)	Solo el enlace local (un salto)
La usa	IP para rutear	La placa de red para aceptar o ignorar una trama

La placa de red Ethernet **no entiende de IP**: mira la MAC destino de cada trama y, si no es la suya (ni broadcast, ni un multicast al que esté suscripta), la descarta sin pasarla al sistema operativo. Entonces, para que un paquete llegue al próximo equipo, **la trama tiene que llevar la MAC de ese equipo**.

La pregunta clave: ¿la MAC de quién?

Acá está lo que más se confunde. La MAC destino de la trama **no es necesariamente la del destino final**: es la del **próximo salto**.

- Si el destino está **en mi misma red**, el próximo salto es el propio destino: necesito su MAC.

- Si el destino está **en otra red**, el próximo salto es mi **default gateway** (el router): necesito la MAC del router, no la del destino.

¿Cómo sabe el host en cuál de los dos casos está? Haciendo **IP destino AND su máscara** y comparando con su propia red (Clase 1, sección 5), o, lo que es lo mismo, consultando su tabla de ruteo.

2. Generalidades de ARP

ARP (RFC 826, noviembre de 1982) es un protocolo "helper" de IP que **traduce una IP en la MAC correspondiente** dentro de una red local, preguntando por broadcast. La clase enumera sus características; acá van con su explicación.

- **"Protocolo de capa 2, suele ser considerado de capa 3".** Está justo en la frontera. Viaja **directamente dentro de la trama Ethernet** (no dentro de un paquete IP: no tiene IP origen ni TTL en una cabecera IP), lo que lo pone en capa 2. Pero su contenido son direcciones IP y existe para servir a IP, por eso muchos lo ubican en capa 3. La trama indica que lleva ARP con el EtherType **0x0806** (IP es 0x0800).
- **Helper de IP:** IP no puede funcionar sobre Ethernet sin él.
- **Trabaja con Ethernet u otros protocolos de capa 2 multiacceso con broadcast:** Token Ring, FDDI, 802.11 (WiFi). "Multiacceso" significa que en el mismo medio hay muchos equipos y por eso hay que identificar a quién va cada trama; "con broadcast" significa que existe una forma de hablarle a todos a la vez, que es justo lo que ARP necesita para preguntar. En un enlace punto a punto (dos routers unidos por un cable serie) no hace falta ARP: hay un solo vecino posible.
- **Mapea direcciones lógicas (IP) a físicas (MAC).**
- **Dinámico, autoaprendizaje, sin configuración:** nadie carga las tablas a mano; se llenan solas a medida que hay tráfico.
- **Se encapsula dentro de una trama de la capa de enlace.**

La regla más importante: ARP no sale del dominio de broadcast

La pregunta ARP (el *ARP Request*) se manda a la MAC de broadcast **FF:FF:FF:FF:FF:FF**. Los switches reenvían el broadcast por todos sus puertos, pero **los routers no reenvían broadcasts**. Por lo tanto, un ARP Request solo llega a los equipos de **la misma red**, es decir, del mismo **dominio de broadcast**.

Consecuencia: **ARP no puede averiguar la MAC de un equipo que está en otra red.** Y tampoco la necesita: para mandarle algo a otra red, el host le entrega la trama a su router, así que solo necesita la MAC del router (sección 5).

Dominio de broadcast = conjunto de equipos que reciben un broadcast de capa 2 mandado por cualquiera de ellos. En la práctica, una LAN (o una VLAN): lo que está entre routers.

3. La tabla ARP

Cada host y cada router guarda en memoria una **tabla ARP** (o *ARP cache*) con pares **IP (32 bits) → MAC (48 bits)** de sus vecinos. Sirve para no tener que preguntar cada vez que se manda un paquete.

- Las entradas son **dinámicas**: se agregan cuando se recibe una respuesta ARP (o una pregunta, ver abajo).
- También se pueden cargar **estáticas**, a mano. Se usa por seguridad en equipos críticos, porque ARP no autentica nada y cualquiera podría responder mintiendo (ataque de *ARP spoofing*).
- Cada entrada dinámica tiene un **tiempo de permanencia configurable**. Se refresca cada vez que se usa; si vence sin usarse, se borra, y la próxima vez habrá que volver a preguntar. Esto evita guardar MAC viejas cuando un equipo cambia de placa o una IP pasa a otro equipo.

Ejemplo de la clase (`arp -a` en Linux):

```
pc-b.redes.local (172.17.1.45) at 56:6f:01:a8:00:1e [ether] on eth0
? (172.17.1.143) at 00:15:5d:01:16:01 [ether] on eth0
pc-j.redes.local (172.17.1.22) at 40:f2:e9:0b:c0:4d [ether] on eth0
```

Cada línea es: nombre (si se puede resolver por DNS inverso; si no, "?"), IP, MAC, tipo de enlace (ether) e interfaz por donde se aprendió. Hoy el comando moderno es `ip neigh show`.

Un detalle que ahorra tráfico

Cuando un equipo recibe un ARP Request dirigido a él, **anota la IP y MAC de quien preguntó** antes de responder. Razonamiento: si alguien me pregunta por mi MAC, es porque me va a mandar algo, y lo más probable es que yo le tenga que contestar. Así no necesita hacer su propio ARP para responder.

4. Caso 1: origen y destino en la misma red

Si el destino está en la misma red, ARP se usa **una sola vez**: el origen pregunta la MAC del destino y después le manda la trama directo.

Datos del ejemplo de la clase (red 172.17.1.0/24, tabla ARP de PC-A vacía):

Equipo	IP	MAC
PC-A	172.17.1.10	ab:da:dc:10:10:10
PC-B	172.17.1.20	dc:d5:dc:20:20:20

PC-A quiere hacer `ping 172.17.1.20` (un ICMP Echo Request).

Paso 0: ¿tiene PC-A todo lo que necesita?

PC-A puede armar el paquete IP: sabe su IP (origen) y la de PC-B (destino, la escribió en el ping). Al compararla con su máscara ve que **PC-B está en su misma red**. Para la trama Ethernet sabe su propia MAC, pero **no la de PC-B**:

MAC destino	MAC origen	IP destino	IP origen	Contenido
?	ab:da:dc:10:10:10	172.17.1.20	172.17.1.10	ICMP Echo Request

No puede mandarlo todavía. El Echo Request queda en espera y arranca ARP.

Paso 1: ARP Request (broadcast)

Campo	Valor	Por qué
Trama: MAC destino	**FF:FF:FF:FF:FF:FF**	No sabe a quién preguntarle, le pregunta a todos
Trama: MAC origen	ab:da:dc:10:10:10	PC-A
ARP: IP origen (sender)	172.17.1.10	Para que el que responda sepa a quién
ARP: MAC origen (sender)	ab:da:dc:10:10:10	Idem
ARP: IP destino (target)	**172.17.1.20**	La IP por la que pregunta
ARP: MAC destino (target)	**?** (todo en 0)	Es justamente lo que no sabe

Atención a un error típico de examen: la MAC destino **dentro del paquete ARP** no se completa con FF:FF:FF:FF:FF:FF. Ese es el destino de la **trama**. Dentro del ARP, la MAC target va en blanco, en ceros o con "?".

El broadcast llega a todos los equipos de la LAN, incluida la interfaz del router conectada a esa red. **Todos lo procesan, pero solo responde el que tiene la IP 172.17.1.20.** Los demás lo descartan en silencio. El router **no lo reenvía** a sus otras redes.

Paso 2: ARP Reply (unicast)

PC-B responde **solo a PC-A**, porque ahora sí sabe su MAC (vino en el Request). Los valores se "dan vuelta":

Campo	Valor
Trama: MAC destino	ab:da:dc:10:10:10 (PC-A)
Trama: MAC origen	dc:d5:dc:20:20:20 (PC-B)
ARP: IP origen	172.17.1.20
ARP: MAC origen	**dc:d5:dc:20:20:20** (la respuesta)
ARP: IP destino	172.17.1.10
ARP: MAC destino	ab:da:dc:10:10:10

PC-B, además, ya anotó a PC-A en su tabla ARP al recibir el Request.

Paso 3: la trama que faltaba

PC-A guarda la MAC de PC-B en su tabla y completa la trama:

MAC destino	MAC origen	IP destino	IP origen	Contenido
dc:d5:dc:20:20:20	ab:da:dc:10:10:10	172.17.1.20	172.17.1.10	ICMP Echo Request

Paso 4: la respuesta del ping

PC-B contesta el Echo Reply **sin hacer ARP**, porque ya tiene a PC-A en su tabla:

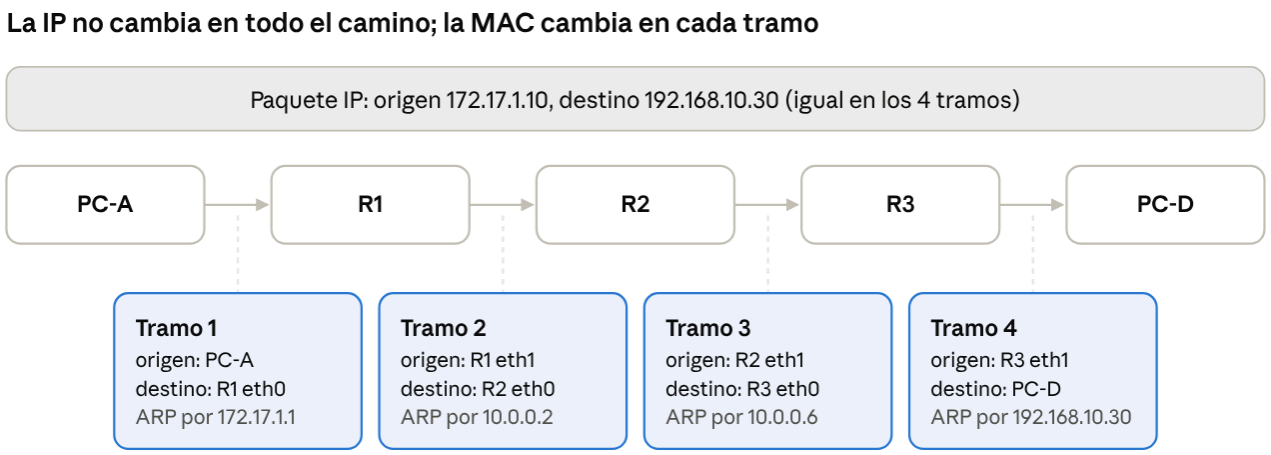
MAC destino	MAC origen	IP destino	IP origen	Contenido
ab:da:dc:10:10:10	dc:d5:dc:20:20:20	172.17.1.10	172.17.1.20	ICMP Echo Reply

Si se repite el ping enseguida, no hay ARP: las dos tablas ya tienen la entrada.

5. Caso 2: el destino está en otra red

Si el destino está en otra red, el host **no pregunta por la MAC del destino** (el broadcast no llegaría), sino por la **MAC de su default gateway**. Después, **cada router repite el proceso** en su red de salida, hasta llegar al destino.

Ejemplo de la diapositiva: PC-A (172.17.1.10, MAC ab:da:dc:10:10:10) hace ping a **PC-D (192.168.10.30)**, que está detrás de tres routers (R1, R2, R3). Todas las tablas ARP están vacías. El default gateway de PC-A es R1 (172.17.1.1), configurado a mano o por DHCP.



ping de PC-A a PC-D · MAC de cada trama por tramo

Los nombres de interfaz e IP de los enlaces entre routers (10.0.0.0/30 y 10.0.0.4/30) son los del dibujo de la clase; los tramos son los que importan.

Paso a paso en la red de PC-A

1. PC-A arma el paquete IP: origen 172.17.1.10, destino 192.168.10.30. Hace el AND con su máscara y ve que **192.168.10.30 no está en su red**. Le toca mandarlo al gateway.
2. Un ARP Request por 192.168.10.30 **no serviría**: es broadcast y R1 no lo reenvía, así que nunca llegaría a PC-D.
3. PC-A manda un **ARP Request por la IP de su gateway**:

Trama: MAC destino	Trama: MAC origen	ARP: IP origen	ARP: MAC origen	ARP: IP destino	ARP: MAC destino
FF:FF:FF:FF:FF:FF	MAC PC-A	172.17.1.10	MAC PC-A	**172.17.1.1**	?

4. R1 responde con un **ARP Reply** unicast con la MAC de su interfaz eth0.
5. PC-A arma la trama del Echo Request con **MAC destino = MAC de R1**, pero **IP destino = 192.168.10.30**:

MAC destino	MAC origen	IP destino	IP origen	Contenido
MAC R1 eth0	MAC PC-A	192.168.10.30	172.17.1.10	ICMP Echo Request

Esta tabla es la idea más importante de la clase: **la trama va al router, el paquete va al destino**.

Lo que hace cada router

Cuando la trama llega a R1:

1. **Verifica** que la trama sea para él (MAC destino = la suya) y que no tenga errores (CRC).
2. **Desarma la trama** y pasa el paquete IP a la capa de red (desencapsulamiento).
3. **Rutea:** busca la IP destino en su tabla de ruteo y obtiene el **next hop** y la interfaz de salida. Baja el TTL y recalcula el checksum.
4. **Vuelve a encapsular** el paquete en una trama nueva para la red de salida. Si no conoce la MAC del next hop, **hace ARP de nuevo**, ahora preguntando por la IP del next hop (o la del destino final, si ya está directamente conectado).
5. Reenvía la trama.

R2 y R3 hacen exactamente lo mismo. En el último tramo, R3 tiene la red 192.168.10.0/24 directamente conectada, así que hace ARP por **192.168.10.30** y le entrega la trama a PC-D.

Lo que cambia y lo que no

	¿Cambia en cada salto?
IP origen y destino	**No** (salvo que haya NAT)
MAC origen y destino	**Sí**: siempre son las de los dos extremos del tramo actual
TTL y checksum IP	Sí (cada router baja el TTL)

La vuelta

PC-D responde con un Echo Reply a 172.17.1.10. Como también está en otra red, lo manda a **su** gateway (R3). Como en la ida cada equipo anotó al que le preguntó, la respuesta normalmente vuelve **sin ningún ARP nuevo**. Las entradas quedan en las tablas hasta que vence su tiempo de permanencia.

Los ejemplos resueltos de la cátedra

El "Ejemplo ARP" usa PC1 (10.0.0.20) con ping a PC2 (10.0.0.21, misma red) y a PC3 (10.0.1.22, otra red, gateway 10.0.0.1). El "Ruteo_Explicación" usa la topología de la Clase 1: PC-A (192.168.10.3) hace ping a PC-C (192.168.10.130); PC-A hace ARP por su gateway 192.168.10.1, y después Rtr-A hace ARP por **192.168.10.210** (el next hop, Rtr-B) en el enlace 192.168.10.208/30. Mismo razonamiento, otros números. Un detalle de ese ejemplo: se ve un hub en la LAN; un hub, como un switch, reenvía el broadcast a todos, y PC2 lo descarta porque la IP no es la suya.

6. El paquete ARP campo por campo

Las diapositivas "Paquetes ARP" muestran capturas de Wireshark de un Request y un Reply. Un paquete ARP sobre Ethernet mide **28 bytes** y va directamente dentro de la trama (EtherType 0x0806). Como el mínimo de datos de una trama Ethernet es 46 bytes, se rellena con ceros (el "Trailer" que muestra Wireshark).

Campo	Tamaño	Valor típico	Qué significa
Hardware type	2 bytes	1 (Ethernet)	Qué tipo de dirección física se resuelve
Protocol type	2 bytes	0x0800 (IPv4)	Qué tipo de dirección lógica se resuelve
Hardware size	1 byte	6	Largo de la MAC en bytes
Protocol size	1 byte	4	Largo de la IP en bytes
Opcode	2 bytes	**1 = request, 2 = reply**	Pregunta o respuesta
Sender MAC address	6 bytes	MAC del que manda	
Sender IP address	4 bytes	IP del que manda	
Target MAC address	6 bytes	En el Request: **00:00:00:00:00:00** (desconocida)	
Target IP address	4 bytes	La IP por la que se pregunta	

Los campos de tipo y tamaño existen porque ARP se diseñó genérico: podría resolver cualquier dirección de red en cualquier dirección física, no solo IPv4 en Ethernet.

En la captura del Request de la clase, la trama va a **Broadcast (ff:ff:ff:ff:ff:ff)**, Opcode = request (1), Target MAC = 00:00:00:00:00:00 y Wireshark lo resume como "Who has 192.168.0.1? Tell 192.168.0.50". En el Reply, la trama es unicast, Opcode = reply (2) y el resumen es "192.168.0.1 is at …".

La misma diapositiva muestra la **tabla de ruteo** del host (`route -n`): la ruta default 0.0.0.0 via 192.168.0.1 (flags UG: *Up* y *Gateway*) y la red local 192.168.0.0/24 directamente conectada (flag U). Por eso el host pregunta por 192.168.0.1: es su gateway.

7. Tipos de ARP

Además del par Request/Reply normal, hay variantes que reutilizan el mismo formato para otros fines.

Proxy ARP

Un equipo responde un ARP Request en nombre de otro, dando su propia MAC. Normalmente es un router: un host pregunta por una IP que en realidad está en otra red (por ejemplo, porque el host tiene mal configurada la máscara y cree que todo está en su red), y el router responde "soy yo". El host le manda la trama al router, que la reenvía. Funciona, pero esconde errores de configuración y llena las tablas ARP; hoy suele estar deshabilitado.

Gratuitous ARP

Un **ARP Reply de tipo broadcast que nadie pidió** (no responde a ningún Request). Un equipo lo manda para **anunciar o actualizar** su par IP–MAC en las tablas de todos los demás. Casos de uso:

- Cambió la placa de red (nueva MAC, misma IP): avisa para que nadie siga usando la MAC vieja.
- En alta disponibilidad, un servidor de respaldo toma la IP del que se cayó y avisa que ahora esa IP está en su MAC.

Es también la herramienta del ataque de ARP spoofing: como nadie verifica nada, un atacante puede anunciar "la IP del gateway está en mi MAC".

ARP Probe y ARP Announcement

Se usan **cuando a un equipo se le asigna una IP** (estática o por DHCP), para no usar una que ya está en uso. Ambos son broadcast.

1. **ARP Probe:** el equipo pregunta por la IP que quiere usar, con IP origen 0.0.0.0 (todavía no es suya). Si alguien responde, la IP está ocupada y no la usa (conflicto de IP). Es la detección de duplicados que en IPv6 se llama DAD.
2. **ARP Announcement:** si nadie respondió, el equipo **reclama la IP** anunciándola, para que todos actualicen sus tablas.

En la clase de DHCP aparece esto mismo: el cliente debe comprobar que la IP ofrecida no esté en uso "por ejemplo, usando ARP".

8. Preguntas para autoevaluarte

1. ¿Por qué la placa de red necesita la MAC y no le alcanza con la IP?
2. ¿Por qué ARP no puede averiguar la MAC de un equipo en otra red? ¿Y por qué no hace falta?
3. En un ARP Request, ¿qué va en la MAC destino de la trama y qué va en la MAC destino del paquete ARP?
4. ¿Por qué el Request es broadcast y el Reply es unicast?
5. PC-A hace ping a un host de otra red. Escribí las MAC e IP de la trama que sale de PC-A.
6. Un router recibe una trama para él. Enumerá lo que hace hasta reenviar el paquete por otra interfaz.
7. ¿Qué campos cambian en cada salto y cuáles no? ¿Qué los cambiaría?
8. Después del primer ping exitoso entre dos hosts de redes distintas, ¿cuántos ARP se hacen en el segundo ping? ¿Por qué?
9. ¿Qué diferencia hay entre un gratuitous ARP y un ARP Announcement? ¿Y entre un ARP Probe y un Request normal?
10. ¿Por qué se puede considerar ARP de capa 2 y también de capa 3?
11. Compará ARP con Neighbor Discovery de IPv6: ¿a qué dirección va la pregunta en cada uno y por qué importa?

Respuestas rápidas: (3) Trama: FF:FF:FF:FF:FF:FF; paquete ARP: desconocida (ceros). (5) MAC destino = la del gateway, MAC origen = PC-A, IP destino = la del host remoto, IP origen = PC-A. (8) Ninguno, mientras las entradas no venzan.

Clase 4 · DHCP: configuración automática de hosts

Oct 4, 2026 · @Joaquin

Apunte para leer a la par de las diapositivas "DHCP – Dynamic Host Configuration Protocol" (Barbieri, Robles). DHCP resuelve un problema muy concreto: cómo consigue un equipo recién conectado su IP y el resto de su configuración, **sin tener todavía una IP para preguntar**.

1. Qué necesita un host para conectarse

La clase arranca enumerando parámetros. La clave es que **cada parámetro habilita una capacidad distinta**: si falta uno, el host funciona "a medias".

Parámetro	Qué habilita	Qué pasa si falta o está mal
Dirección IP	Tener identidad en la red: poder mandar y recibir paquetes	No hay comunicación IP
Máscara de red	Saber qué destinos están en mi red (se mandan directo, con ARP) y cuáles no (van al gateway)	El host clasifica mal los destinos: intenta ARP por equipos remotos o manda al router cosas locales
Default gateway (router por defecto)	Llegar a **otras redes**	Solo se habla con la propia LAN
Servidor(es) DNS	Usar **nombres** (www.unlp.edu.ar) en vez de IP	Todo funciona con IP, pero no con nombres. Por eso la clase dice que no es "totalmente necesario", aunque en la práctica sí

Un buen ejercicio mental: un host con IP y máscara pero sin gateway puede hacer ping a su vecino pero no a 8.8.8.8; uno con todo menos DNS puede hacer ping a 8.8.8.8 pero no abrir google.com.

2. Formas de asignar una IP

Hay tres formas: que la cargue una persona, que la entregue un servidor o que el host se la invente. DHCP es la segunda, y es la que se usa en casi todas las redes.

Asignación manual o estática

Un administrador configura IP, máscara, gateway y DNS en cada equipo. Funciona para pocos equipos, pero:

- **Es difícil de administrar** en redes medianas o grandes: cientos de equipos para configurar a mano, y errores de tipeo que generan conflictos de IP.
- **No escala:** si cambia el gateway o el DNS, hay que tocar cada equipo. Y las IP de equipos que ya no están quedan "tomadas" en una planilla, sin reutilizarse.

Asignación dinámica

Los nodos **piden** su configuración a un **servidor**. Hubo tres protocolos, cada uno mejorando al anterior:

Protocolo	Qué daba	Limitación
RARP (Reverse ARP)	Solo la IP, a partir de la MAC	Trabaja en capa 2, así que el servidor tiene que estar en la misma LAN. Y no da máscara, gateway ni DNS
BOOTP	IP y otros parámetros, sobre UDP (atraviesa routers con relay)	Asignación fija: el administrador carga una tabla MAC → IP. No reutiliza direcciones
DHCP	Todo lo anterior, con **asignación por tiempo limitado** (lease) y reutilización	Es el que se usa hoy. Es compatible con BOOTP (usa el mismo formato y puertos)

Autoasignación (link-local IPv4)

Si el host no consigue una IP por ningún otro método (por ejemplo, el servidor DHCP está caído), se configura solo una dirección del bloque **169.254.0.0/16** (en Windows se llama APIPA). Elige una al azar y verifica con ARP que nadie la use.

- Solo sirve para comunicaciones **link-local**: con otros equipos del mismo enlace que también estén en 169.254.x.x. No hay gateway, así que no sale a Internet.
- En la práctica, ver una IP 169.254.x.x en una PC es el síntoma típico de "DHCP no respondió". Es la contraparte en IPv4 de la link-local FE80:: de IPv6, con una diferencia importante: en IPv6 la link-local existe siempre, en IPv4 solo como último recurso.

3. DHCP en general

DHCP (RFC 2131, marzo de 1997) es un protocolo **cliente/servidor sobre UDP**: el cliente es el host que necesita configuración, el servidor es el que la tiene.

Puertos: cliente 68, servidor 67

En la mayoría de los protocolos solo el servidor tiene puerto fijo y el cliente usa uno efímero al azar (Clase 6). DHCP es una excepción: **el cliente también tiene puerto fijo, el 68**. ¿Por qué? Porque al principio el servidor muchas veces tiene que responder **por broadcast** (el cliente todavía no tiene IP). Un broadcast le llega a todos los equipos de la LAN; si el cliente usara un puerto al azar, el servidor no sabría a qué puerto mandarlo. Con el 68 fijo, solo los procesos cliente DHCP lo reciben.

¿Y por qué UDP y no TCP? TCP necesita un saludo de 3 vías entre dos IP conocidas, y el cliente no tiene IP. Además, TCP no admite broadcast. Un intercambio de pocos mensajes cortos es exactamente el caso de uso de UDP.

Los dos componentes de DHCP

1. **Un protocolo** para enviar parámetros de configuración del servidor a los hosts (el formato de los mensajes y el intercambio DORA).
2. **Un mecanismo para alocar direcciones**: cómo el servidor decide qué IP darle a cada uno y por cuánto tiempo.

Los tres mecanismos de alocación

Mecanismo	Qué hace	Uso típico
Automática	El servidor elige una IP del pool y se la asigna al cliente **para siempre**	Redes donde los equipos no rotan
Dinámica	El servidor asigna una IP **por un tiempo limitado** (lease). Si el cliente no la renueva, vuelve al pool	Lo más común: WiFi de un bar, una facultad, tu casa
Manual	El administrador decide qué IP le toca a cada cliente (normalmente por MAC); DHCP solo **transporta** esa decisión	Equipos que necesitan IP fija pero se quiere administrar centralizado ("reserva DHCP")

La recomendación de la clase

No usar DHCP dinámico en equipos que no deberían cambiar su IP: routers, impresoras, teléfonos IP, servidores. Si la IP de la impresora cambia, todas las PCs que la tenían configurada dejan de encontrarla. Y el router es el gateway de todos: si su IP cambiara, la red entera se quedaría sin salida. Para estos equipos se usa configuración estática o la alocación manual de DHCP.

4. Conceptos: pool, lease y opciones

El servidor administra un **conjunto de IP disponibles** (pool) y las **presta** por un tiempo (lease). Prestar en vez de regalar es lo que permite reutilizar direcciones.

- **Pool de direcciones:** el rango de IP que el servidor puede asignar. Ej.: 192.168.10.100 a 192.168.10.200. Las IP fijas (router, impresoras) se dejan fuera del pool para que no se repitan.
- **Lease (préstamo):** el período durante el cual una IP está asignada a un cliente. Es configurable: horas en una oficina, minutos en un aeropuerto (donde la gente va y viene y hay que liberar IP rápido).
- **Qué pide el cliente:** una IP y otros parámetros: **máscara, default gateway, servidor DNS**.
- **Qué más puede dar el servidor:** información adicional en forma de **opciones**: servidores NTP (hora), SMTP y POP (correo), HTTP (proxy), nombre de dominio, etc.
- **Parámetros por tiempo limitado:** con alocación dinámica, todo lo que el servidor da vale solo mientras dure el lease.
- **Configuración según el cliente:** el servidor puede decidir qué darle en base a lo que el cliente informa (por ejemplo, su tipo de equipo o de qué red viene).
- **Renovación:** el cliente puede pedir extender el lease con nuevas solicitudes antes de que venza (sección 6).
- **Liberación:** el cliente puede avisar que ya no usa la IP (*DHCP Release*) para que vuelva al pool.
- **Identificación del cliente:** el servidor lo reconoce por su **MAC** (el campo *chaddr*, Client hardware address) o por un **DUID** (DHCP Unique Identifier, RFC 4361), que no depende de la placa de red. Es lo que permite que el mismo equipo reciba siempre la misma IP en la alocación manual, o que el servidor le intente devolver la misma IP que tenía.

5. El proceso DORA

El cliente consigue su configuración en **4 mensajes: Discover, Offer, Request y ACK** (de ahí "DORA"). Son 4 y no 2 porque puede haber **varios servidores** en la red: hace falta un mensaje para que el cliente elija y otro para que el elegido confirme.



DORA con dos servidores DHCP en la LAN · 4 mensajes

1. DHCP Discover (cliente → todos)

- IP origen **0.0.0.0** ("este host": todavía no tiene IP) e IP destino **255.255.255.255** (limited broadcast: "todos en mi red"). MAC destino FF:FF:FF:FF:FF:FF. UDP del puerto 68 al 67.
- Significa: "necesito configuración, ¿hay algún servidor DHCP?".
- El cliente indica con el **flag de broadcast** si puede o no recibir mensajes unicast antes de tener IP configurada. Si dice que **no puede**, el servidor le responde por **broadcast**; si puede, le responde **unicast a la IP que le está ofreciendo** (y a su MAC). En la captura de la clase el flag está en 0 ("Unicast"), y por eso el Offer va de 192.168.10.1 a 192.168.10.160.

2. DHCP Offer (cada servidor → cliente)

- **Cada** servidor que escuchó el Discover responde con una IP de **su pool** y los parámetros. En el dibujo, Server 2 ofrece IP Y y Server 1 ofrece IP X.
- Cada servidor reserva provisoriamente la IP que ofreció.

3. DHCP Request (cliente → todos)

- El cliente elige una oferta (normalmente la primera que llega) y responde con un Request que dice "quiero la IP X **del Server 1**".
- Sigue yendo **por broadcast** desde 0.0.0.0, aunque ya tenga una oferta. ¿Por qué? Porque si hay varios servidores, **todos tienen que enterarse de a quién eligió**: Server 2 ve en el Request que no fue seleccionado y libera la IP Y que había reservado.
- Antes de aceptar la IP, el cliente debe **comprobar que no esté en uso**, por ejemplo con un ARP Probe (Clase 3). Si alguien responde, la rechaza (mensaje *DHCP Decline*) y empieza de nuevo.

4. DHCP ACK (servidor elegido → cliente)

- El servidor elegido confirma la asignación y manda los **parámetros de configuración** definitivos: IP, máscara, gateway, DNS, duración del lease y las opciones que corresponda. Termina el proceso y el cliente configura su interfaz.
- Si el servidor ya no puede darle esa IP, responde un **NAK** y el cliente arranca de nuevo con un Discover.

La captura de la clase

#	Origen	Destino	Mensaje
1	0.0.0.0	255.255.255.255	DHCP Discover, Transaction ID 0xf70a2e61
2	192.168.10.1	192.168.10.160	DHCP Offer, mismo Transaction ID
3	0.0.0.0	255.255.255.255	DHCP Request, mismo Transaction ID
4	192.168.10.1	192.168.10.160	DHCP ACK, mismo Transaction ID

Campos del mensaje que se ven en el detalle del Discover (formato heredado de BOOTP):

Campo	Valor en el Discover	Para qué sirve
Message type	Boot Request (1)	1 = de cliente a servidor, 2 = de servidor a cliente (Boot Reply)
Hardware type / length	Ethernet (0x01) / 6	Tipo y largo de la dirección física
Hops	0	Cuántos relays atravesó

Campo	Valor en el Discover	Para qué sirve
Transaction ID	0xf70a2e61	Número al azar que elige el cliente; **todos los mensajes de un mismo intercambio lo llevan**. Así el cliente sabe qué respuestas son para él, aunque viajen por broadcast
Seconds elapsed	0	Tiempo desde que el cliente empezó
Bootp flags	0x0000 (Unicast)	El flag de broadcast explicado arriba
Client IP address (ciaddr)	0.0.0.0	La IP actual del cliente (se usa al renovar)
Your IP address (yiaddr)	0.0.0.0	La IP que el **servidor le da** al cliente: se completa en Offer y ACK
Next server IP (siaddr)	0.0.0.0	Servidor siguiente (por ejemplo, para boot por red)
Relay agent IP (giaddr)	0.0.0.0	IP del relay, si lo hay (sección 7)
Client MAC address (chaddr)	00:50:79:66:68:02	Identifica al cliente
Opciones	Message type = Discover, etc.	El tipo DHCP real (Discover, Offer, Request, ACK…) va en una opción, porque el formato BOOTP solo distingue request y reply

La captura también muestra una etiqueta **802.1Q VLAN, ID 10**: la red del ejemplo está en una VLAN. No cambia nada del funcionamiento de DHCP.

6. Renovación y liberación del lease

La segunda diapositiva de "Intercambio de mensajes" muestra lo que pasa después: el cliente **renueva** su lease con solo **2 mensajes** (Request + ACK) y, al final, la **libera** con un Release.

Renovación

- Ya no hace falta Discover ni Offer: el cliente **ya tiene IP y sabe quién es su servidor**. Le manda un **DHCP Request unicast** directamente al Server 1 pidiendo extender la IP X, y el servidor responde con un **ACK** que reinicia el lease.

- ¿Cuándo? La RFC fija dos tiempos (la clase no los nombra, pero ayudan a entender): en **T1 = 50 %** del lease el cliente intenta renovar con su servidor; si no responde, en **T2 = 87,5 %** prueba por broadcast con **cualquier** servidor (*rebinding*). Si el lease vence sin respuesta, el cliente pierde la IP y vuelve a empezar con un Discover.
- Mientras el equipo esté encendido y el servidor responda, conserva siempre la misma IP.

Liberación

- El cliente manda un **DHCP Release** al servidor con la IP que ya no va a usar (por ejemplo, al apagarse o al ejecutar `dhclient -r` en Linux o `ipconfig /release` en Windows).
- El servidor la devuelve al pool sin esperar a que venza el lease. No hay respuesta.
- Si el equipo se desconecta sin avisar (lo más común, por ejemplo al irse de un WiFi), la IP queda ocupada hasta que vence el lease. Por eso los lugares con mucha rotación usan leases cortos.

7. DHCP Relay

El Discover es broadcast y **los routers no reenvían broadcasts**, así que en principio haría falta un servidor DHCP en cada red. El **DHCP Relay** evita eso: el router de cada red escucha los broadcasts DHCP y **los reenvía por unicast** a un servidor central.

Cómo funciona

1. Un cliente de la red de "Bangkok" (en el dibujo de la clase) manda su Discover por broadcast.
2. El router de esa red, configurado como **relay agent**, lo recibe. Completa el campo **giaddr** (Relay agent IP address) con **su propia IP en la interfaz por donde llegó el Discover**, incrementa Hops y lo reenvía **unicast** al servidor DHCP (en el dibujo, en "Berlin", a través de la WAN).
3. El servidor responde el Offer **al relay** (a la IP del giaddr), y el relay hace el **proceso inverso**: lo entrega al cliente en la LAN local (por broadcast o unicast según el flag).
4. Lo mismo para Request y ACK.

Cómo sabe el servidor de qué pool asignar

Un servidor central atiende muchas redes, así que tiene **un pool por red**. Si un cliente de 10.1.1.0/24 recibiera una IP de 10.2.2.0/24, no podría comunicarse. El servidor lo resuelve mirando el **giaddr**: la IP del relay pertenece a la red del cliente, así que el servidor elige el

pool que contiene esa IP. Si el giaddr es 0.0.0.0, el cliente está en la misma red que el servidor.

Por qué se hace así

Tener **un servidor DHCP centralizado** es más fácil de administrar (un solo lugar para pools, reservas y logs) que tener uno en cada sucursal. En equipos Cisco, configurar el relay es un solo comando en la interfaz (`ip helper-address <IP del servidor>`).

8. Preguntas para autoevaluarte

1. Un host tiene IP y máscara correctas pero no tiene gateway. ¿Qué puede hacer y qué no?
2. ¿Qué problema de RARP resolvió BOOTP, y qué problema de BOOTP resolvió DHCP?
3. Ves una IP 169.254.37.12 en una PC. ¿Qué te dice eso?
4. ¿Por qué el cliente DHCP usa un puerto fijo (68) y no uno efímero?
5. ¿Por qué DHCP corre sobre UDP y no sobre TCP?
6. ¿Qué IP origen y destino lleva el Discover? ¿Por qué esas?
7. ¿Por qué el Request va por broadcast si el cliente ya sabe qué servidor eligió?
8. ¿Para qué sirve el Transaction ID?
9. ¿Por qué la renovación usa 2 mensajes y la obtención inicial 4?
10. ¿Qué diferencia hay entre alocación automática, dinámica y manual? ¿Cuál usarías para una impresora?
11. Tenés 5 sucursales y un solo servidor DHCP. ¿Qué configurás en los routers y cómo sabe el servidor qué IP darle a cada cliente?
12. ¿Qué pasa con una IP prestada si el equipo se desconecta sin mandar Release?

Respuestas rápidas: (3) Que no obtuvo respuesta de ningún servidor DHCP y se autoasignó una link-local. (6) Origen 0.0.0.0 (no tiene IP), destino 255.255.255.255 (no sabe dónde está el servidor). (11) Relay agent en cada router; el servidor elige el pool según el giaddr.

Clase 5 · NAT: traducción de direcciones y puertos

Oct 4, 2026 · @Joaquin

Apunte para leer a la par de las diapositivas "NAT – Arquitectura y Funcionamiento" (Barbieri, Robles). Casi toda la clase son dibujos de ejemplo; acá están explicados con sus tablas.

1. El problema: IPv4 se queda sin direcciones

IPv4 tiene el espacio de direcciones **casi agotado**. La solución definitiva es IPv6, pero mientras tanto se usaron varias **soluciones temporales**, cada una atacando una parte del problema. La clase las ubica así:

Técnica	Qué ataca	Cómo ayuda	Clase
Subnetting fijo y variable (VLSM)	Desperdicio dentro de una organización	Bloques del tamaño justo	1
CIDR	Escalabilidad del ruteo	Bloques de cualquier tamaño y agregación de rutas	1
DHCP	Direcciones "tomadas" por equipos que no están	**Reuso**: solo tiene IP quien está conectado, y facilita la administración	4
NAT	Que cada equipo necesite una IP pública	Muchos equipos con IP **privada** salen a Internet con **pocas o una sola IP pública**	5
IPv6	Todo lo anterior	2^{128} direcciones: deja de haber escasez	2

NAT es, por lejos, la que más estiró la vida de IPv4: en tu casa, todos los dispositivos comparten una sola IP pública gracias a NAT.

2. Qué es NAT y por qué hace falta

NAT (Network Address Translation) es **reescribir las direcciones IP de los paquetes al pasar por un router**: las de un espacio privado (que no se puede rutear en Internet) se cambian por direcciones de un espacio público, y a la vuelta al revés.

Las direcciones privadas (RFC 1918)

Bloque	Equivale a
10.0.0.0/8	1 clase A
172.16.0.0/12	16 clases B (172.16 a 172.31)
192.168.0.0/16	256 clases C

Cualquier organización puede usarlas sin pedir permiso. Eso es lo que permite que **el mismo conjunto de direcciones se reuse en distintas partes de Internet**: tu casa y la del vecino pueden usar las dos 192.168.0.0/24.

Por qué una IP privada no puede salir así como está

La clase lo resume en "problemas con direcciones privadas":

- **Se asignan a muchas redes a la vez.** Si llegara a Internet un paquete con destino 192.168.0.30, ¿a cuál de los millones de 192.168.0.30 del mundo iría? Las rutas se confundirían. Y la respuesta a un paquete con **origen** privado nunca podría volver.
- Por eso **los routers de borde las filtran**: no dejan salir ni entrar paquetes con esas direcciones.
- **Algunos protocolos no funcionan bien** detrás de NAT, como **FTP** o **VoIP** (SIP). El motivo: esos protocolos escriben direcciones IP y puertos **dentro de sus propios datos** ("conectate a mí en 192.168.0.30, puerto 5000"), y NAT solo cambia las cabeceras, no el contenido. El otro extremo recibe una IP privada inservible.

Definición formal y transparencia

- NAT tradicional está definido en la **RFC 3022**, que deja obsoleta a la RFC 1631. La terminología está en la RFC 2663.
- Es **transparente para los usuarios finales**: ni la PC ni el servidor saben que hubo traducción. La PC cree que habla desde 192.168.0.30; el servidor cree que le habla una IP pública.

Pregunta de la clase: **¿cómo accedemos a Internet desde nuestro hogar?** El proveedor le da una sola IP pública al router de tu casa; todos tus dispositivos tienen IP privadas (192.168.0.x o similar, por DHCP del mismo router) y el router hace NAT con esa única IP pública. Es el caso de **masquerading** (sección 7).

3. El proceso de traslación

La traducción la hace un router (generalmente el **router de borde**, el que conecta la red interna con Internet) usando una **tabla de traslaciones** que recuerda qué dirección interna corresponde a cuál externa.

Las reglas del proceso

- **Lo hacen los routers de borde**, porque son el punto por donde el tráfico cruza de "adentro" (privado) a "afuera" (público).
- **Se mantienen tablas de traslaciones.** Cada entrada dice "lo que adentro es X, afuera es Y". Sin esa tabla, el router no sabría a quién entregar las respuestas.
- **Todo el tráfico de una sesión tiene que pasar por el mismo router NAT.** Si la ida sale por un router y la vuelta entra por otro, el segundo no tiene la entrada en su tabla y descarta la respuesta. Esto complica las redes con dos salidas a Internet.

Qué campos modifica

NAT no solo cambia una dirección:

1. **La IP** (origen al salir, destino al entrar).
2. En NAPT, también **los puertos** de TCP o UDP.
3. **El checksum de la cabecera IP**, porque la cabecera cambió.
4. **El checksum de la capa de transporte (TCP/UDP)**, aunque NAT sea "de capa 3". ¿Por qué? Porque esos checksums se calculan sobre una **pseudocabecera que incluye las IP origen y destino** (Clase 6). Si NAT cambia la IP y no actualiza el checksum de TCP, el destino descartaría el segmento como corrupto.

Los tipos que ve la clase ("Traditional NAT")

- **NAT básico** (solo cambia IP): **estático** o **dinámico**.
- **NAPT** (cambia IP y puertos): **dinámico sobre un pool** de IP públicas, o **dinámico sobre una sola dirección** (overload / masquerade).

Vocabulario de las tablas

En las tablas de la clase aparecen estas columnas:

Columna	Qué es	Ejemplo
Interna (local)	La dirección real del host, privada, como se ve **adentro**	192.168.0.30

Columna	Qué es	Ejemplo
Externa (local)	La dirección con que ese host se ve **afuera**, después de traducir	193.168.0.30
Externa global	La dirección del **otro extremo** (el servidor de Internet)	150.0.0.1:80
Timer	Cuánto falta para que la entrada se borre si no hay tráfico	0:22

En la terminología de Cisco se las llama *inside local*, *inside global* y *outside global*.

4. NAT básico estático

NAT básico **solo reescribe direcciones IP** (no toca puertos). En su forma estática es **uno a uno (one-to-one)**: cada IP privada tiene asignada **para siempre** una IP pública fija, cargada a mano en la tabla.

Características

- **Requiere tantas IP públicas como privadas** que necesiten salir. No ahorra direcciones; lo que da es la posibilidad de usar direccionamiento privado adentro.
- **Permite acceso en ambas direcciones.** Como la correspondencia es fija, alguien de afuera puede iniciar una conexión hacia la IP pública y el router sabe a qué host interno entregarla. Por eso se usa para **servidores** internos que tienen que ser accesibles desde Internet.

El ejemplo de la clase

La red de la derecha tiene dos LAN privadas detrás del router NAT (que sale a Internet por 200.3.4.2/30): **Red D 192.168.0.0/24** (hosts 192.168.0.30 y .31) y **Red C 10.10.3.0/24** (host 10.10.3.10). El destino es un servidor en **150.0.0.1**, en otra red de Internet. Tabla estática del router:

Interna	Externa
192.168.0.30	193.168.0.30
192.168.0.31	193.168.0.31
10.10.3.10	193.168.1.31

Las 193.168.x.x son las IP públicas que la organización tiene asignadas (otra vez, números de ejemplo).

Paso a paso, cuando 192.168.0.30 le manda algo a 150.0.0.1:

Tramo	IP origen	IP destino	Qué pasó
1. Red D → router NAT	192.168.0.30	150.0.0.1	El paquete sale con su IP privada
2. Router NAT → Internet	**193.168.0.30**	150.0.0.1	El router busca 192.168.0.30 en la tabla y **cambia la IP origen** por la externa
3. Internet → router NAT (respuesta)	150.0.0.1	**193.168.0.30**	El servidor responde a la IP que vio, la pública
4. Router NAT → Red D	150.0.0.1	**192.168.0.30**	El router busca 193.168.0.30 en la columna externa y **cambia la IP destino** por la interna

Un detalle del dibujo que vale la pena notar: del lado del servidor **también hay una red 192.168.0.0/24 con un host 192.168.0.30**. No hay conflicto, porque ninguna de las dos 192.168.0.30 sale a Internet como tal. Es exactamente el "reuso de direcciones en distintas partes de Internet" que permite NAT.

Regla para recordar sin memorizar: **al salir se traduce el origen; al entrar se traduce el destino.** Siempre se traduce la dirección del lado privado.

5. NAT básico dinámico

En NAT dinámico el router tiene un **pool de IP públicas** y se las va **prestando** a los hosts internos a medida que salen, por un tiempo. Sigue siendo uno a uno, pero no fijo.

- **No hace falta igual cantidad de públicas que de privadas**: alcanza con tantas como hosts que salgan **al mismo tiempo**.
- A cambio, **cada entrada necesita un timer**: si un host deja de generar tráfico, su entrada vence y la IP pública vuelve al pool para otro.
- **Limita el acceso simultáneo al tamaño del pool.** Si el pool tiene 14 IP y hay 15 hosts queriendo salir a la vez, el 15.º no puede. La pregunta de la clase ("¿qué cantidad de usuarios necesitan acceder a Internet simultáneamente?") es la que define el tamaño del pool.
- Desde afuera **no se puede iniciar una conexión** hacia adentro de forma confiable: la IP pública de un host cambia y solo existe mientras él tenga tráfico.

El ejemplo de la clase

Pool: **193.168.0.0/28** (16 direcciones). Tabla en un momento dado:

Interna	Externa	Timer
10.10.3.10	193.168.0.1	0:22
192.168.0.30	193.168.0.2	0:45

10.10.3.10 salió primero y recibió la primera libre (.1); 192.168.0.30 salió después y recibió .2. El paquete de 192.168.0.30 a 150.0.0.1 sale como [193.168.0.2 → 150.0.0.1] y la respuesta [150.0.0.1 → 193.168.0.2] se traduce de vuelta a 192.168.0.30. Si mañana vuelve a salir, puede que le toque otra.

6. NAPT: traducir también los puertos

NAPT (Network Address **Port** Translation), también llamado **PAT**, usa los **puertos** de TCP o UDP para que **muchos hosts compartan la misma IP pública al mismo tiempo**: es **uno a muchos (one-to-many)**.

Por qué hace falta

Con NAT básico, si tenés un pool chico (o una sola IP), solo pueden salir tantos hosts como IP haya. Para que más hosts compartan una IP, el router necesita otro dato para distinguir a quién pertenece cada respuesta: el **puerto**. Por eso la clase dice que hay que "trabajar o modificar campos de la capa de transporte o del payload".

Cómo funciona

- La tabla guarda, por cada conexión: **protocolo**, **IP y puerto internos**, **IP y puerto externos** que se asignaron, y la **IP y puerto del destino**.
- **Se intenta conservar el puerto origen** del host. Si ya está usado por otra conexión con esa misma IP externa, el router **elige otro puerto libre** (sección 7).
- Si el protocolo **no tiene puertos**, se usa otro campo que cumpla esa función. Para ICMP (ping) se usa el **Identifier** del Echo Request.
- Las entradas se borran por **timer** o siguiendo la **sesión** del protocolo (por ejemplo, cuando se ve el FIN de TCP).
- **Rompe los límites de las capas:** un router, que es un dispositivo de capa 3, mira y modifica datos de capa 4. Es una de las críticas a NAT.

Dos variantes: **NAPT sobre un pool** de IP públicas, o **NAPT sobre una sola IP** (overloading / masquerading).

NAPT con pool: el ejemplo de la clase

Pool: 193.168.0.0/28. Tabla:

Interna local	Externa local	Externa global	Timer
192.168.0.30:1500	193.168.0.3:1500	150.0.0.1:80	…

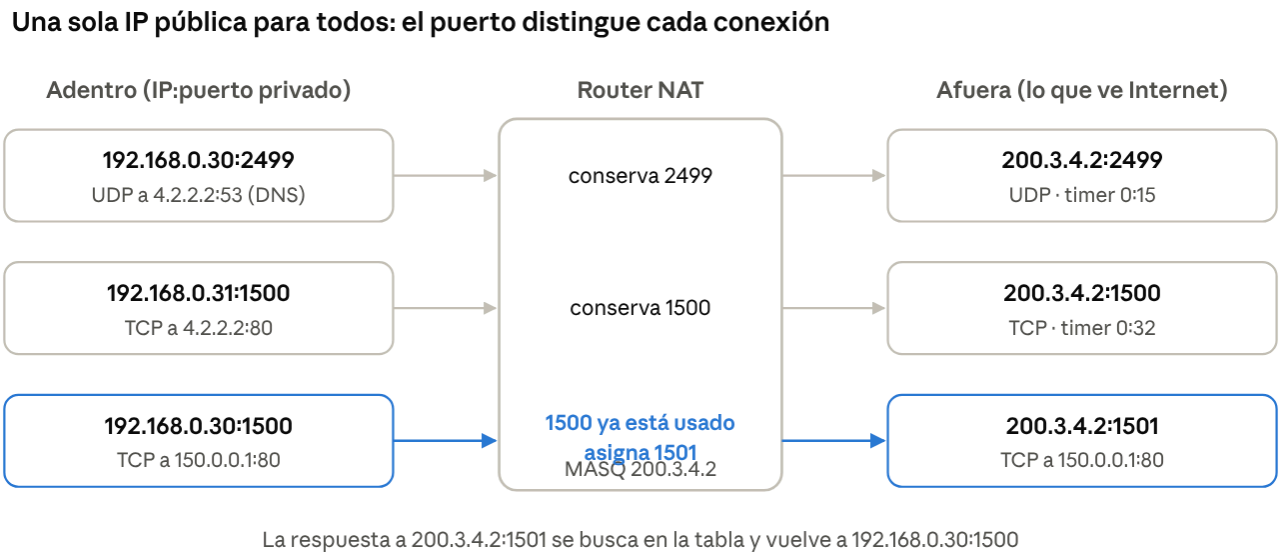
El host 192.168.0.30 abre una conexión TCP desde su puerto 1500 al puerto 80 (web) de 150.0.0.1. El router le asigna 193.168.0.3 del pool y **conserva el puerto 1500** porque está libre.

Tramo	Paquete (origen → destino)
Ida, adentro	192.168.0.30:1500 → 150.0.0.1:80
Ida, afuera	**193.168.0.3:1500** → 150.0.0.1:80
Vuelta, afuera	150.0.0.1:80 → 193.168.0.3:1500
Vuelta, adentro	150.0.0.1:80 → **192.168.0.30:1500**

(En las diapositivas se escribe como "TCP,1500,80" o "TCP,80,1500": protocolo, puerto origen y puerto destino del paquete en ese tramo.)

7. NAPT con una sola IP: overloading o masquerading

Es NAPT llevado al extremo: el router tiene **una única IP pública** (en el ejemplo, la de su interfaz externa, **200.3.4.2**) y **todos** los hosts internos salen con ella. Lo único que diferencia una conexión de otra, del lado de afuera, es el **puerto externo**. Cisco lo llama *overloading* ("sobrecargar" una IP); Linux lo llama *masquerading* (los hosts internos se "enmascaran" detrás del router). Es exactamente lo que hace el router de tu casa.



NAPT con una sola IP (masquerading) · el ejemplo de la clase

La tabla del ejemplo

Protocolo	Interna local	Externa local	Externa global	Timer
UDP	192.168.0.30:2499	200.3.4.2:2499	4.2.2.2:53	0:15
TCP	192.168.0.31:1500	200.3.4.2:1500	4.2.2.2:80	0:32
TCP	192.168.0.30:1500	200.3.4.2:**1501**	150.0.0.1:80	…

Leída fila por fila:

1. **192.168.0.30 hace una consulta DNS** (UDP al puerto 53 de 4.2.2.2) desde su puerto 2499. Nadie usa el 2499 afuera, así que el router lo **conserva**.
2. **192.168.0.31 abre una conexión web** (TCP al 80) desde su puerto 1500. El 1500 externo está libre: se conserva.
3. **192.168.0.30 abre otra conexión TCP, también desde su puerto 1500**, hacia 150.0.0.1:80. Adentro no hay problema (son dos hosts distintos con el mismo puerto), pero **afuera los dos se verían como 200.3.4.2:1500**. Si el router lo dejara así, cuando llegue una respuesta a 200.3.4.2:1500 no sabría si es para .30 o para .31. Por eso **reemplaza el puerto por uno libre, el 1501**.

El paquete de la fila 3 sale entonces como **[200.3.4.2 → 150.0.0.1 (TCP, 1501, 80)]**: cambiaron la IP origen **y** el puerto origen. La respuesta llega como [150.0.0.1 → 200.3.4.2 (TCP, 80, 1501)], el router busca "TCP, puerto externo 1501" en la tabla y la reescribe a [150.0.0.1 → 192.168.0.30 (TCP, 80, 1500)].

Lo que hay que entender

- **La clave de búsqueda a la vuelta es (protocolo, puerto externo).** La IP externa es siempre la misma, así que no sirve para distinguir. Por eso el puerto externo no puede repetirse dentro de un mismo protocolo. Fijate que la fila 1 usa UDP: aunque coincidiera con un puerto TCP no habría conflicto, porque el protocolo también está en la tabla.
- Un detalle fino: las filas 2 y 3 van a **destinos distintos** (4.2.2.2 y 150.0.0.1), así que en teoría un NAT podría distinguirlas por el destino y reusar el 1500. El ejemplo de la clase sigue el criterio más simple, que es el que te van a pedir: **si el puerto externo está ocupado, se elige otro**.
- **Cuántas conexiones aguanta**: un puerto es un número de 16 bits, así que hay unos 65.000 puertos por protocolo por IP pública. Por eso una sola IP alcanza para una casa o una oficina.
- **Solo funciona para tráfico iniciado desde adentro.** Las entradas se crean cuando un host interno sale. Si alguien de Internet le manda un paquete a 200.3.4.2:80 sin que exista una entrada, el router no sabe a qué host interno entregarlo y lo descarta. Eso lleva a la sección siguiente.

8. Port forwarding: dejar entrar tráfico desde afuera

El problema: con masquerading nadie de afuera puede iniciar una conexión hacia adentro. Pero si tenés un servidor web o de mail en la red privada (192.168.0.1), querés que Internet llegue a él, y solo tenés una IP pública.

La solución: cargar a mano en la tabla de NAPT entradas **estáticas** que digan "lo que llegue a tal puerto de mi IP pública, mandalo a tal IP y puerto internos". Es un **mapeo reverso estático de puertos**. La clase remarca que **no hace falta NAT estático** (que gastaría una IP pública entera por servidor): se hace con NAPT, compartiendo la misma IP que usa todo el mundo para salir.

La tabla del ejemplo

Protocolo	Interna local	Externa local	Externa global	Timer
TCP	192.168.0.1:80	200.3.4.2:80	0.0.0.0:*	static
TCP	192.168.0.1:25	200.3.4.2:**1025**	0.0.0.0:*	static

- **Externa global 0.0.0.0:*** significa "cualquier IP y cualquier puerto": la entrada vale para quien sea que se conecte desde Internet, porque no sabés de antemano quién será el cliente.
- **Timer "static"**: no vence nunca, porque no la creó el tráfico sino el administrador.

- **Fila 1**: lo que llegue al puerto 80 de 200.3.4.2 va al servidor web 192.168.0.1:80. Mismo puerto afuera y adentro.
- **Fila 2**: lo que llegue al puerto **1025** de 200.3.4.2 va al **25** (SMTP, mail) de 192.168.0.1. Los puertos **no tienen por qué coincidir**. Eso sirve, por ejemplo, para publicar dos servidores del mismo tipo con una sola IP (uno en el 80 y otro en el 8080), o cuando el puerto estándar ya está tomado.

Paso a paso

Un cliente 150.0.0.1 quiere ver la web de la organización:

Tramo	Paquete	Qué pasó
Llega al router	150.0.0.1 → **200.3.4.2** (TCP, 2500, 80)	El cliente solo conoce la IP pública
Sale hacia la LAN	150.0.0.1 → **192.168.0.1** (TCP, 2500, 80)	El router encuentra "TCP, puerto externo 80" en la tabla y cambia **el destino**
Respuesta del servidor	192.168.0.1 → 150.0.0.1 (TCP, 80, 2500)	El servidor responde a quien le habló
Sale a Internet	**200.3.4.2** → 150.0.0.1 (TCP, 80, 2500)	El router cambia **el origen** por la IP pública

Es la misma regla de siempre (se traduce el lado privado), solo que ahora la conexión empieza afuera. Con el puerto 1025 sería igual, pero además cambiaría el puerto: el paquete llega a 200.3.4.2:1025 y entra a 192.168.0.1:25.

En el router de tu casa esto es la opción "Port forwarding" o "Virtual server" del menú: se usa para hostear un servidor de juegos, una cámara o un servidor propio.

9. Comparación y conclusiones

Los cuatro tipos lado a lado

	Estático	Dinámico	NAPT con pool	Masquerading
Qué traduce	IP	IP	IP y puerto	IP y puerto
Relación	1 a 1 fija	1 a 1 temporal	muchos a pocos	muchos a 1

	Estático	Dinámico	NAPT con pool	Masquerading
IP públicas necesarias	una por host	una por host que sale **a la vez**	pocas	**una**
Timer en las entradas	no	sí	sí (o fin de sesión)	sí (o fin de sesión)
¿Se puede entrar desde afuera?	**sí**	no (en la práctica)	no	no, salvo **port forwarding**
Uso típico	servidores internos publicados	organización con un pool grande	organización con pocas IP	casa, oficina chica

La progresión tiene lógica: cada fila hacia la derecha **ahorra más IP públicas** a cambio de **más estado en el router** (timers, puertos) y **menos posibilidad de entrar desde afuera**.

Las conclusiones de la clase, explicadas

- **NAT/NAPT resuelve *temporalmente* la escasez de IPv4.** No crea direcciones; las comparte. La solución de fondo sigue siendo IPv6.
- **Algunos servicios no funcionan sin modificarlos.** Los que llevan IP o puertos dentro de sus datos (FTP, VoIP/SIP), y en general cualquier cosa que necesite recibir conexiones (servidores, P2P). Los routers suelen incluir módulos especiales (ALG, *application level gateways*) que leen y corrigen esos datos, protocolo por protocolo.
- **Da una "sensación" de seguridad, aunque no siempre es cierta.** Como desde afuera no se puede iniciar conexiones, NAT bloquea de rebote mucho tráfico entrante y parece un firewall. Pero **no lo es**: no filtra contenido, no protege de lo que un host interno descargue por su cuenta, y con port forwarding o técnicas de atravesar NAT el tráfico entra igual. La seguridad tiene que venir de un firewall de verdad.
- **Rompe la idea original de IP: "end-to-end".** IP se diseñó para que cualquier host pudiera hablar directamente con cualquier otro con su propia dirección. Con NAT, el host interno no es alcanzable y su dirección real no viaja; hay un intermediario que reescribe los paquetes y guarda estado. Por eso también hay que pasar siempre por el mismo router (sección 3).
- **Firewalls más complejos.** Las reglas de filtrado tienen que tener en cuenta en qué momento se traduce (antes o después del NAT, con la IP privada o con la pública) y el estado de la tabla de traslaciones.

Bibliografía que cita la clase

Kurose y Ross (*Computer Networking*), los cursos CCNA de Cisco, y las RFC **1918** (direcciones privadas), **2663** (terminología de NAT) y **3022** (NAT tradicional).

10. Preguntas para autoevaluarte

1. ¿Por qué un paquete con IP origen 192.168.1.5 no puede salir tal cual a Internet? Da dos razones.
2. NAT es "de capa 3". ¿Por qué igual tiene que recalcular el checksum de TCP o UDP, incluso en NAT básico (que no toca puertos)?
3. Una empresa tiene 200 empleados, de los cuales nunca más de 40 navegan a la vez, y 50 IP públicas. ¿Le alcanza NAT dinámico? ¿Y si mañana son 60 a la vez?
4. ¿Qué tipo de NAT usarías para publicar un servidor interno si te sobran IP públicas? ¿Y si tenés una sola?
5. En masquerading con IP 200.3.4.2, los hosts 10.0.0.5 y 10.0.0.6 abren una conexión TCP desde su puerto 4000. Escribí cómo quedarían las dos entradas de la tabla.
6. ¿Por qué el router mira el protocolo además del puerto para encontrar una entrada?
7. En port forwarding, ¿qué significa 0.0.0.0:* en la columna "externa global"? ¿Y "static" en el timer?
8. ¿Por qué se dice que NAT da una falsa sensación de seguridad?
9. Tu red tiene dos routers de salida a Internet, cada uno con NAT. ¿Qué problema puede aparecer?
10. ¿Cómo hace NAPT con un ping, si ICMP no tiene puertos?

Respuestas rápidas

1. Porque esa dirección la usan millones de redes a la vez (la respuesta no sabría a dónde volver y las rutas serían ambiguas), y porque los routers de borde filtran las direcciones privadas.
2. Porque el checksum de TCP/UDP se calcula sobre una pseudocabecera que incluye las IP origen y destino. Si cambia la IP, el checksum queda mal y el destino descarta el segmento.
3. Sí: dinámico necesita tantas IP como hosts que salen al mismo tiempo (40 de 50). Con 60 simultáneos, 10 no podrían salir: habría que pasar a NAPT.
4. Con IP de sobra, NAT estático (uno a uno, permite entrar). Con una sola, masquerading más port forwarding.
5. TCP 10.0.0.5:4000 ↔ 200.3.4.2:4000, y TCP 10.0.0.6:4000 ↔ 200.3.4.2:otro puerto libre (por ejemplo 4001). Afuera no pueden repetirse.

6. Porque TCP y UDP tienen espacios de puertos independientes: el puerto 2499 de UDP y el 2499 de TCP son cosas distintas.
7. Que la entrada vale para cualquier cliente de Internet, sea cual sea su IP y puerto. "Static" quiere decir que la cargó el administrador y no vence.
8. Porque bloquea conexiones entrantes como efecto colateral, pero no inspecciona ni filtra nada: no es un firewall.
9. Si la ida sale por un router y la vuelta entra por el otro, el segundo no tiene la entrada en su tabla y descarta la respuesta. Todo el tráfico de una sesión tiene que pasar por el mismo router NAT.
10. Usa el campo Identifier del Echo Request como si fuera un puerto.

Clase 6 · UDP: el transporte mínimo

Oct 4, 2026 · @Joaquin

Apunte para leer a la par de las diapositivas de UDP.

1. Para qué existe la capa de transporte

Hasta acá (clases 1 a 5) todo fue capa de red: IP lleva un paquete **de un host a otro host**. Pero en un host no hay "uno" que reciba: hay un navegador, un cliente de mail, un servidor DNS, todos a la vez. IP entrega el paquete en la máquina correcta y ahí se le termina el trabajo. **Alguien tiene que decidir a qué programa le toca.** Ese es el trabajo mínimo de la capa de transporte.

Lo que dice la clase, explicado

- **Comunicación lógica entre procesos** que corren en hosts distintos. "Lógica" porque, para la aplicación, es como si hubiera un cable directo hasta el otro proceso; en realidad hay routers, enlaces y pérdidas en el medio.
- **Se implementa solo en los extremos**, no en los routers. Un router mira la cabecera IP y reenvía; no le importa (en principio) qué hay adentro. TCP, UDP o QUIC viven en el sistema operativo de los hosts. (NAPT, de la clase 5, es justamente la excepción que "rompe las capas".)
- **Es la interfaz entre la aplicación y el sistema operativo.** La aplicación no arma paquetes IP: le pide al SO "mandá estos bytes a tal lugar" y el SO hace el resto.
- **Recibe datos de la aplicación, los parte en pedazos y les agrega una cabecera.** Esa unidad se llama **segmento** en TCP y **datagrama** en UDP. Después se mete entera como payload de un paquete IP (encapsulamiento).
- **Multiplexación y demultiplexación de aplicaciones** (sección 2): muchas aplicaciones comparten la misma IP y la misma placa de red.
- **Extiende el servicio *host-to-host* de IP a uno *process-to-process*.** Esta es la frase que resume todo lo anterior.
- **Protocolos disponibles: TCP, UDP y QUIC**, pero no son los únicos (existen SCTP, DCCP, etc.). QUIC, en rigor, corre **encima** de UDP (clase 10).

Dónde está en el modelo TCP/IP

Capa	Ejemplos	Qué identifica
Aplicación	HTTP, DNS, SMTP	el mensaje

Capa	Ejemplos	Qué identifica
Transporte	**TCP, UDP, QUIC**	**el proceso (puerto)**
Red	IPv4, IPv6	el host (IP)
Enlace	Ethernet, Wi-Fi	la interfaz en el enlace (MAC)
Física	cable, fibra	bits

Cada capa agrega su propio "nombre" para el destinatario, y cada uno es más fino que el anterior: MAC para el salto, IP para la máquina, puerto para el programa.

2. Puertos y sockets

Las 4 cosas que hacen falta para una comunicación

Los sistemas operativos son **multiusuario y multiprogramados**: corren muchos procesos a la vez. Para que dos procesos se hablen hacen falta **4 datos**: host local, proceso local, host remoto y proceso remoto. Los hosts ya están resueltos (las **direcciones IP**). Falta cómo nombrar a los procesos.

El dibujo de "Funcionalidad" muestra eso: un host con tres procesos (P1, P3, P4 en el emisor; P2 en otro host). La aplicación entrega datos (M), transporte les pone su cabecera (Ht) y forma el **segmento**, y del otro lado transporte tiene que saber a cuál de los procesos se lo da.

Por qué no alcanza con lo que ya había

La clase descarta dos ideas "obvias":

- **Extender la dirección IP** para que nombre también al proceso: imposible, **no quedan bits** (IPv4 tiene 32 y están todos usados para red y host).
- **Usar algo del sistema operativo** como el **PID** (process ID), el número de tarea o el nombre del job: no sirve, porque **cada SO lo hace distinto** y el protocolo tiene que **funcionar en todos los sistemas**. Además, el PID cambia cada vez que el programa arranca: el cliente no podría saber de antemano cuál es el PID del servidor web en otra máquina.

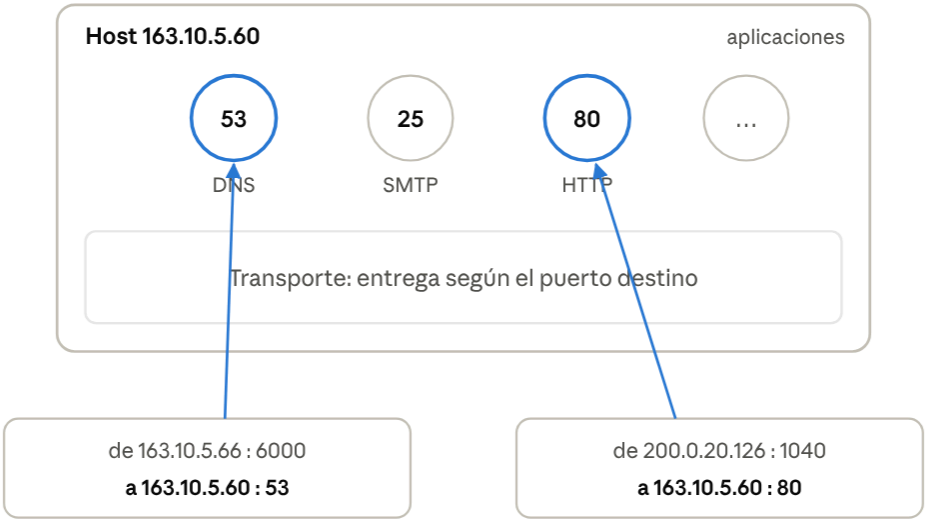
La solución: el puerto

Se crea una **abstracción nueva**, el **puerto (port)**:

- Es **un número** (16 bits, de 0 a 65535). Está en la cabecera de transporte, no en la de IP.

- **Cómo se implementa depende del SO**, generalmente como **una cola**: los datagramas que llegan para el puerto 53 se encolan ahí hasta que el proceso los lee.
- Lo usan **TCP y UDP** (cada uno con su propio espacio: el 53/UDP y el 53/TCP son puertos distintos).
- **Identifica unívocamente a la aplicación** que transmite o recibe en ese host.
- Es lo que permite **multiplexar** (muchas aplicaciones salen por la misma IP) y **demultiplexar** (lo que llega se reparte a la aplicación correcta).

Misma IP, distintos procesos: el puerto destino elige



IP los trajo al host; el puerto los separa

Demultiplexación por puerto · dibujo "Relación dirección IP y puertos" de la clase

En el dibujo, el host 163.10.5.60 tiene abiertos el 53 (DNS), 25 (SMTP) y 80 (web). Llegan dos datagramas, los dos con IP destino 163.10.5.60. La capa de red no puede distinguirlos; transporte mira el **puerto destino** y entrega uno a DNS y otro al servidor web. El puerto **origen** (6000, 1040) no sirve para elegir; sirve para saber **a dónde responder**.

Los rangos de puertos

Rango	Nombre	Para qué
0 a 1023	**Well-known** (bien conocidos)	Servicios estándar: 53 DNS, 80 HTTP, 25 SMTP, 21 FTP, 443 HTTPS. En Unix hace falta ser administrador para abrirlos.
1024 a 49151	**Registered** (registrados)	Aplicaciones que se registran en IANA (por ejemplo 3306 MySQL).
49152 a 65535	**Ephemeral** (efímeros o dinámicos)	Los elige el SO **al azar para el cliente**, por el tiempo que dure la comunicación.

Error en la diapositiva: dice "49151..65535", pero el 49151 ya está en el rango anterior. El rango efímero empieza en **49152**. (En la práctica, cada SO usa su propio rango efímero; Linux, por ejemplo, arranca en 32768.)

Por qué el servidor tiene un puerto fijo y el cliente no: el cliente tiene que saber de antemano a dónde ir ("el DNS está en el 53"). Al servidor, en cambio, no le importa de qué puerto viene el cliente: lo lee del datagrama y le responde ahí.

Sockets

Un **socket** es **la interfaz entre la aplicación y la capa de transporte**: el "enchufe" del proceso. El proceso **escribe** en el socket para enviar y **lee** del socket para recibir; no ve ni cabeceras ni paquetes.

- **Dirección de socket**: la terna **(protocolo, puerto local, IP local)**. En la clase: (tcp, 2456, 193.44.234.3). La diapositiva dice "proceso local", pero se refiere al **puerto** local, que es la forma de nombrar al proceso.
- **Sesión (quíntupla)**: **(protocolo, puerto local, IP local, puerto remoto, IP remota)**. En la clase: (tcp, 1500, 193.44.234.3, 21, 193.44.234.5), es decir, un cliente en 193.44.234.3 desde el puerto 1500 hablando con un servidor FTP (21) en 193.44.234.5.

La quíntupla **identifica una comunicación de forma única en toda Internet**. Si cambia cualquiera de los cinco valores, es otra comunicación. Por eso un servidor web puede atender a mil clientes en el mismo puerto 80: las mil quíntuplas difieren en IP o puerto remoto. (Es también la clave de la tabla de NAPT de la clase 5.)

3. Qué es UDP y, sobre todo, qué no hace

UDP (User Datagram Protocol, RFC 768) es el transporte más chico posible: le agrega a IP **solo los puertos** (y un checksum opcional). Todo lo demás que IP no garantiza, UDP tampoco lo garantiza. Para entenderlo conviene leer cada característica de la clase como "hereda el problema de IP y no lo arregla".

Best effort ("hago lo que puedo")

Igual que IP, UDP no promete nada. Los datagramas:

- **pueden perderse**: un router con la cola llena lo descarta y nadie lo retransmite;
- **pueden duplicarse**: por ejemplo, si una capa de enlace retransmite;
- **pueden llegar desordenados**: cada paquete IP puede ir por un camino distinto y no hay números de secuencia para reordenar.

Si la aplicación necesita que eso no pase, **lo tiene que resolver ella** (números de secuencia propios, reintentos con timeout, etc.).

Sin conexión

- **No hay handshake**: no hay un "¿estás?" "sí" antes de mandar. El primer datagrama ya lleva datos. Eso ahorra un RTT entero (TCP necesita 3 mensajes antes de mandar el primer byte útil).
- **Los datagramas son independientes**: UDP no guarda estado entre uno y otro. No sabe si el anterior llegó, ni siquiera si el otro extremo existe.

Un mensaje de la aplicación = un datagrama

UDP **respeta los límites de los mensajes**: si la aplicación hace un `send` de 300 bytes, sale **un** datagrama de 300 bytes de datos, y del otro lado un `recv` devuelve esos 300 bytes juntos (o nada, si se perdió). Contrastá con TCP, que ve los datos como un **flujo de bytes** y puede partirlos o juntarlos como quiera (clase 7). Si el mensaje es más grande que la MTU, quien lo parte es **IP, fragmentando** (clase 1), no UDP.

No tiene control de flujo, de errores ni de congestión

Control	Qué sería	Consecuencia de no tenerlo
De flujo	No mandarle al receptor más rápido de lo que puede leer	Si el receptor no lee a tiempo, su cola del puerto se llena y los datagramas nuevos se tiran
De errores	Detectar pérdidas y retransmitir	Solo **detecta** datos corruptos (checksum) y los descarta; no avisa ni retransmite
De congestión	Bajar la velocidad si la red está saturada	UDP manda tan rápido como la aplicación quiera, aunque la red esté colapsada (clase 9)

Encapsulamiento

Cada envío de la aplicación se convierte en **un datagrama UDP**, y ese datagrama va **entero como payload de un paquete IP**:

Cabecera IP (20 bytes sin opciones)	Cabecera UDP (8 bytes)	Datos de la aplicación
	datagrama UDP (cabecera UDP + datos)	

El **datagrama UDP** son las dos últimas columnas, y es a la vez el **payload de IP**. Todo junto es el **paquete IP**.

En la cabecera IP, el campo **Protocol = 17** (Next Header = 17 en IPv6) le dice al receptor "lo que viene adentro es UDP". Así IP sabe a qué protocolo de transporte entregárselo (TCP es el 6).

El ejemplo de la clase: una consulta DNS

Al **servidor DNS** le corresponde el puerto **53** (well-known). El **cliente** pide un puerto al SO y le toca el **28900** (efímero).

Datagrama	Puerto origen	Puerto destino
Consulta (cliente → servidor)	28900	53
Respuesta (servidor → cliente)	53	28900

Los puertos **se invierten** en la respuesta: el servidor responde al puerto desde el que le hablaron. (La diapositiva tiene un "28900.d" que es un error de tipeo.)

El dibujo "Conexión UDP Cliente-Servidor" es lo mismo con otros números: cliente 10.0.1.50 desde el puerto 1038, servidor 10.0.1.25 en el puerto 13 (el servicio *daytime*, que devuelve la hora). Ida p.o. 1038, p.d. 13; vuelta p.o. 13, p.d. 1038. Los sockets son 10.0.1.50:1038 y 10.0.1.25:13. Ojo: aunque el título diga "conexión", en UDP **no hay conexión**; son dos datagramas sueltos.

¿Y si no hay nadie escuchando en ese puerto?

Si llega un datagrama a un puerto que **no está abierto**, UDP lo **descarta** y le pide a ICMP que le mande al emisor un **ICMP Destination Unreachable, código Port Unreachable** (tipo 3, código 3, clase 1). Es la única "respuesta de error" que puede recibir quien usa UDP, y no la genera UDP sino ICMP. (La diapositiva escribe "Unrecheable".) Herramientas como `traceroute` en Unix usan justamente esto: mandan UDP a puertos altos que nadie usa y esperan ese ICMP para saber que llegaron.

4. La cabecera UDP: 8 bytes, 4 campos

La cabecera mide **siempre 8 bytes** (no tiene opciones, a diferencia de IP y TCP). Son cuatro campos de **16 bits** cada uno, en dos filas de 32 bits:

Bits 0 a 15	Bits 16 a 31
Puerto origen	**Puerto destino**
Longitud	**Checksum**
Datos…	

Que cada campo tenga 16 bits explica de dónde sale el **65535**: es el número más grande que entra en 16 bits ($2^{16} - 1$), y por eso los puertos van de 0 a 65535.

Campo por campo

Campo	Qué es	Lo que remarca la clase
Puerto origen	Puerto del proceso que envía	**Opcional: puede ser 0.** Sirve para que el otro sepa a dónde responder; si el emisor no espera respuesta, puede no ponerlo.
Puerto destino	Puerto del proceso que recibe	Es el que se usa para demultiplexar. Siempre está.
Longitud	Tamaño del datagrama **completo: cabecera + datos**, en bytes	**Redundante**: se puede calcular desde IP (longitud total del paquete IP menos la cabecera IP). El mínimo es 8 (cabecera sin datos).
Checksum	Control de errores sobre todo el datagrama **más una pseudocabecera**	**Opcional en IPv4** (si no se calcula, va en 0) y **obligatorio en IPv6**. Si da mal en el destino, el datagrama se **descarta en silencio**. Ver sección 5.

Por qué cada detalle es así

- **¿Por qué no hay longitud de cabecera, como en IP (IHL)?** Porque la cabecera nunca cambia de tamaño: siempre son 8 bytes, no hace falta indicarlo.
- **¿Por qué el checksum pasa a ser obligatorio en IPv6?** Porque **IPv6 sacó el checksum de su propia cabecera** (clase 2) para que los routers no tengan que recalcularlo en cada salto. Sin checksum en IP ni en UDP, nadie verificaría que las direcciones lleguen sanas. En IPv4 al menos estaba el checksum de la cabecera IP.

- **¿Por qué "en silencio"?** Porque si el datagrama está corrupto no se puede confiar en ningún campo, ni siquiera en el puerto o la IP de origen: no hay a quién avisarle con seguridad. Y como UDP no retransmite, simplemente se pierde.

Tamaños máximos (para cuentas)

- Por el campo Longitud: un datagrama UDP puede medir hasta **65535 bytes**, es decir **65527 bytes de datos**.
- Pero tiene que entrar en un paquete IPv4, cuyo total también es como mucho 65535: **65535 − 20 (IP) − 8 (UDP) = 65507 bytes de datos** como máximo real sobre IPv4.
- Cualquier datagrama más grande que la MTU (1500 en Ethernet) **se fragmenta en IP**. Por eso muchas aplicaciones UDP mandan mensajes chicos: DNS clásico se limitaba a 512 bytes de datos.

Ejemplo leído en hexadecimal

Cabecera de la consulta DNS del ejemplo (cliente 28900 → servidor 53, con 30 bytes de datos):

```
70 E4 | 00 35 | 00 26 | xx xx
```

- `70E4` en hexa = 7·4096 + 14·16 + 4 = **28900** → puerto origen.
- `0035` = **53** → puerto destino.
- `0026` = **38** → longitud: 8 de cabecera + 30 de datos.
- `xx xx` → el checksum, que depende de todo lo demás.

5. El checksum y la pseudocabecera

Cómo se calcula (lo que dice la diapositiva, paso por paso)

1. Se arma un bloque con **la pseudocabecera + la cabecera UDP + los datos**, con el campo checksum en **0** mientras se calcula.
2. Si la cantidad de bytes es **impar**, se agrega **un byte en cero al final** para completar palabras de 16 bits. Ese byte no se envía; solo se usa para la cuenta.
3. Se suman todas las palabras de 16 bits en **complemento a 1**: cada vez que la suma pasa de 16 bits, **el acarreo que sobra se vuelve a sumar abajo** ("end-around carry").
4. Se **complementa** el resultado (se invierten todos los bits). Eso es el checksum.
5. Si el checksum diera **todo ceros**, se envía **todo unos** (0xFFFF).

El receptor hace la misma suma, **incluyendo el checksum recibido**. Si nada se corrompió, el resultado da **todo unos** (0xFFFF). Cualquier otra cosa significa error: el datagrama se descarta.

Ejemplo chico

Suponé que todo el bloque fueran tres palabras: `8F00`, `9A10`, `0011`.

Paso	Cuenta	Resultado
Sumar las dos primeras	8F00 + 9A10	1**2910** (se pasó de 16 bits: sobra un 1)
Acarreo de vuelta	2910 + 1	2911
Sumar la tercera	2911 + 0011	2922
Complementar	FFFF − 2922	**D6DD** ← checksum
Verificación en el receptor	2922 + D6DD	**FFFF** → correcto

(Complementar a 1 en hexa es restarle cada dígito a F: 2→D, 9→ 6, 2→D, 2→D.)

Por qué "todo ceros se reemplaza por todo unos"

En IPv4, un checksum **0 significa "no lo calculé"**. Si el cálculo real diera 0x0000, el receptor creería que no hay checksum y no verificaría nada. Se manda 0xFFFF en su lugar, y no rompe nada porque **en complemento a 1 hay dos ceros**: 0x0000 (+0) y 0xFFFF (−0) valen lo mismo para la suma. La verificación del receptor da bien con cualquiera de los dos.

La pseudocabecera

Es un conjunto de campos **sacados de la cabecera IP** que se ponen **adelante del datagrama solo para calcular el checksum**. **No se envía**: el receptor la reconstruye con la cabecera IP del paquete que le llegó.

Pseudocabecera de IPv4 (12 bytes), como en la diapositiva:

Bits 0 a 7	Bits 8 a 15	Bits 16 a 31
Dirección IP de origen (32 bits)		
Dirección IP de destino (32 bits)		

Bits 0 a 7	Bits 8 a 15	Bits 16 a 31
00000000 (relleno)	**Protocolo = 00010001 (17)**	**Longitud del datagrama UDP**

En IPv6 es la misma idea con los tamaños de IPv6: direcciones de 128 bits, longitud de 32 bits, 24 bits en cero y **Next Header = 17** (40 bytes en total). La diapositiva muestra solo la de IPv4.

Para qué sirve

La clase lo dice así: **permite a UDP comprobar que IP no se equivocó (ni lo engañó) en la entrega.** Los puertos solos no alcanzan para decir que el datagrama es para mí; también tiene que venir a **mi IP**, con el **protocolo correcto**. Si un error de bits en el camino cambiara la IP destino o el campo protocolo y aun así llegara a algún host, el checksum UDP lo detectaría, porque esos valores están dentro de la cuenta. Es un control **de punta a punta** que se apoya en datos de la capa de abajo.

Consecuencias que conectan con otras clases

- **NAT (clase 5)** tiene que recalcular el checksum de UDP y TCP aunque solo cambie la IP, justamente porque la IP está en la pseudocabecera.
- **TCP (clase 7)** usa exactamente el mismo mecanismo, con protocolo = 6.
- El valor **17** del campo Protocol (IPv4) o Next Header (IPv6) es lo que identifica a UDP; aparece tanto en la cabecera IP real como en la pseudocabecera.

6. Cuándo conviene UDP

Si UDP no garantiza nada, ¿por qué se usa? Porque **lo que TCP garantiza tiene un costo** (handshake, retransmisiones, esperar lo perdido, frenar ante congestión), y hay aplicaciones a las que ese costo les molesta más de lo que les sirve. La clase da tres perfiles:

Perfil	Por qué UDP le conviene
Multimedia (voz, video en vivo, juegos)	Un paquete que llega tarde es igual que uno perdido: no sirve retransmitir audio de hace un segundo. TCP frenaría todo el flujo esperando lo perdido; UDP sigue, y la aplicación tolera un corte breve.

Perfil	Por qué UDP le conviene
Multicast o broadcast	TCP es una conexión **entre dos** extremos; no tiene sentido con muchos receptores. UDP no tiene conexión, así que un datagrama puede ir a un grupo. (DHCP, clase 4, usa UDP con broadcast por esto mismo.)
Intercambio simple de mensajes (pregunta y respuesta)	Para mandar una pregunta corta y recibir una respuesta corta, un handshake de 3 mensajes y un cierre de 4 son más caros que la consulta misma. Si no llega respuesta, la aplicación pregunta de nuevo.

Las aplicaciones que nombra la clase

Aplicación	Qué hace	Por qué UDP
DNS	Traduce nombres a IP	Una consulta, una respuesta: si se pierde, se repite
SNMP	Monitorear y administrar equipos de red	Consultas cortas y periódicas; tiene que funcionar aun con la red en problemas
TFTP	Transferencia de archivos simple (por ejemplo, para bootear equipos)	Muy liviano de implementar; hace su propio control (manda un bloque y espera el ACK)
RPC	Llamadas a procedimientos remotos	Pedido y respuesta, como una llamada a función
VoIP	Voz sobre IP	Tiempo real: prefiere perder que esperar
HTTP/3	La web	Corre sobre **QUIC**, que va sobre UDP (clase 10). HTTP/1.1 y HTTP/2 usan TCP.

El caso de HTTP/3 muestra la idea de fondo: **UDP es una base mínima sobre la que la aplicación construye la confiabilidad que necesite**. QUIC reimplementa encima de UDP casi todo lo que hace TCP (secuencia, ACKs, retransmisión, control de flujo y de congestión), pero a su manera y en el espacio de usuario, sin depender del TCP del sistema operativo.

UDP frente a TCP, en una tabla

	UDP	TCP
Conexión	No	Sí (handshake de 3 vías)
Entrega	Best effort	Confiable y en orden
Unidad	Mensajes (datagramas)	Flujo de bytes
Cabecera	8 bytes fijos	20 a 60 bytes
Control de flujo / congestión	No	Sí
Multicast / broadcast	Sí	No
Estado en los extremos	Ninguno	Mucho (secuencias, ventanas, timers)

7. Preguntas para autoevaluarte

1. IP ya entrega el paquete en el host correcto. ¿Qué problema queda sin resolver que justifica la capa de transporte?
2. ¿Por qué no se usa el PID del proceso para identificar a la aplicación destino?
3. Un servidor web en el puerto 80 atiende a 500 clientes a la vez. ¿Cómo distingue el SO a qué conversación pertenece cada datagrama o segmento que llega?
4. Un cliente manda una consulta desde 10.0.0.5:51000 a 8.8.8.8:53. Escribí los puertos origen y destino de la respuesta.
5. ¿Qué pasa si mandás un datagrama UDP a un puerto cerrado? ¿Quién te avisa?
6. El campo Longitud de un datagrama vale 0x0040. ¿Cuántos bytes de datos lleva?
7. ¿Por qué se dice que el campo Longitud es redundante?
8. ¿Por qué el checksum de UDP es opcional en IPv4 pero obligatorio en IPv6?
9. ¿Qué es la pseudocabecera, se envía o no, y para qué sirve?
10. Calculá el checksum de un bloque formado por las palabras 1234 y F00F.
11. Si el cálculo del checksum da 0x0000, ¿qué se envía y por qué?
12. Nombrá dos razones por las que una aplicación de videollamada preferiría UDP antes que TCP.

Respuestas rápidas

1. Saber a qué **proceso** de ese host entregarlo (multiplexación y demultiplexación), es decir, pasar de host-to-host a process-to-process.
2. Porque depende de cada SO, cambia cada vez que el programa arranca y el otro extremo no puede conocerlo de antemano. Hace falta algo independiente del sistema: el puerto.
3. Por la quíntupla: protocolo, IP y puerto locales, IP y puerto remotos. Los 500 clientes difieren en la IP o el puerto remoto.
4. Origen 53, destino 51000 (los puertos se invierten).
5. UDP lo descarta, y ICMP le manda al emisor un Destination Unreachable con código Port Unreachable.
6. 0x40 = 64 bytes en total, menos 8 de cabecera: **56 bytes de datos**.
7. Porque se deduce de IP: longitud total del paquete menos la longitud de la cabecera IP.
8. Porque IPv6 no tiene checksum en su cabecera; sin el de UDP, nada verificaría las direcciones ni los datos. En IPv4 al menos está el de la cabecera IP.
9. Son campos de la cabecera IP (IP origen y destino, protocolo 17, longitud UDP) que se agregan solo para calcular el checksum. No se envía. Sirve para detectar que IP entregó el datagrama al host o al protocolo equivocado.
10. 1234 + F00F = 1**0243** → 0243 + 1 = 0244 → complemento: **FDBB**.
11. Se envía 0xFFFF, porque 0 significa "checksum no calculado" y en complemento a 1 0xFFFF también vale cero.
12. No quiere esperar retransmisiones de datos que ya llegarían tarde, y no quiere el freno del control de congestión de TCP ni el handshake inicial.

Clases 7 y 8 · TCP: transporte confiable

Oct 4, 2026 · @Joaquin

Apunte para leer a la par de las diapositivas de TCP (la clase 8 es la versión corregida de la 7; son el mismo contenido).

1. Qué es TCP y qué le agrega a UDP

TCP (Transmission Control Protocol) se definió en la **RFC 793 (1981)**, que en 2022 quedó reemplazada por la **RFC 9293**. Las dos son el "Std 7": es el mismo estándar, reescrito y actualizado.

Como UDP, TCP da **puertos** (multiplexación de procesos) y **checksum**. La diferencia es todo lo que agrega encima. La forma de entender cada propiedad es preguntarse **qué problema de IP resuelve**:

Propiedad	Problema de IP que resuelve	Con qué mecanismo (sección)
Orientado a conexión	IP no sabe si el otro extremo existe ni guarda estado	Handshake de 3 vías y cierre (3, 4)
Confiable	Los paquetes se pierden o se corrompen	ACKs, checksum y retransmisión por timeout (6, 8)
Ordenado	Los paquetes llegan desordenados o duplicados	Números de secuencia por byte (6)
Control de flujo	El emisor puede saturar al receptor	Ventana de recepción anunciada (7)
Control de congestión	Los emisores pueden saturar la red	Ventana de congestión (**clase 9**)
Byte stream	— (es el modelo que ve la aplicación)	Ver abajo

Ojo con la diferencia entre los dos "controles": el **de flujo** protege al **receptor** ("no me mandes más de lo que puedo guardar"); el **de congestión** protege a **la red** ("no metas más de lo que los routers aguantan"). Se confunden mucho en los exámenes.

Byte stream: la aplicación ve un caño de bytes

En UDP, cada `send` es un datagrama. En TCP **no**: la aplicación escribe bytes en un flujo continuo y TCP decide cómo cortarlo.

- **Packetization**: TCP convierte el flujo en un conjunto de mensajes que IP pueda transportar. Cada pedazo (*chunk*) se llama **segmento**.
- **TCP decide el tamaño de cada segmento**, y lo elige **para que no haya que fragmentarlo** en IP. El límite es el **MSS** (sección 3).
- Los segmentos **pueden ser de distinto tamaño** y TCP **podría combinarlos** (*repacketization*): si una aplicación hace tres `send` de 10 bytes, puede salir un solo segmento de 30; si hace un `send` de 10.000, salen varios.
- Del otro lado, la aplicación **no ve los límites**: un `recv` puede devolver 30 bytes que se escribieron en tres `send`. Si la aplicación necesita separar mensajes, los delimita ella (por ejemplo, HTTP usa encabezados con longitud).
- **Cada segmento lleva su posición en el flujo** (número de secuencia) y los **puertos** origen y destino.

Encapsulamiento

Cabecera IP (20 bytes sin opciones)	Cabecera TCP (20 bytes sin opciones)	Datos (payload TCP)

El segmento TCP (cabecera + datos) es el payload de IP. En la cabecera IP, **Protocol = 6** (en UDP era 17).

Del lado de las conexiones

TCP es **full duplex**: una conexión son **dos flujos independientes**, uno en cada sentido, cada uno con sus propios números de secuencia, su ventana y su MSS. Casi todo lo que sigue (ISN, ventanas, FIN) se entiende mejor pensando "esto pasa en cada sentido por separado".

2. La cabecera TCP

La parte fija mide **20 bytes** (5 filas de 32 bits); con opciones puede llegar a **60**.

Bits 0 a 15	Bits 16 a 31
Source Port (16)	**Destination Port** (16)
Sequence Number (32)	
Acknowledgment Number (32)	
Header Length (4) · Reservado (4) · **Flags** (8)	**Window Size** (16)

Bits 0 a 15	Bits 16 a 31
Checksum (16)	**Urgent Pointer** (16)
Options (0 a 40 bytes)	

Campo por campo

Campo	Qué indica	Para entenderlo
Source / Destination Port (16 bits c/u)	Puerto de la aplicación que envía y de la que recibe	Igual que en UDP.
Sequence Number (32)	El número del **primer byte de datos** que lleva este segmento	TCP numera **bytes**, no segmentos. Si un segmento tiene SN = 5000 y 500 bytes, lleva los bytes 5000 a 5499.
Acknowledgment Number (32)	El **próximo byte que espero recibir** del otro	ACK = 5500 quiere decir "tengo todo hasta el 5499 inclusive; mandame desde el 5500". Solo vale si el flag A está en 1.
Header Length (4)	Largo de la cabecera **en palabras de 4 bytes**	Con 4 bits el máximo es 15 → 15 × 4 = **60 bytes**; el mínimo es 5 → **20 bytes**. Por eso las opciones son como mucho 40 bytes. Es lo que dice dónde empiezan los datos.
Flags (control bits)	La finalidad del segmento	Ver la tabla de abajo.
Window Size (16)	Cuántos bytes, **contando desde el número de ACK**, puedo recibir todavía	Es el control de flujo (sección 7). Máximo 65535 sin la opción *window scale*.
Checksum (16)	Control de errores del segmento entero **más una pseudocabecera**	**Obligatorio** en TCP (en UDP/IPv4 era opcional). Mismo algoritmo que UDP (clase 6).
Urgent Pointer (16)	Hasta dónde llegan los datos urgentes	Solo vale con el flag U. Se **suma al número de secuencia** para obtener dónde terminan. Casi no se usa.

Campo	Qué indica	Para entenderlo
Options (hasta 40 bytes)	Información opcional	MSS, window scale, timestamps, SACK (sección 9).

(La diapositiva dice "número asinado": es "asignado".)

Los flags

En la figura de la clase aparecen 8 bits de flags (y 4 reservados en 0):

Flag	Nombre	Significa
C (CWR)	Congestion Window Reduced	"Ya reduje mi ventana", respuesta a ECE
E (ECE)	ECN Echo	"La red me marcó congestión" (ECN, notificación explícita de congestión)
U (URG)	Urgent	El Urgent Pointer es válido
A (ACK)	Acknowledgment	El campo ACK es válido. Está en 1 en **todos** los segmentos salvo el primer SYN
P (PSH)	Push	"Entregá esto ya a la aplicación, no esperes a juntar más"
R (RST)	Reset	Cierre inmediato y abrupto de la conexión (sección 4)
S (SYN)	Synchronize	Pedido de conexión: "sincronicemos números de secuencia" (sección 3)
F (FIN)	Finish	"No tengo más datos para mandar" (sección 4)

En las diapositivas y ejercicios se abrevian con su letra: un segmento "S,A" es un SYN+ACK; "F,A" es FIN+ACK.

La pseudocabecera de TCP

Es la misma idea que en UDP: se arma **solo localmente para calcular el checksum, no se envía**, y sirve para verificar la integridad del segmento junto con las direcciones que le puso IP.

Dirección IP de origen (32 bits)		
Dirección IP de destino (32 bits)		
00000000	**00000110** (protocolo **6** = TCP)	**Longitud del segmento TCP**

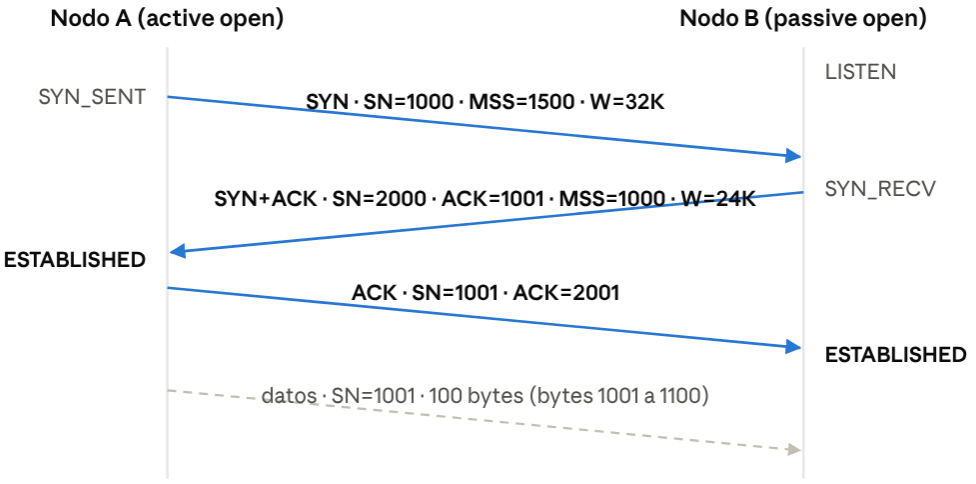
La diapositiva escribe "$110_2 = 6_{10}$": 110 en binario es 6.

3. Abrir la conexión: handshake de 3 vías, ISN y MSS

Por qué hace falta un handshake

Antes de mandar datos confiables, los dos extremos tienen que ponerse de acuerdo en tres cosas: que **el otro existe y está escuchando**, desde **qué número de secuencia** va a numerar cada uno sus bytes, y **parámetros** como el MSS y la ventana. Como la conexión es full duplex, cada sentido necesita su número inicial **y la confirmación de que el otro lo recibió**. Eso daría 4 mensajes (SYN, ACK, SYN, ACK), pero el ACK de B y el SYN de B viajan juntos en un solo segmento: **3 vías**.

3-way handshake: cada lado elige su ISN y el otro lo confirma con +1



Los números son los del diagrama de estados de la clase

Los tres segmentos

1. **A → B: SYN.** Flag S. SN = ISN de A (1000). Lleva la opción **MSS** y su ventana. No tiene flag A (todavía no hay nada que confirmar). A pasa a **SYN_SENT**; B estaba en **LISTEN** (había hecho un *passive open*: espera conexiones).

2. **B → A: SYN+ACK.** Flags S y A. SN = ISN de B (2000). **ACK = ISN de A + 1 = 1001**. También lleva el MSS y la ventana de B. B pasa a **SYN_RECV**.
3. **A → B: ACK.** Flag A. SN = 1001, **ACK = ISN de B + 1 = 2001**. A pasa a **ESTABLISHED** al mandarlo, B al recibirlo.

Forma general (la de la diapositiva "Establecimiento de la sesión"): SYN con SN = ISN_A; SYN+ACK con SN = ISN_B y ACK = ISN_A + 1; ACK con SN = ISN_A + 1 y ACK = ISN_B + 1.

¿Por qué "+1" si el SYN no lleva datos? Porque **el SYN consume un número de secuencia**, como si fuera un byte. Así se lo puede confirmar con un ACK igual que a cualquier dato; si no ocupara un número, no habría forma de decir "recibí tu SYN". Lo mismo pasa con el FIN (sección 4). Por eso el primer byte de datos de A es el 1001, no el 1000.

ISN: el número de secuencia inicial

- El ISN (Initial Sequence Number) lo **elige cada extremo localmente**, sin coordinar con el otro; por eso hay dos ISN, uno por sentido.
- No empieza en 0 sino en un valor que varía (hoy, pseudoaleatorio): así un segmento viejo de una conexión anterior entre los mismos puertos no cae dentro de los números válidos de la nueva, y es más difícil que un atacante adivine los números e inyecte segmentos.
- A partir del ISN, **cada byte de datos tiene un número** que indica su ubicación en el flujo.

MSS (Maximum Segment Size)

Es **el tamaño máximo de datos que un extremo está dispuesto a recibir en un segmento**. Puntos de la clase:

- **No cuenta la cabecera**: son solo los datos.
- **No es un campo de la cabecera** sino una **opción** (sección 9), y su implementación es **obligatoria**.
- Se manda **solo en los segmentos con SYN** (los dos primeros del handshake). Después ya no se puede cambiar.
- Si no se manda, se asume el valor por defecto: **536 bytes en IPv4** y **1220 en IPv6**.
- Puede ser **distinto en cada sentido**, aunque cada extremo debería usar **el mínimo** de los anunciados.
- Se calcula como **MSS = MTU − (cabecera IP + cabecera TCP)**. En Ethernet: 1500 − 20 − 20 = **1460**.

De dónde salen los valores por defecto: IPv4 garantiza que cualquier host acepta datagramas de **576** bytes → 576 − 20 − 20 = 536. IPv6 garantiza una MTU mínima de **1280** → 1280 − 40 − 20 = 1220. Elegir el MSS para que el segmento **entre en la MTU** es lo que

permite que TCP **no dependa de la fragmentación de IP**.

(En el ejemplo de la clase A anuncia MSS = 1500, que no podría ser sobre Ethernet porque el segmento quedaría en 1540 bytes. Tomalo como un número ilustrativo.)

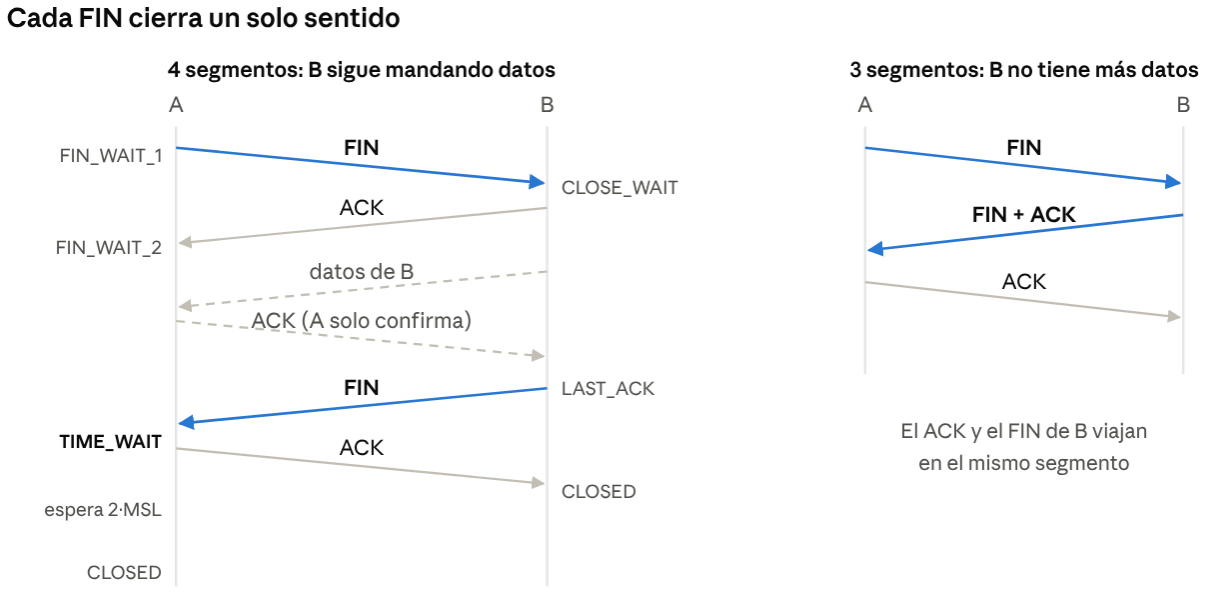
¿Y si nadie escucha en ese puerto?

Si llega un SYN a un puerto donde no hay aplicación escuchando, TCP responde con un segmento con los flags **R (Reset) y A (Ack)**, y la sesión se cierra antes de existir. Es el equivalente al ICMP Port Unreachable de UDP, pero lo genera el propio TCP.

4. Cerrar la conexión: FIN, half-close y RST

La idea: cada sentido se cierra por separado

Como la conexión son dos flujos, **un FIN significa "yo no tengo más datos para mandarte", no "cortemos"**. Por eso la clase insiste en que **enviar FIN no es el cierre definitivo de la sesión**: el otro sentido sigue abierto hasta que el otro extremo mande su propio FIN. **Cualquiera de los dos extremos** puede iniciar el cierre.



Los dos cierres del dibujo "Fin de la sesión" de la clase, con los estados

Cierre de 4 segmentos (con half-close)

1. A manda **FIN**: terminó de enviar. (Pasa a FIN_WAIT_1.)
2. B confirma con **ACK**. B pasa a CLOSE_WAIT; A, al recibir el ACK, a FIN_WAIT_2.
3. **B puede seguir mandando datos** todo lo que necesite, y **A solo puede mandar ACKs**. Esto se llama **half-close** (conexión medio cerrada).

4. Cuando B termina, manda su **FIN** (pasa a LAST_ACK) y A responde con el **ACK** final.

Ejemplo típico: un cliente manda todo un archivo, cierra su lado para avisar "listo, eso es todo", y espera a que el servidor le conteste con el resultado.

Cierre de 3 segmentos

Si cuando llega el FIN de A, B tampoco tiene nada más para mandar, **junta su ACK y su FIN en un mismo segmento**: FIN → FIN+ACK → ACK. Es la misma idea que el SYN+ACK del handshake.

El diagrama de estados de la clase usa este cierre: A manda **F,A SN=9000 ACK=6000**; B responde **F,A SN=6000 ACK=9001**; A cierra con **A SN=9001 ACK=6001**. Fijate que cada ACK es **el SN del FIN + 1**: **el FIN, como el SYN, consume un número de secuencia**.

Error en la diapositiva: el texto dice "3 (a) ó 4 (b) segmentos", pero en el dibujo es al revés: el (a) tiene 4 segmentos (con datos de B en el medio) y el (b) tiene 3. El texto que sigue ("en (a), Nodo B puede seguir enviando datos") confirma que (a) es el de 4.

TIME_WAIT: por qué el que cierra primero espera

Después de mandar el último ACK, A no pasa directo a CLOSED: se queda en **TIME_WAIT** durante **2·MSL** (MSL = *Maximum Segment Lifetime*, lo máximo que un segmento puede sobrevivir en la red). Dos razones:

- **El último ACK se puede perder.** Si se pierde, B retransmite su FIN; A tiene que seguir existiendo para volver a confirmarlo. Si A ya no recordara la conexión, contestaría con un RST.
- **Que mueran los segmentos viejos.** Si se abriera enseguida una conexión nueva con los mismos puertos e IP, un segmento retrasado de la anterior podría colarse como válido. Esperando 2·MSL, cualquier resto de la vieja ya desapareció.

RST: el cierre abrupto

Un segmento con el flag **R (Reset)** provoca el **cierre inmediato** de la conexión: no hay ACK, no se espera nada, los datos pendientes se descartan. Según la clase:

- **Cualquier extremo puede resetear en cualquier momento** (por ejemplo, una aplicación que aborta).
- TCP también manda un reset solo, ante ciertos eventos:
  - Llega un segmento que **no corresponde a ninguna conexión** conocida.
  - Llega un **intento de conexión a un puerto cerrado** → responde con **R y A** (sección 3).

    - Hay una **half-open connection**: uno de los extremos se cayó o reinició y perdió el estado, y el otro sigue creyendo que la conexión existe. Cuando el que sigue vivo manda algo, el que la había perdido **le contesta con un Reset**. (Ojo: *half-open* es distinto de *half-close*; el half-close es un cierre ordenado de un solo sentido.)
- **Cuándo se acepta un RST** (para que no se pueda cortar conexiones ajenas mandando RST falsos):
    - En **SYN_SENT**, solo si su **ACK confirma el SYN** que mandé.
    - En cualquier otro estado, solo si su **número de secuencia cae dentro de la ventana de recepción**. Un atacante tendría que adivinar un número de 32 bits dentro de un rango chico.

5. Estados, sesiones y netstat/ss

Los estados de una conexión

TCP guarda en cada extremo **en qué etapa está cada conexión**. Los nombres aparecen en el diagrama de la clase y en la salida de `netstat`:

Estado	Quién	Qué significa
CLOSED	ambos	No hay conexión
LISTEN	servidor	*Passive open*: espera SYN en un puerto
SYN_SENT	cliente	*Active open*: mandó el SYN, espera SYN+ACK
SYN_RECV	servidor	Recibió SYN y mandó SYN+ACK, espera el ACK
ESTABLISHED	ambos	Conexión abierta, se intercambian datos
FIN_WAIT_1	el que cierra primero (*active close*)	Mandó su FIN, espera el ACK
FIN_WAIT_2	active close	Su FIN fue confirmado; espera el FIN del otro (half-close)
CLOSE_WAIT	el otro (*passive close*)	Recibió un FIN; espera que su aplicación también cierre
LAST_ACK	passive close	Mandó su FIN, espera el último ACK
TIME_WAIT	active close	Mandó el último ACK; espera 2·MSL por las dudas

Estado	Quién	Qué significa
CLOSING	ambos	Los dos mandaron FIN al mismo tiempo (cierre simultáneo, poco común)

Una pista para leerlos: los estados **WAIT** del que cierra primero esperan «al otro» (FIN_WAIT = espero su FIN); **CLOSE_WAIT** espera «a mi aplicación» (espero que el programa haga `close`). Muchas conexiones colgadas en CLOSE_WAIT suelen ser un bug del programa que nunca cierra el socket.

El diagrama de la clase, segmento por segmento

Segmento	A	B	Comentario
S, SN=1000, MSS=1500, W=32K	SYN_SENT	LISTEN → SYN_RECV	A anuncia ventana de 32K
S,A, SN=2000, ACK=1001, MSS=1000, W=24K	→ ESTABLISHED		B anuncia ventana de 24K
A, SN=1001, ACK=2001		→ ESTABLISHED	Fin del handshake
A, SN=1001, ACK=2001, **100 bytes**			Bytes 1001 a 1100
A, SN=2001, ACK=**1101**			B confirma esos 100 bytes
… intercambio de datos …			
F,A, SN=9000, ACK=6000	FIN_WAIT_1	→ CLOSE_WAIT	A cierra
F,A, SN=6000, ACK=9001		LAST_ACK	B cierra en el mismo segmento
A, SN=9001, ACK=6001	TIME_WAIT	→ CLOSED	

Errores en la diapositiva:

- En los segmentos que no son SYN escribe "ISN=…"; debería decir **SN**. El ISN es solo el inicial.
- El ACK de B a los 100 bytes dice **ACK=2101**, pero tiene que ser **ACK=1101**: B confirma bytes de A, que iban del 1001 al 1100, así que el próximo que espera es el 1101. (El 2101 mezcla con la numeración de B.)

- Con el cierre de 3 segmentos, A recibe FIN+ACK estando en FIN_WAIT_1 y pasa **directo a TIME_WAIT**; el rótulo FIN_WAIT_2 del dibujo solo aplicaría si el ACK y el FIN llegaran por separado.

Muchas sesiones en el mismo puerto

Los dos dibujos de la clase muestran que **lo que identifica una conexión es el par de sockets** (la quíntupla de la clase 6), no un puerto solo:

- **Sesiones a un mismo servidor:** el servidor telnet 10.0.1.25 tiene un solo socket escuchando, **10.0.1.25:23**, y dos conexiones simultáneas: con 10.0.1.50:**1038** y con 10.0.2.47:**1038**. Los dos clientes usan el mismo puerto, pero tienen IP distinta: son conexiones distintas.
- **Sesiones desde un mismo host:** el cliente 10.0.1.80 abre dos conexiones al mismo servidor 10.0.1.25:23, desde los puertos **1038** y **1039**. Misma IP, mismo destino, distinto puerto origen: conexiones distintas.

En los dos casos, el servidor demultiplexa mirando **los cuatro valores** (IP y puerto de cada lado). Por eso un servidor web atiende miles de clientes en el mismo puerto 80.

netstat y ss

`netstat -a -n -p TCP` (Windows; en Linux `netstat -tan` o el más moderno `ss -tan`) lista las conexiones TCP del equipo:

Columna	Qué muestra
Local Address	IP:puerto de este equipo
Foreign Address	IP:puerto del otro extremo
State	El estado de la tabla de arriba

En la captura de la clase se ven:

- Sockets en **LISTENING** con dirección remota **0.0.0.0:0** (todavía no hay otro extremo), y local **0.0.0.0:135**: escucha en **todas** las interfaces.
- Conexiones **ESTABLISHED** como 10.10.2.38:1132 ↔ 10.10.6.170:139, con IP y puerto reales de los dos lados.
- Varias conexiones desde la misma IP local con puertos distintos (1132, 1200, 1203, 1207): es el caso "sesiones desde un mismo host".

6. Confiabilidad: secuencia, ACK y retransmisión

Números de secuencia

- Son de **32 bits** (de 0 a $2^{32} - 1$; cuando llegan al final vuelven a 0).
- El valor inicial se fija en el handshake a partir del **ISN**, que cada extremo elige localmente.
- **Cada byte de datos** tiene su número. El SN de un segmento es el número de **su primer byte**.
- **Indican la ubicación de cada byte en el flujo**, y eso es lo que permite **reordenar** lo que llega desordenado y **descartar duplicados** (si ya tengo el byte 5000, otro segmento con el 5000 es repetido).

El dibujo de la clase marca tres segmentos sobre el flujo:

Segmento	SN	Siguiente SN	Bytes que lleva
8	13450	14950	14950 − 13450 = **1500**
9	14950	16050	16050 − 14950 = **1100**
10	16050	17550	17550 − 16050 = **1500**

La regla para cuentas: **SN del siguiente = SN + cantidad de bytes**. Y se ve que **los segmentos pueden tener tamaños distintos** (el 9 es más chico), como dice la parte de *packetization*.

Intercambio de datos: lo que dice la clase, explicado

- **Canal full duplex**: los dos mandan datos a la vez, cada uno con su numeración.
- **Un extremo puede mandar datos hasta el tamaño de la ventana que le anunció el otro, sin esperar ACK.** No manda uno y espera (eso desperdiciaría un RTT por segmento): manda varios seguidos (sección 7).
- **Todo segmento que llega bien tiene que ser confirmado** (ACK).
- **Piggybacking**: en vez de mandar un ACK solo, el receptor **espera un poco a tener datos propios** para mandar y "sube" el ACK a ese segmento. Ahorra segmentos: el ACK viaja de arriba en los datos del otro sentido.
- **El número de ACK indica el próximo byte que se espera recibir.**
- **Los ACK son acumulativos y crecen uniformemente**: ACK = 2500 confirma **todo** hasta el 2499, no solo el último segmento. Nunca retroceden.

- **Si un segmento llega con errores, tiene que ser retransmitido.** Más precisamente: el receptor lo **descarta** (checksum mal) y **no lo confirma**; el emisor, al no ver el ACK, lo vuelve a mandar cuando vence su timer (**RTO**, sección 8). TCP no tiene "ACK negativo".

Un ejemplo con pérdida

A manda cuatro segmentos de 500 bytes a partir del byte 1000, y se pierde el segundo:

Llega a B	B responde	Por qué
1000–1499	ACK 1500	Todo bien, espero el 1500
(1500–1999 se pierde)	—	
2000–2499	**ACK 1500**	Me falta el 1500: **repito** el ACK (lo que llega lo guardo)
2500–2999	**ACK 1500**	Sigo esperando el 1500
1500–1999 (retransmitido)	**ACK 3000**	Ahora tengo todo hasta el 2999: el ACK acumulativo salta

Dos cosas para ver acá:

- **El ACK acumulativo no puede decir "tengo del 2000 al 2999"**: solo dice "tengo todo hasta el 1499". El emisor no sabe si se perdió solo uno o los tres. Eso es lo que resuelve **SACK** (sección 9).
- Los **ACK repetidos** ("duplicados") son una señal de que algo se perdió sin esperar al timer. TCP los usa para el **fast retransmit** (clase 9).

7. Control de flujo: la ventana deslizante

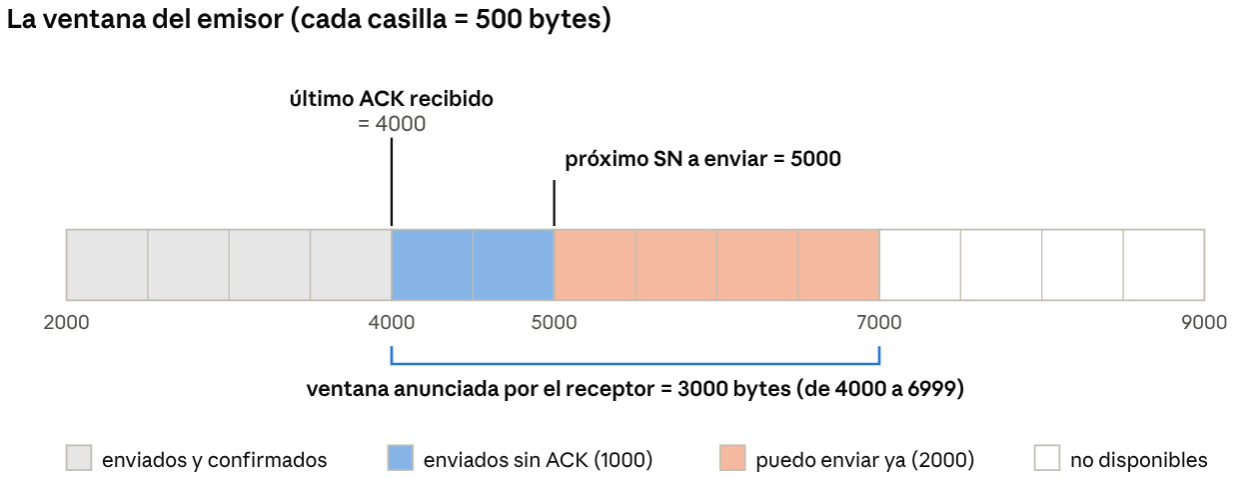
El problema

El receptor guarda lo que llega en un **buffer** hasta que la aplicación lo lee. Si el emisor manda más rápido de lo que la aplicación del receptor lee, el buffer se llena y lo que siga llegando se tira. El control de flujo evita eso: **el receptor le dice al emisor cuánto espacio libre le queda**, en el campo **Window** de cada segmento.

Las reglas de la clase

- **Cada extremo anuncia su ventana de recepción al inicio de la sesión**, y pueden ser distintas (en el diagrama de estados: A 32K, B 24K). Después la sigue anunciando en cada segmento.
- **Un ACK significa que TCP recibió el segmento, no que la aplicación lo leyó.** Los datos confirmados pueden seguir ocupando el buffer.
- Si un extremo manda más rápido de lo que el otro procesa y la aplicación no lee, el receptor **anuncia una ventana más chica**.
- **La ventana anunciada puede llegar a 0**: "no me mandes nada". El emisor se detiene hasta que la ventana vuelva a abrirse (y para no quedarse trabado usa el **persist timer**, sección 8).

La ventana del lado del emisor



Cuando llega un ACK nuevo, el borde izquierdo avanza y la ventana se desliza hacia la derecha

Las cuatro zonas del dibujo "Ventana deslizante" de la clase, con números

El emisor divide su flujo en **cuatro zonas** (las del dibujo de la clase):

1. **Enviados y confirmados**: ya están; se pueden borrar del buffer de envío.
2. **Enviados y no confirmados**: hay que guardarlos por si hay que retransmitirlos.
3. **OK para enviar**: entran en la ventana pero todavía no salieron.
4. **No disponibles**: están fuera de la ventana; no se pueden mandar aunque la aplicación ya los haya escrito.

La **ventana** son las zonas 2 y 3 juntas: empieza en el **último ACK recibido** y mide lo que el receptor anunció. La fórmula para cuentas:

> **Bytes que todavía puedo mandar = (ACK recibido + ventana anunciada) − próximo SN**

En el dibujo: (4000 + 3000) − 5000 = **2000 bytes**.

Por qué "deslizante": si llega un ACK = 4500 con ventana 3000, el borde izquierdo pasa a 4500 y el derecho a 7500. La ventana "se corre" hacia la derecha a medida que llegan ACKs. Si el receptor anuncia una ventana más chica, el borde derecho deja de avanzar (o la ventana se achica); con ventana 0, la zona 3 desaparece.

La ventana del lado del receptor

El receptor tiene tres zonas:

- **Recibidos y confirmados.**
- **Aceptables**: desde el **próximo byte esperado** (su ACK) hasta el **máximo aceptable** (ACK + ventana − 1). Lo que llega en este rango se guarda, aunque llegue desordenado.
- **No disponibles**: más allá de la ventana; si llega algo acá, se descarta.

El dibujo "Ventana deslizante – Ejemplo"

Muestra **qué campo de cada segmento mueve cada borde**:

- En el segmento **enviado**, el **Sequence Number** marca hasta dónde llegó el emisor (separa "enviados" de "a enviar").
- En el segmento **recibido**, el **Acknowledgment** marca el borde izquierdo (separa "acknowledged" de "enviados") y la **Window** dice cuánto se extiende la ventana a partir de ahí (hasta dónde llega "a enviar").

Es decir: el emisor nunca decide la ventana solo; **el borde derecho lo fija el receptor** con ACK + Window.

Límite del campo Window

Con 16 bits, la ventana máxima es **65535 bytes**. En una red rápida y con RTT largo eso no alcanza para mantener el enlace ocupado: con 64 KB por RTT de 100 ms, el máximo es 64 KB / 0,1 s ≈ 5 Mbps, por más rápido que sea el enlace. Para eso existe la opción **window scale** (sección 9).

8. Timers: RTO, Karn, persist y keepalive

TCP tiene varios relojes; cada uno resuelve un problema distinto de "esperar algo que quizás nunca llegue".

RTO: cuánto esperar un ACK antes de retransmitir

Cuando TCP manda un segmento arranca un timer, el **RTO (Retransmission Timeout)**. Si vence sin que llegue el ACK, retransmite. La pregunta es **cuánto vale el RTO**:

- **Demasiado largo → subutilización**: si se perdió un segmento, el emisor se queda parado esperando mucho más de lo necesario.
- **Demasiado corto → retransmisiones inútiles**: retransmite segmentos que no se perdieron, solo se demoraron, y carga más la red.

El valor correcto depende del **RTT (Round Trip Time)**, el tiempo de ida y vuelta, y el RTT **varía continuamente** (no es lo mismo una LAN que otro continente, y en una misma conexión cambia con la carga). Por eso el RTO no puede ser fijo: **se estima dinámicamente, por conexión, a partir del RTT medido**.

- **Muestra de RTT (RTT sample)**: el tiempo desde que se envía un byte con cierto número de secuencia hasta que vuelve el ACK que lo confirma.

Estimar el RTT

La clase lo justifica con la **desigualdad de Chebyshev**: para **cualquier** distribución de muestras, la probabilidad de que un valor se aleje más de k desvíos de la media es **como mucho $1/k^2$**. Entonces, si tomo como RTT máximo razonable:

> **Max RTT = Avg RTT + k · Desv**

con k = 4, la probabilidad de equivocarme (de que el ACK tarde más y retransmita sin necesidad) es ⩽ 1/16 ≈ 6 %. De ahí sale:

> **Timeout = Avg RTT + 4 · Desv**

El promedio se actualiza con cada muestra como un **promedio móvil exponencial**:

> **Avg RTT = (1 − y) · Avg RTT + y · Muestra RTT**, con y = 0,1

Es decir, el promedio nuevo es 90 % el viejo y 10 % la muestra nueva: se adapta a los cambios pero sin saltar con una muestra rara. El desvío se actualiza de forma parecida con la diferencia entre muestra y promedio (la diapositiva no da la fórmula; en el estándar, RFC 6298, es **Desv = (1 − ¼) · Desv + ¼ · |Muestra − Avg|**, y el promedio usa 1/8 en vez de 0,1).

Ejemplo: Avg = 100 ms, Desv = 10 ms, llega una muestra de 150 ms.

- Desv = 0,75 · 10 + 0,25 · |150 − 100| = 7,5 + 12,5 = **20 ms**
- Avg = 0,9 · 100 + 0,1 · 150 = **105 ms**

- Timeout = 105 + 4 · 20 = **185 ms**

En la práctica, el timeout se redondea a **múltiplos de 200, 500 o 1000 ms**, según la granularidad del reloj del SO.

Ambigüedades y el algoritmo de Karn

El dibujo "Ambigüedades" muestra el problema: **después de una retransmisión, el ACK que llega no dice de cuál de las dos copias es.**

Caso	Qué pasó en realidad	Si mido desde el envío original	Error
Izquierda	El original se perdió; el ACK es de la retransmisión	Cuento el RTO entero + el RTT real	Muestra **demasiado grande**
Derecha	El original llegó pero el ACK se demoró; retransmití de más	Si mido desde la retransmisión, el ACK (del original) llega enseguida	Muestra **demasiado chica**

Cualquiera que elija puede estar mal. El **estimador de Karn** lo resuelve así:

1. **Ante una retransmisión, no actualiza el RTT**: las muestras de segmentos retransmitidos se ignoran.
2. **Timer backoff: RTO = 2 · RTO.** Si hubo que retransmitir, probablemente el RTO era corto o la red está congestionada: se duplica (y se sigue duplicando en cada retransmisión del mismo segmento).
3. **Vuelve a estimar después de una transmisión exitosa** (un segmento que llegó y se confirmó sin retransmitirse).

Y la clase agrega que **las muestras se toman con la opción Timestamp** (sección 9): el ACK devuelve la marca de tiempo del segmento que confirma, así que no hay ambigüedad posible.

Persist timer (RFC 1122): no quedarse trabado con ventana 0

El deadlock: B anuncia ventana 0 y A se detiene. Cuando la aplicación de B lee y libera espacio, B manda un ACK con la ventana nueva. **Si ese ACK se pierde**, A sigue creyendo que la ventana es 0 y B espera datos que nunca llegan. Nadie retransmite, porque **los ACK sin datos no se retransmiten**: los dos esperan para siempre.

La solución es un *polling* por parte del emisor:

- Al **recibir una ventana 0**, el emisor activa el **persist timer**, normalmente **5 segundos**.

- Cuando vence, manda un **segmento de prueba de 1 byte** (*window probe*) para preguntar el estado del receptor.
- El receptor contesta con su ventana actual. Si sigue en 0 (ACK que **no confirma** el byte de prueba, con win = 0), el timer hace **back-off binario**: 10, 20, 40 segundos…
- **Diferencia con el RTO**: el RTO se rinde después de cierta cantidad de intentos; el persist **sigue probando indefinidamente** hasta que la ventana se abra o se haga un reset.

Keepalive timer: ¿sigue vivo el otro?

El problema: en TCP, si no hay datos, **no viaja nada**. Una conexión *idle* puede quedar "viva" para siempre, incluso si el otro extremo se cayó o se reinició. Un servidor acumularía conexiones muertas ocupando recursos.

El keepalive interroga al otro extremo por su estado:

- Si no hay actividad durante un tiempo configurable (**2 horas** por defecto), el extremo que lo tiene habilitado manda un **segmento de prueba**, parecido al del persist timer, o vacío como un ACK con un número de secuencia inesperado (que obliga al otro a responder).
- Según cómo esté el otro extremo:

Estado del otro	Qué pasa
Activo	Responde. Se reinicia el timer por otras 2 horas (y se reinicia cada vez que hay tráfico).
Caído o reiniciándose	No responde. Timeout de **75 s**, se repite **10 veces** cada 75 s; si nunca responde, **se cierra la conexión**.
Ya reinició (fin-reboot)	Responde con un **reset**, porque perdió el estado de esa conexión (es una half-open, sección 4).
Activo pero inalcanzable	Igual que "caído": el emisor no recibe respuesta, timeout de 75 s y retransmite la prueba.

Por qué es polémico (lo que dice la clase): no está universalmente aceptado y no se recomienda en general, porque **una falla transitoria** (un enlace que se cae un rato) puede hacer que se corte una conexión que se habría recuperado sola, y porque **consume ancho de banda** como cualquier tráfico de control. Pero **a veces es necesario**: por ejemplo en Telnet, donde un servidor necesita liberar las sesiones de clientes que se apagaron sin cerrar.

9. Opciones, wrap around, PAWS y SACK

Cómo están armadas las opciones

- Hay **hasta 40 bytes** de opciones (60 de cabecera máxima menos 20 fijos).
- Dos categorías: de **un byte** (solo el tipo) o de **varios bytes**.
- Todas empiezan con un byte **Kind** que dice qué opción es. Las de varios bytes siguen con un byte **Length** (el largo **total** de la opción, contando Kind y Length) y después los datos.
- La cabecera tiene que medir un múltiplo de 4 bytes (Header Length cuenta palabras de 4), así que se rellena con **NOP**.

La tabla de opciones (de *TCP/IP Illustrated*)

Kind	Length	Nombre	Para qué
0	1	**EOL** (End of Option List)	"No hay más opciones"
1	1	**NOP** (No Operation)	Relleno, para que la opción siguiente empiece alineada a 4 bytes
2	4	**MSS**	Tamaño máximo de segmento que espero recibir (16 bits). Solo en SYN
3	3	**WSopt** (Window Scale)	Factor de escala de la ventana. Solo en SYN
4	2	**SACK-Permitted**	"Puedo usar SACK". Solo en SYN
5	variable	**SACK**	Bloques recibidos fuera de orden
8	10	**TSopt** (Timestamps)	Marcas de tiempo para RTT y PAWS
28	4	UTO (User Timeout)	Cuánto esperar antes de abortar una conexión sin respuesta
29	variable	TCP-AO	Autenticación de segmentos
253, 254	variable	Experimentales	Reservadas para pruebas

(La RFC 1323, que definía window scale y timestamps, fue actualizada por la **RFC 7323**.)

Window Scale

- **Problema:** la ventana tiene 16 bits (máx. 65535 bytes) y no alcanza en enlaces rápidos con RTT largo (sección 7).
- **Solución:** un **exponente** (*shift count*) que se manda **solo en el handshake**. A partir de ahí, la ventana real es **Window × 2^shift** (el valor del campo corrido *shift* bits a la izquierda).
- Ejemplo: shift = 7 y Window = 1000 → ventana real = 1000 × 128 = **128.000 bytes**. El máximo shift es 14, que lleva la ventana a casi 1 GB.
- Formato: Kind 3, Length 3, y un byte con el shift.

Timestamps

- Kind 8, Length 10: **TSval** (mi marca de tiempo, 4 bytes) y **TSecr** (*echo reply*: la marca del otro que estoy devolviendo, 4 bytes).
- Se le anteponen **dos NOP** para que quede alineada a 32 bits (2 + 10 = 12 bytes).
- **Para qué:** medir el **RTT de cada segmento confirmado**: el emisor pone su TSval, el receptor lo devuelve en TSecr, y el emisor resta. Esto elimina la ambigüedad de las retransmisiones (sección 8). Y es la base de **PAWS**.

Wrap around: cuando los números de secuencia dan la vuelta

El número de secuencia tiene 32 bits. A velocidades altas, el emisor puede **recorrer todo el espacio y volver a usar los mismos números** mientras un segmento viejo todavía anda dando vueltas por la red. Si ese segmento llega tarde, podría tomarse como válido y meter datos viejos en el lugar equivocado.

Para que no pase, el tiempo que tarda en reciclarse el espacio tiene que ser mayor que la vida máxima de un segmento (**MSL**):

> **$T_{wrap} = 2^{31} / B > MSL$**, con B = velocidad efectiva en bytes por segundo

Se usa 2^{31} y no 2^{32} porque TCP compara números de secuencia en forma circular: un número está "adelante" de otro si está a menos de media vuelta (2^{31}).

Red	B (MBps)	T_wrap (s)
Ethernet (10 Mbps)	1,25	$2^{31} / 1{,}25 \cdot 10^6 \approx$ **1700**
FDDI (100 Mbps)	12,5	≈ **170**
Gigabit	125	≈ **17**

Con un MSL típico de 2 minutos (120 s), **a Gigabit el espacio se recicla en 17 s**: ya no alcanza. Por eso hace falta PAWS.

PAWS (Protection Against Wrapped Sequences, RFC 7323)

- **Rechaza segmentos duplicados y "reencarnaciones"** (segmentos viejos cuyos números de secuencia coinciden con los actuales por la vuelta).
- **Usa la opción Timestamp**: es como agregarle más bits al número de secuencia.
- Supone que el **TS de los segmentos que llegan es monótono no decreciente** (cada segmento nuevo tiene una marca igual o mayor que la anterior).
- Si llega un segmento con **TS menor que uno ya recibido**, se descarta como duplicado viejo, aunque su número de secuencia parezca válido.
- "Menor" se define en forma circular: **$s < t$ si $0 < (t - s) < 2^{31}$**, porque el timestamp también es de 32 bits y también da la vuelta.
- Los TS que van en el **SYN y el SYN+ACK inicializan** PAWS.
- **No hace falta sincronizar relojes** entre los dos extremos: cada uno solo compara los timestamps del otro entre sí, nunca contra su propio reloj.

SACK (Selective Acknowledgment, RFC 2018)

El problema (visto en la sección 6): el ACK acumulativo solo dice "tengo todo hasta X". Si se perdió un segmento en el medio, el emisor no sabe qué llegó después y puede terminar retransmitiendo de más o de a uno por RTT. **SACK permite la retransmisión selectiva**: el receptor dice exactamente qué bloques tiene.

Son **dos opciones**:

- **SACK-Permitted** (Kind 4, Length 2): va en el **SYN** del handshake y habilita SACK para cuando la conexión esté establecida. Los dos tienen que mandarla.
- **SACK** (Kind 5, largo variable): va en los ACK y lista **bloques** de datos recibidos correctamente fuera de orden.
  - **Left Edge**: el primer número de secuencia del bloque.
  - **Right Edge**: el número **siguiente** al último byte del bloque (igual que un ACK).
  - Se anteponen NOP para alinear a 32 bits.
  - **Máximo 4 bloques**: cada bloque ocupa 8 bytes, y $2 + 4 \times 8 = 34 \leqslant 40$. Si además va la opción Timestamps (12 bytes), solo entran 3.

Error en la diapositiva: el dibujo de SACK-Permitted muestra los campos "00000010 | 00000010 | Max. Seg. Size", copiado del de MSS. El correcto es **Kind = 4 (00000100), Length = 2** y nada más: son solo 2 bytes.

Los ejemplos de SACK de la clase

El transmisor manda **8 segmentos de 500 bytes desde el SN 5000**: 5000, 5500, 6000, 6500, 7000, 7500, 8000 y 8500.

Caso 1: se pierde el primero (5000) y llegan los otros 7. El ACK queda clavado en 5000 y el bloque SACK va creciendo:

Llega	ACK	Bloque SACK
5500	5000	5500–6000
6000	5000	5500–6500
6500	5000	5500–7000
…	5000	…
8500	5000	5500–9000

Con esto el emisor sabe que **solo** le falta 5000–5499 y retransmite solo ese.

Caso 2: se pierden los segmentos pares (5500, 6500, 7500, 8500):

Llega	ACK	1.er bloque	2.º bloque	3.er bloque
5000	5500	—	—	—
6000	5500	6000–6500		
7000	5500	7000–7500	6000–6500	
8000	5500	8000–8500	7000–7500	6000–6500

El **primer bloque es siempre el del segmento que disparó el ACK** (el más reciente), y después se repiten los anteriores, por si se perdió un ACK previo.

Continuación: llega el cuarto segmento (6500). Llena el hueco entre 6000–6500 y 7000–7500, y esos dos bloques se **fusionan**:

Llega	ACK	1.er bloque	2.º bloque
6500	5500	**6000–7500**	8000–8500

Después llega el segundo (5500). Ahora todo es contiguo hasta el 7499, así que **el ACK acumulativo salta** y solo queda un bloque suelto:

Llega	ACK	1.er bloque
5500	**7500**	8000–8500

Lo único que falta es 7500–7999 y 8500–8999, que el emisor puede deducir exactamente.

10. El ejemplo resuelto de la Práctica 4

Está en el `ejemplos.zip` ("Ejemplo TCP"). Lo repaso con el razonamiento detrás de cada número.

Enunciado: A quiere abrir una sesión TCP con B.

	ISN	MSS	Ventana
A	21578	1220	64 KB
B	68921	1460	48 KB

(La cátedra toma **1 KB = 1000 bytes**: 64 KB = 64000 B y 48 KB = 48000 B. Las dos entran en los 16 bits del campo Window, así que no hace falta window scale.)

a) El handshake

#	Sentido	Flags	SN	ACK	Opciones / ventana
1	A → B	SYN	21578	—	MSS = 1220, W = 64000
2	B → A	SYN + ACK	68921	21578 + 1 = **21579**	MSS = 1460, W = 48000
3	A → B	ACK	**21579**	68921 + 1 = **68922**	

Los +1 son porque cada SYN consume un número de secuencia (sección 3).

b) ¿Cuántos segmentos de tamaño máximo puede mandar A sin esperar un ACK al comenzar?

Respuesta de la cátedra: 1, porque la sesión empieza en **slow start** (clase 9): la ventana de congestión arranca en 1 MSS.

Ojo con el razonamiento, porque hay dos límites y manda el más chico:

- Si solo miráramos **control de flujo**: la ventana de B es 48000 y el tamaño de segmento es el mínimo de los MSS (1220) → 48000 / 1220 = 39,3 → **39 segmentos**.

- Pero el **control de congestión** limita a 1 segmento al principio. El emisor manda lo que permita **el mínimo entre la ventana del receptor y la de congestión**.

(Hoy los sistemas suelen arrancar con una ventana inicial de hasta 10 segmentos, RFC 6928. Para la materia, usá 1 como en la resolución.)

c) ¿Cuántos segmentos de tamaño máximo manda B a A para llenar la ventana máxima, sin congestión?

- Lo que limita a B es la **ventana de A: 64000 bytes**.
- B anunció MSS 1460, pero **A solo acepta segmentos de hasta 1220** (su MSS). Entonces B manda segmentos de **1220**.
- 64000 / 1220 = 52,46 → **52 segmentos completos** (52 × 1220 = 63440) **y uno más de 64000 − 63440 = 560 bytes**.

La trampa del ejercicio es usar el MSS de B (1460). **El MSS que anuncia cada uno es lo que él puede recibir**, así que para mandarle a A se usa el de A.

d) El cierre

- **Escenario 1: A cierra y B no tiene más datos → cierre de 3 segmentos**: A manda FIN, B responde FIN+ACK, A manda ACK.
- **Escenario 2: A cierra pero B todavía quiere mandar datos → cierre de 4 segmentos** con half-close: A manda FIN, B responde ACK, B sigue mandando datos (A solo confirma), B manda su FIN y A responde ACK.

Es exactamente el dibujo de la sección 4.

11. Preguntas para autoevaluarte

1. ¿Qué diferencia hay entre control de flujo y control de congestión? ¿A quién protege cada uno?
2. Una aplicación hace tres `send` de 100 bytes por TCP. ¿Cuántos segmentos salen? ¿Y con UDP?
3. Un segmento tiene SN = 7000 y lleva 1460 bytes. ¿Qué ACK manda el receptor si todo llegó bien?
4. ¿Por qué el ACK del SYN es ISN + 1 si el SYN no lleva datos?
5. Calculá el MSS de una conexión sobre un enlace con MTU 1400, sin opciones IP ni TCP. ¿Y si es IPv6?
6. Un extremo anuncia MSS 1460 y el otro 536. ¿De qué tamaño son los segmentos en cada sentido?

7. ¿Qué es un half-close y en qué cierre aparece? ¿En qué se diferencia de una half-open connection?
8. ¿Por qué existe el estado TIME_WAIT y quién pasa por él?
9. Un servidor tiene 300 conexiones en CLOSE_WAIT. ¿Qué sospechás?
10. El emisor recibió ACK = 10000 con ventana 4000 y ya mandó hasta el byte 12999. ¿Cuántos bytes más puede mandar?
11. Avg RTT = 200 ms, Desv = 20 ms, y llega una muestra de 300 ms. Calculá el nuevo promedio con y = 0,1 y el timeout (usá Desv = 30 ms para el cálculo final).
12. ¿Qué problema resuelve el algoritmo de Karn y qué hace?
13. ¿Por qué sin el persist timer dos extremos podrían quedar esperando para siempre?
14. Nombrá un argumento a favor y uno en contra del keepalive.
15. ¿Por qué en Gigabit hace falta PAWS y en Ethernet de 10 Mbps no?
16. Se mandan segmentos de 1000 bytes desde SN 1000 (1000, 2000, 3000, 4000) y se pierde el de 2000. ¿Qué ACK y qué bloque SACK manda el receptor al recibir el de 4000?

Respuestas rápidas

1. El de flujo protege al receptor (no llenar su buffer, con la ventana anunciada); el de congestión protege a la red (no saturar routers, con la ventana de congestión).
2. Con TCP, no se sabe: TCP puede juntarlos en uno o mandarlos por separado (byte stream). Con UDP, siempre 3 datagramas.
3. ACK = 7000 + 1460 = **8460**.
4. Porque el SYN consume un número de secuencia, para poder ser confirmado como cualquier byte.
5. IPv4: 1400 − 20 − 20 = **1360**. IPv6: 1400 − 40 − 20 = **1340**.
6. En los dos sentidos se usa el mínimo, 536 (y en todo caso, nadie puede mandarle a un extremo más que el MSS que él anunció).
7. Half-close: uno mandó FIN y el otro sigue mandando datos; aparece en el cierre de 4 segmentos. Half-open: un extremo perdió el estado (se cayó o reinició) y el otro cree que la conexión sigue; se resuelve con un RST.
8. Para poder reenviar el último ACK si se pierde y para que mueran los segmentos viejos antes de reusar los mismos puertos. Pasa el que cierra primero (active close).
9. Que la aplicación no está cerrando los sockets: el otro extremo ya mandó FIN y el programa nunca hace `close`.
10. (10000 + 4000) − 13000 = **1000 bytes**.
11. Avg = 0,9 · 200 + 0,1 · 300 = **210 ms**; timeout = 210 + 4 · 30 = **330 ms**.

12. La ambigüedad de no saber si un ACK corresponde al original o a la retransmisión. No usa muestras de segmentos retransmitidos, duplica el RTO (backoff) y vuelve a estimar tras una transmisión exitosa.
13. Porque si se pierde el ACK que reabre la ventana, nadie lo retransmite (los ACK sin datos no se retransmiten): el emisor espera la ventana y el receptor espera datos.
14. A favor: libera recursos de conexiones con el otro extremo caído (ej. Telnet). En contra: puede cortar conexiones por fallas transitorias y consume ancho de banda.
15. Porque a 125 MBps el espacio de 2^{31} se recicla en unos 17 s, menos que el MSL; a 1,25 MBps tarda unos 1700 s, mucho más que el MSL.
16. ACK = **2000**, bloque SACK **3000–5000** (tiene del 3000 al 4999 contiguo).

Clase 9 · TCP: control de congestión

Oct 4, 2026 · @Joaquin

Apunte para leer a la par de las diapositivas de control de congestión en TCP.

1. Qué es la congestión

Control de flujo frente a control de congestión

El dibujo de las canillas de la clase separa dos problemas que se parecen pero no son lo mismo:

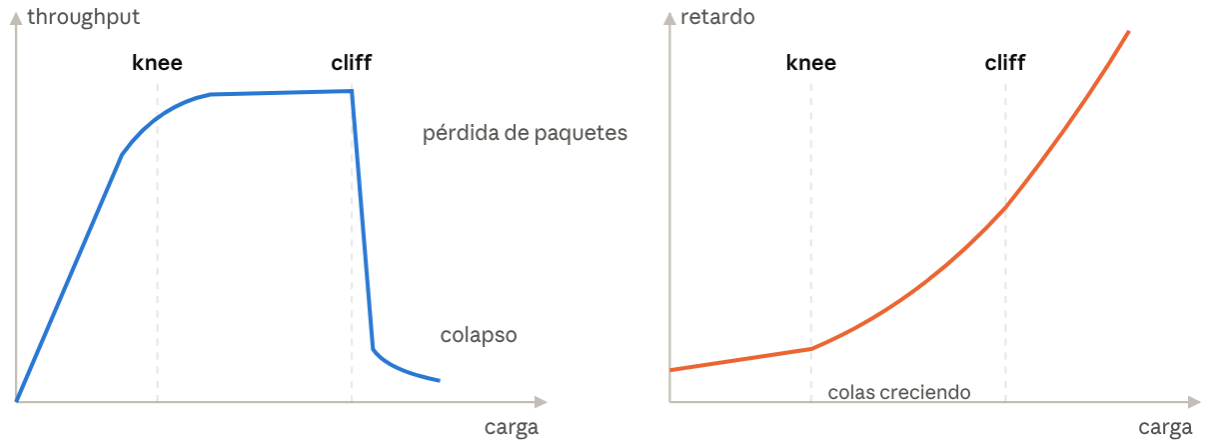
	Control de flujo	Control de congestión
El dibujo	Red de gran capacidad que llena un **vaso chico**	Un **cuello de botella** en el caño, aunque el balde sea enorme
Dónde está el límite	En el **receptor** (su buffer)	En **la red** (routers y enlaces intermedios)
Quién avisa	El receptor, con la **ventana anunciada** (clase 7)	Nadie avisa explícitamente: el emisor **lo deduce** (pérdidas, demoras)
Variable en TCP	rcv_win	**cwnd**

En los dos casos la solución es "ajustar la velocidad de la canilla", pero por razones distintas. Un receptor potente detrás de un enlace saturado tiene problema de congestión, no de flujo.

Qué pasa en la red cuando hay demasiada carga

Los routers tienen **colas** (buffers). Si llegan paquetes más rápido de lo que el enlace de salida los puede despachar, la cola crece; cuando se llena, **los paquetes nuevos se descartan**.

Más carga no siempre es más throughput



Evitar congestión: quedarse antes del knee. Controlar congestión: no pasar el cliff.

Los dos gráficos de "Throughput vs. Congestión" de la clase, redibujados

Los gráficos muestran tres zonas a medida que aumenta la carga (lo que los emisores meten en la red):

1. **Antes del knee ("rodilla")**: el throughput crece casi igual que la carga y el retardo es bajo. La red está holgada.
2. **Entre el knee y el cliff ("acantilado")**: el throughput casi no crece (el enlace ya está casi lleno), pero **el retardo sí crece** porque las colas se van llenando. Meter más no da más; solo agrega espera.
3. **Después del cliff**: las colas desbordan y hay **pérdida de paquetes**. Los emisores retransmiten, eso agrega todavía más carga, se pierden más… y el throughput útil **cae en picada: colapso por congestión**. La red está ocupada transportando retransmisiones.

Controlar o evitar

- **Controlar** la congestión = **mantenerse a la izquierda del cliff**: reaccionar cuando hay pérdidas para no caer al colapso. Es lo que hace TCP Reno (secciones 3 a 5).
- **Evitar** la congestión = **mantenerse a la izquierda del knee**: detectar que las colas empiezan a crecer y frenar antes de perder nada. Es lo que intenta TCP Vegas (sección 6).

El punto ideal de operación es el knee: máximo throughput con mínimo retardo.

Dos modelos: ¿quién se encarga?

	Modelo end-to-end	Modelo basado en la red
Quién actúa	**Los extremos**, que son la fuente de la demanda	**Los nodos de la red** (routers), que controlan el tráfico
Cómo	Estiman el grado de congestión (por pérdidas y tiempos) y **reducen lo que mandan**	Descartan, marcan o regulan tráfico directamente
Supuesto	Los nodos intermedios monitorean el estado de la red, pero la decisión es del extremo	**Los extremos no son confiables** (pueden no cooperar)
Ventaja	No requiere nada especial en los routers	**Acciones más rápidas**: el router ve la cola directamente

TCP usa el modelo end-to-end: IP no le avisa nada (salvo ECN, que es opcional), así que cada emisor tiene que inferir la congestión solo, y confiar en que los demás hagan lo mismo. Por eso un emisor UDP que no frena (clase 6) puede perjudicar a todos los TCP que sí lo hacen.

2. Las tres variables de TCP

TCP implementa el modelo end-to-end con **tres variables** por conexión:

Variable	Qué es	Quién la fija
cwnd (*congestion window*)	Cuánto cree el emisor que **la red** aguanta sin congestionarse	El **emisor**, con los algoritmos de esta clase
rcv_win (*receiver window*)	Cuánto puede recibir **el receptor**	El **receptor**, publicada en el campo Window de cada segmento (clase 7)
ssthresh (*slow start threshold*)	El **umbral** que decide si cwnd crece rápido (slow start) o despacio (congestion avoidance)	El **emisor**; se actualiza cuando detecta congestión

La ventana que realmente usa el emisor es:

> **ventana = min(rcv_win, cwnd)**

Es decir, el emisor respeta **el límite más estricto** de los dos: el del receptor o el de la red. Por eso en el ejercicio de la Práctica 4 (clase 7) A puede mandar un solo segmento al principio aunque B le ofrezca 48000 bytes: cwnd arranca en 1.

Unidades: en las diapositivas y en los ejercicios, cwnd y ssthresh se cuentan en **segmentos** (MSS). En la implementación real se cuentan en bytes, pero la idea es la misma.

3. Slow start

Para qué sirve

Al empezar una conexión, el emisor **no tiene idea de cuánta capacidad tiene la red**. Slow start resuelve eso:

- **Inicializa el sistema y descubre la congestión rápidamente.**
- **Incrementa cwnd hasta que aparece congestión.**
- Así **estima el cwnd óptimo**.
- **Detecta la congestión por la pérdida de segmentos**: TCP no tiene otra señal (en el modelo end-to-end, una pérdida se interpreta como cola llena en algún router).

Las reglas

1. **Al comienzo, o después de una congestión: cwnd = 1.**
2. **Por cada segmento validado (confirmado por un ACK): cwnd = cwnd + 1.**
3. **Se detiene cuando cwnd ⩾ ssthresh**, y ahí pasa a congestion avoidance (sección 4).

Por qué "pese al incremento unitario resulta exponencial"

El nombre engaña: slow start **no es lento**. Sumar 1 por cada segmento confirmado significa que, en cada RTT, **cwnd se duplica**. Si mandé 4 segmentos y vuelven sus 4 ACKs, sumo 4: paso de 4 a 8. Es lo que muestra el ejemplo de la clase:

RTT	cwnd	Segmentos que salen	Al volver los ACK
1	1	segmento 1	+1 → cwnd = 2
2	2	segmentos 2 y 3	+2 → cwnd = 4
3	4	segmentos 4, 5, 6 y 7	+4 → cwnd = 8
4	8	…	

Después de n RTT, cwnd = 2^n. Se llama "slow" por comparación con lo que se hacía antes de que existiera: mandar de golpe toda la ventana del receptor.

Un detalle del dibujo: el receptor manda un solo "ACK for segments 2+3" y uno para "4+5+6+7" (ACKs acumulativos), y aun así cwnd llega a 4 y a 8. Eso cierra porque la regla de la clase cuenta **segmentos validados**, no mensajes ACK. Si se contara +1 por cada mensaje ACK, el crecimiento sería más lento.

Desventajas que nombra la clase

- **Detección tardía**: la congestión se descubre recién cuando ya se perdió algo, es decir, cuando ya se pasó del cliff.
- **En enlaces de alta velocidad, las ventanas son mayores y la pérdida también**: como cwnd se duplica, en el último RTT antes de la pérdida se manda el doble de lo que la red aguantaba, y se puede perder hasta la mitad de esa ventana de golpe.
- **Interacción con el algoritmo de retransmisión**: si se pierden muchos segmentos a la vez, hay que esperar RTOs, que también se duplican (Karn, clase 7), y la conexión queda parada.

4. Congestion avoidance (AIMD) y el timeout

Una vez que cwnd alcanza ssthresh, seguir duplicando sería temerario: ssthresh marca la zona donde antes hubo problemas. La estrategia de la clase tiene dos mitades, que juntas se llaman **AIMD** (*Additive Increase, Multiplicative Decrease*):

Incremento aditivo (additive increase)

- Se arranca desde ssthresh y **se incrementa cwnd lentamente**.
- Regla: **si cwnd > ssthresh, por cada ACK: cwnd = cwnd + 1/cwnd**.
- Efecto: **cwnd sube 1 por RTT**, y solo **si todos los segmentos de la ventana fueron confirmados**. ¿Por qué? Si cwnd = 10, en un RTT vuelven 10 ACKs y cada uno suma 1/10: en total, +1.

Es decir: en slow start se suma 1 **por ACK** (se duplica por RTT); en congestion avoidance se suma 1 **por RTT** (crece linealmente). Esa es la diferencia que hay que tener clara.

Disminución multiplicativa (multiplicative decrease)

- **Si se detecta una pérdida, se corta la ventana drásticamente.**
- Ante un **timeout** (venció el RTO): **ssthresh = ventana / 2** y **cwnd = 1**. Se vuelve a slow start, pero ahora con un umbral más bajo.

Por qué asimétrico (subir de a 1, bajar a la mitad o a 1): si la red está congestionada, cada RTT que pasa con demasiado tráfico empeora todo (retransmisiones, colapso). Reaccionar fuerte es lo seguro. En cambio, subir de a poco es la forma de "tantear" la capacidad sin volver a romperla. Además, AIMD hace que varias conexiones que comparten un enlace **tiendan a repartirlo en partes iguales**.

El pseudocódigo de la clase ("Armando el rompecabezas"), completado

```
Inicialmente:
    cwnd = 1
    ssthresh = ∞

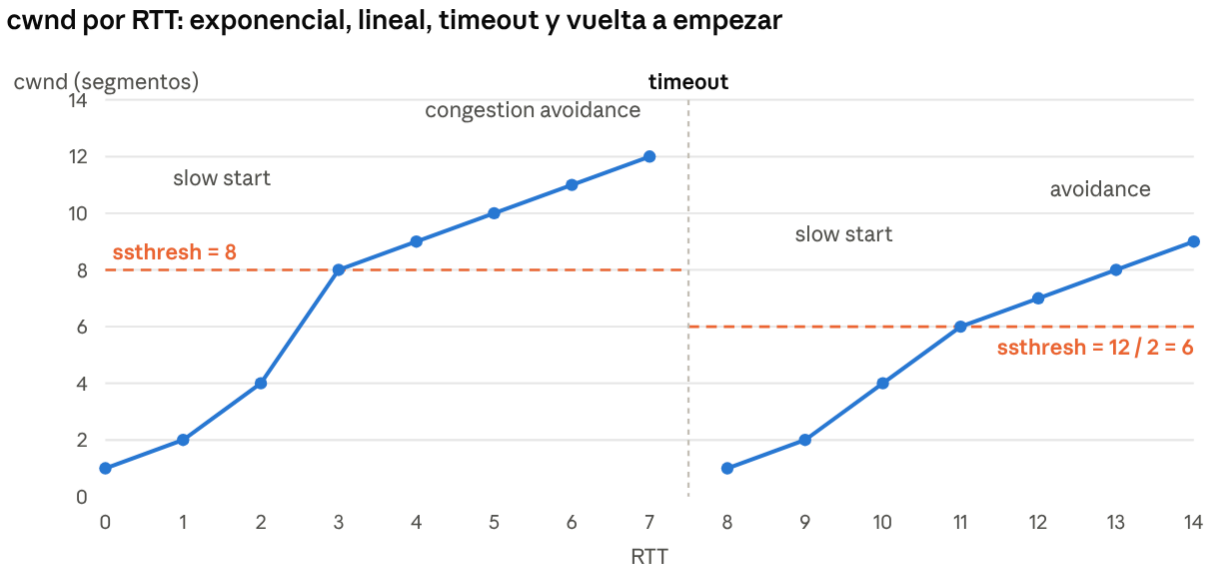
Mientras (próx < unack + win):        # hay lugar en la ventana
    transmitir el segmento próx
    con win = min(cwnd, rcv_win)

Al recibir un ACK:
    si cwnd < ssthresh:  cwnd = cwnd + 1          # slow start
    si no:               cwnd = cwnd + 1/cwnd     # congestion avoidance

Al vencer el RTO (timeout):              # multiplicative decrease
    ssthresh = win / 2
    cwnd = 1
```

- **unack** es el primer byte enviado y no confirmado (el borde izquierdo de la ventana), y **próx** es el próximo a enviar. Mientras próx no llegue a unack + win, se puede seguir transmitiendo. Es la ventana deslizante de la clase 7, con win = min(cwnd, rcv_win).
- La diapositiva solo muestra la rama de slow start del ACK; la otra rama es la de la diapositiva "Estrategia" (+1/cwnd). Las junté.
- Como **ssthresh arranca en ∞**, el primer slow start sigue hasta que haya una pérdida: es el que "descubre" la capacidad.

Los gráficos de la clase



Es el gráfico "Combinando" (ssthresh = 8) seguido de un timeout, como en "Finalmente"

- **"Combinando"** (ssthresh = 8): cwnd sigue 1, 2, 4, 8 (exponencial) y desde ahí 9, 10, 11, 12 (lineal).
- **"Finalmente"**: slow start, timeout, caída a 1, slow start hasta la mitad de donde estaba, congestion avoidance hasta el próximo timeout, y así. Es el **diente de sierra** típico de TCP.

En el dibujo de arriba, cuando ocurre el timeout cwnd valía 12: ssthresh pasa a 6 y cwnd a 1. Al volver a crecer, 1, 2, 4 y **6** (no 8: slow start no puede pasar el umbral, así que se recorta en ssthresh), y desde ahí 7, 8, 9… Ese recorte es la convención habitual en los ejercicios.

Detectar pérdidas sin esperar el RTO

La clase cierra esta parte con un problema: **esperar el RTO es caro**.

- El RTO suele ser del orden de **dos veces el RTT** (o más), y durante ese tiempo no sale nada.
- Encima, volver a cwnd = 1 en cada pérdida **degrada mucho la performance**.
- Propuesta: **no esperar el RTO** cuando se pueda, usando **mecanismos alternativos**, y dejar el RTO solo para cuando esos mecanismos no actúan. Eso es la sección siguiente.

5. Fast retransmit, fast recovery y TCP Reno

De dónde salen los ACK duplicados

- **Frente a un segmento fuera de orden, el receptor tiene que mandar un ACK** igual, inmediatamente.
- Como el ACK es acumulativo (clase 7), ese ACK **repite el último número**: "sigo esperando el byte X". Eso es un **ACK duplicado**.
- La duplicación puede deberse a dos cosas: **un paquete perdido** (el X no va a llegar) o **un reordenamiento** (el X viene atrás, por otro camino).
- **El emisor no puede distinguir** una cosa de la otra mirando un solo ACK duplicado.

Fast retransmit

- Regla: **si se reciben 3 ACKs duplicados, se considera que hay un paquete perdido.**
- **Al recibir el tercer ACK repetido, se retransmite el segmento sin esperar el RTO.**

Por qué 3: un reordenamiento normal suele generar uno o dos duplicados (el segmento atrasado llega enseguida). Tres seguidos es muy improbable por reordenamiento, así que se apuesta a pérdida. Es el compromiso entre reaccionar rápido y no retransmitir de más.

Ejemplo: se mandan los segmentos 1 a 6 y se pierde el 2. Llegan el 3, 4, 5 y 6, y cada uno dispara un "ACK = 2". El primero es el ACK normal del 1; los siguientes son duplicados. **Al tercer duplicado** (generado por el 5), el emisor retransmite el 2 sin esperar ningún timer.

Fast recovery

Después de un fast retransmit, ¿hay que volver a cwnd = 1 como en un timeout? **No.** La lógica es: si llegan ACKs duplicados, quiere decir que **los segmentos posteriores al perdido sí están llegando**. La red no está colapsada; se perdió uno. Castigar con cwnd = 1 sería exagerado.

Entonces:

1. **ssthresh = cwnd / 2** (igual que antes, se baja el umbral a la mitad).
2. **cwnd = ssthresh** (no 1): la ventana queda a la mitad, no en el piso.
3. Se sigue directamente en **congestion avoidance** (crecimiento lineal), **sin pasar por slow start**.

(En la implementación real, mientras siguen llegando duplicados, cwnd se "infla" en 1 por cada uno, porque cada duplicado indica que un segmento salió de la red; cuando llega el ACK nuevo, vuelve a ssthresh. Para los ejercicios alcanza con cwnd = ssthresh.)

Sobre la diapositiva: el texto dice "Luego se ejecuta *congestion avoidance*? NO. Slow Start". Leído literalmente parece decir lo contrario de lo que corresponde. Lo correcto, y lo que dice la diapositiva siguiente ("Integrando"), es: **después de fast retransmit se ejecuta congestion avoidance, NO slow start**. Eso es fast recovery.

Integrando: TCP Reno

Situación	Qué hace	Por qué
Comienzo de la conexión	**Slow start** (cwnd = 1, crece exponencial)	No sabe nada de la red
cwnd llega a ssthresh	**Congestion avoidance** (+1 por RTT)	Zona de riesgo: tantear de a poco
3 ACKs duplicados	**Fast retransmit + fast recovery**: ssthresh = cwnd/2, cwnd = ssthresh, sigue en **congestion avoidance**	Pérdida aislada: la red todavía funciona
Vence el RTO	ssthresh = win/2, **cwnd = 1**, **slow start**	No llega nada: congestión grave

Todo junto es **TCP Reno**. Lo que hay que entender es que **la reacción es proporcional a la gravedad de la señal**: tres duplicados son una señal leve (algo se perdió, pero lo demás pasa), un timeout es una señal grave (no pasa nada).

Tahoe (la versión anterior, no aparece en las diapositivas pero se nombra mucho): ya tenía slow start, congestion avoidance y fast retransmit, pero **no fast recovery**: ante 3 duplicados también volvía a cwnd = 1. Reno es Tahoe + fast recovery.

6. TCP Vegas: evitar en vez de reaccionar

El problema de Reno que Vegas ataca

Reno **necesita perder paquetes para saber que hay congestión**. Su ciclo es: subir hasta que algo se pierde (pasar el cliff), cortar a la mitad, volver a subir… Eso produce **oscilaciones** (el diente de sierra) y llena las colas de los routers una y otra vez, con el retardo que eso implica. Vegas, de **1994**, intenta **quedarse cerca del knee** (sección 1): detectar que las colas empiezan a llenarse **antes** de perder nada.

Las tres novedades de la clase

1. **Crecimiento más lento que el slow start** clásico: duplica cwnd cada dos RTT en vez de cada uno, para no pasarse tanto en el primer "descubrimiento".

2. **Nuevo mecanismo de retransmisión**: **chequea el timeout al recibir el primer ACK duplicado**. Si el segmento lleva más tiempo afuera que el RTT estimado, lo retransmite ahí mismo, sin esperar tres duplicados ni el RTO.
3. **Nuevo algoritmo de congestion avoidance**, basado en el **RTT**, no en las pérdidas:
   - **Evita las oscilaciones de Reno.**
   - **Monitorea la diferencia entre el throughput esperado y el real.**
   - **Trata de reducir a cero los paquetes almacenados en los buffers de los routers** (en la práctica, mantener unos pocos).

La idea del algoritmo, sin fórmulas

- Si la red no tuviera colas, el RTT sería el mínimo posible: **d_s** (el menor RTT que vio la conexión, el "RTT base").
- El RTT que se mide ahora es **$D_s(t)$**. Si es mayor que d_s, la diferencia es **tiempo esperando en colas**.
- Con ventana w, el throughput **esperado** (sin colas) es **w / d_s**, y el **real** es **$w / D_s(t)$**.
- Si los dos se parecen: no hay casi nada encolado, **hay lugar para crecer**. Si el real es bastante menor que el esperado: estoy llenando colas, **tengo que bajar**.

La fórmula de la diapositiva "Ventana TCP Vegas"

Sea **$dif = w_s(t)/d_s - w_s(t)/D_s(t)$** (throughput esperado menos real). Entonces:

Si…	Nueva ventana $w_s(t+1)$	Lectura
$dif < \alpha_s$	$w_s(t) + 1/D_s(t)$	Casi sin colas: subo un poco
$dif > \beta_s$	$w_s(t) - 1/D_s(t)$	Demasiado encolado: bajo un poco
$\alpha_s < dif < \beta_s$	$w_s(t)$	Estoy en la zona buena: me quedo

donde **$D_s(t)$** = RTT actual de la fuente s, **d_s** = mínimo RTT de la fuente s, y **α y β** son parámetros (umbrales, con α < β). Lo importante es la estructura: **subir, bajar o quedarse según cuánto se está encolando**, siempre con ajustes chicos. No hay cortes a la mitad.

"Throughput TCP Vegas"

El gráfico muestra el throughput subiendo y después **oscilando cada vez menos** hasta estabilizarse. Es la consecuencia de ajustar de a poco con una banda (α, β) donde no se toca nada: converge a un valor, en vez de hacer el diente de sierra infinito de Reno.

El problema práctico de Vegas (no está en las diapositivas, pero ayuda a entender por qué no se impuso): compartiendo un enlace con conexiones Reno, Vegas pierde. Vegas frena apenas ve crecer las colas, mientras Reno sigue empujando hasta perder; Reno se queda con el ancho de banda.

Para ubicarte (fuera del programa)

Hoy Linux usa por defecto **CUBIC** (crece con una función cúbica del tiempo desde la última pérdida, pensado para enlaces rápidos), y Google usa **BBR**, que como Vegas se basa en medir RTT y ancho de banda en vez de esperar pérdidas. QUIC (clase 10) implementa estos mismos algoritmos por encima de UDP.

7. Ejercicio guiado: seguir la ventana RTT por RTT

Este es el tipo de ejercicio que suele aparecer. No está en las diapositivas; lo armé para practicar las reglas de Reno.

Datos: TCP Reno, ssthresh inicial = **16** segmentos, la ventana del receptor es grande (no limita). Durante el RTT 8 llegan **3 ACKs duplicados**. Durante el RTT 12 **vence el RTO**.

Convenciones: cwnd en segmentos; en slow start se duplica por RTT sin pasar de ssthresh; en congestion avoidance suma 1 por RTT; los cortes enteros se redondean hacia abajo.

RTT	cwnd	Fase	Qué pasa
0	1	Slow start	Arranque
1	2	Slow start	
2	4	Slow start	
3	8	Slow start	
4	16	Slow start	Llega a ssthresh = 16
5	17	Cong. avoidance	+1 por RTT
6	18	Cong. avoidance	
7	19	Cong. avoidance	
8	20	Cong. avoidance	**3 ACKs duplicados** → fast retransmit; ssthresh = 20 / 2 = **10**; cwnd = **10**
9	10	Cong. avoidance	Fast recovery: no vuelve a 1
10	11	Cong. avoidance	

RTT	cwnd	Fase	Qué pasa
11	12	Cong. avoidance	
12	13	Cong. avoidance	**Timeout** → ssthresh = 13 / 2 = **6**; cwnd = **1**
13	1	Slow start	
14	2	Slow start	
15	4	Slow start	
16	6	Slow start	Sería 8, pero se recorta en ssthresh = 6
17	7	Cong. avoidance	
18	8	Cong. avoidance	

Preguntas sobre la tabla

- **¿Cuántos segmentos se mandaron en el primer slow start (RTT 0 a 4)?** 1 + 2 + 4 + 8 + 16 = **31**.
- **¿Qué cambiaría con TCP Tahoe?** En el RTT 8, ante los 3 duplicados, Tahoe pone ssthresh = 10 pero **cwnd = 1**, y vuelve a slow start: 1, 2, 4, 8, 10, 11… Tarda 4 RTT más en volver a 10.
- **¿Y si la ventana del receptor fuera 18?** La ventana efectiva es min(cwnd, rcv_win): en los RTT 7 y 8 se mandarían 18 segmentos, no 19 y 20, aunque cwnd siga creciendo. (Y el corte del RTT 8 se calcularía sobre la ventana efectiva, 18 / 2 = 9, si se usa win como en el pseudocódigo de la clase.)
- **¿Por qué el timeout del RTT 12 castiga más que los duplicados del RTT 8?** Porque un timeout significa que no llegó nada (ni siquiera duplicados): señal de congestión grave. Los duplicados significan que el resto de los segmentos sí pasa.

8. Preguntas para autoevaluarte

1. Un servidor potente recibe datos de un cliente a través de un enlace saturado. ¿Es un problema de flujo o de congestión? ¿Qué variable de TCP lo maneja?
2. ¿Qué pasa con el throughput y con el retardo entre el knee y el cliff? ¿Y después del cliff?
3. ¿Qué diferencia hay entre controlar y evitar la congestión? ¿Qué variante de TCP hace cada cosa?
4. ¿Por qué TCP usa el modelo end-to-end y qué desventaja tiene?
5. cwnd = 12 y rcv_win = 8. ¿Cuántos segmentos puede tener el emisor sin confirmar?

6. ¿Por qué slow start crece exponencialmente si suma de a 1?
7. En congestion avoidance con cwnd = 20, ¿cuánto suma cada ACK y cuánto crece cwnd en un RTT?
8. ¿Por qué se espera a 3 ACKs duplicados y no a 1?
9. Después de un fast retransmit con cwnd = 24, ¿cuánto valen ssthresh y cwnd en Reno? ¿Y en Tahoe?
10. Después de un timeout con ventana 24, ¿cuánto valen ssthresh y cwnd?
11. ssthresh = 12, se arranca de cwnd = 1. Escribí cwnd en los primeros 7 RTT.
12. ¿Qué mide TCP Vegas para decidir si sube o baja la ventana?
13. ¿Por qué Vegas pierde contra Reno cuando comparten un enlace?

Respuestas rápidas

1. De congestión: el límite está en la red, no en el receptor. Lo maneja **cwnd**.
2. Entre knee y cliff el throughput casi no sube y el retardo crece (se llenan colas). Después del cliff hay pérdidas, retransmisiones y el throughput útil colapsa.
3. Controlar: no pasar el cliff, reaccionando a pérdidas (Reno). Evitar: quedarse antes del knee, detectando colas por el RTT (Vegas).
4. Porque IP no informa congestión: los extremos la infieren. Desventaja: reacciona tarde y depende de que todos los emisores cooperen.
5. **8**: ventana = min(12, 8).
6. Porque suma 1 por cada segmento confirmado, y en un RTT se confirman cwnd segmentos: cwnd se duplica por RTT.
7. Cada ACK suma 1/20; en un RTT vuelven 20 ACKs: **+1**.
8. Porque 1 o 2 duplicados pueden ser reordenamiento; 3 seguidos casi seguro es pérdida.
9. Reno: ssthresh = 12, cwnd = 12 (sigue en congestion avoidance). Tahoe: ssthresh = 12, cwnd = 1.
10. ssthresh = 12, cwnd = 1 (slow start).
11. 1, 2, 4, 8, **12** (recortado en ssthresh), 13, 14.
12. La diferencia entre el throughput esperado (w / RTT mínimo) y el real (w / RTT actual), que indica cuánto está encolado en los routers.
13. Porque Vegas baja en cuanto crecen las colas, y Reno sigue subiendo hasta perder paquetes: Reno ocupa el ancho de banda que Vegas libera.

Clase 10 · QUIC: transporte sobre UDP

Oct 4, 2026 · @Joaquin

Apunte para leer a la par de las diapositivas de QUIC.

1. Por qué existe QUIC: los problemas de TCP

QUIC no aparece porque TCP "ande mal", sino porque TCP se diseñó en 1981 y la web de hoy (muchos recursos por página, todo cifrado, celulares que cambian de red) choca con cuatro limitaciones que la clase enumera. Cada una tiene su respuesta en QUIC:

Problema de TCP	Qué significa	Cómo lo resuelve QUIC (sección)
Head of Line blocking (HOL)	TCP entrega un único flujo de bytes **en orden**. Si se pierde un segmento, todo lo que vino después queda esperando en el buffer hasta que llegue la retransmisión, **aunque sean datos de otra cosa** (otra imagen, otro archivo)	**Streams independientes** dentro de una conexión: una pérdida solo frena a los streams afectados (4)
Retardo del establecimiento de conexión	Handshake de 3 vías (1 RTT) antes de mandar datos, y **si se usa TLS, otro RTT más** para negociar el cifrado	Handshake de transporte y criptográfico **combinados en 1 RTT**, y **0-RTT** si ya hubo una conexión previa (6)
Cabecera de tamaño fijo	20 bytes fijos + hasta 40 de opciones: no hay lugar para extender el protocolo (las opciones ya están casi llenas, por ejemplo con SACK + timestamps)	Cabeceras flexibles y **frames** de tipos variados; nuevas funciones = nuevos tipos de frame (5)
Identificación de la conexión	Una conexión TCP es la quíntupla (IPs y puertos). **Si cambia la IP** (pasás del Wi-Fi a datos móviles, o un NAT cambia el puerto), **la conexión se pierde**	**Connection IDs** independientes de IP y puerto (3)

Por qué no arreglar TCP directamente: TCP vive en el núcleo del sistema operativo de miles de millones de equipos, y además los routers, firewalls y NAT del camino "conocen" TCP y a veces descartan lo que no reconocen (por ejemplo, opciones nuevas). Cambiarlo tarda décadas. QUIC esquiva eso: va sobre **UDP** (que todos dejan pasar) y se implementa **en la aplicación**, no en el kernel.

Historia (la línea de tiempo de la clase)

Año	Hito
2012	Jim Roskind lo desarrolla en **Google**
2013	Google lo hace público y lo presenta a la **IETF**
2014	Google considera desplegarlo gradualmente
2015	La IETF crea el **QUIC Working Group**
2017	Prácticamente todos los Chrome lo usan
2021	Se publica la **RFC 9000**, *QUIC: A UDP-Based Multiplexed and Secure Transport*

(En la diapositiva el 2014 aparece después del 2015; lo ordené cronológicamente.) La RFC 9000 es la base de **HTTP/3**, que es HTTP corriendo sobre QUIC.

2. Qué es QUIC y dónde se ubica

La definición de la clase, desarmada

- **Protocolo de transporte seguro y orientado a conexión.** Como TCP, los extremos establecen una **conexión con estado**; a diferencia de TCP, la seguridad (cifrado) **no es opcional**: viene incluida.
- **El establecimiento combina la negociación de parámetros de transporte y la criptográfica** en un solo intercambio.
- **Diseñado para intercambiar datos lo antes posible (0-RTT).**
- **Intercambia paquetes QUIC que viajan dentro de datagramas UDP.** UDP solo aporta los puertos y el checksum (clase 6); todo lo demás lo hace QUIC.
- **Se implementa en el espacio de usuario**, no en el kernel. Es una biblioteca dentro del navegador o del servidor; por eso se puede actualizar tan rápido como una aplicación.
- **Los paquetes QUIC contienen frames**, que llevan información de control y datos de la aplicación (sección 5).

- **Autentica cada paquete completo y cifra todo lo que puede**, incluso partes de la cabecera. Los equipos del camino ven muy poco (a diferencia de TCP, donde un router puede leer números de secuencia, ACKs, ventana…).
- **Las aplicaciones intercambian datos usando streams** (sección 4).

Las pilas, comparadas

Capa	Web "clásica"	Web con QUIC
Aplicación	HTTP/1.1 o HTTP/2	**HTTP/3**
Seguridad	TLS (encima de TCP)	**TLS 1.3, integrado en QUIC**
Transporte	**TCP** (en el kernel)	**QUIC** (en la aplicación) sobre **UDP**
Red	IP	IP

Lo que en la pila clásica hacen dos capas separadas (TCP la confiabilidad, TLS el cifrado), en QUIC lo hace una sola capa que coopera con TLS (sección 6). El dibujo de la clase lo muestra así: arriba, los mensajes de TLS (handshake y alertas) y las aplicaciones (h3); en el medio, **QUIC Transport** (streams, confiabilidad, congestión); abajo, **QUIC Packet Protection** (el cifrado de cada paquete).

3. Connection IDs: la conexión ya no es la quíntupla

Qué son

- **Cada conexión tiene un conjunto de identificadores**, los **Connection IDs (CID)**.
- **Cada extremo los elige de forma independiente**, y **cada uno elige el CID que va a usar el otro** para mandarle paquetes. Es decir: el servidor le dice al cliente "cuando me escribas, poné este CID", y viceversa. Así cada uno elige un identificador que le sirva para encontrar rápido la conexión en su propia tabla.
- **Función principal:** que los paquetes de una conexión **no se entreguen al extremo equivocado si cambian las direcciones de las capas inferiores** (IP o puerto UDP).

Por qué importa: migración de conexión

En TCP, la conexión **es** la quíntupla (clase 6). Si tu celular pasa del Wi-Fi a 4G, cambia tu IP, la quíntupla deja de coincidir y la conexión muere: hay que abrir otra (handshake de nuevo, TLS de nuevo). Lo mismo si un NAT (clase 5) vence una entrada y te asigna otro puerto.

En QUIC, el servidor encuentra la conexión **por el CID**, no por la IP y el puerto. Si llega un paquete con el CID correcto desde otra dirección, sigue siendo la misma conexión: **la descarga sigue sin cortarse**.

En qué cabeceras van

- **Cabecera larga** (durante el establecimiento): lleva **Destination Connection ID y Source Connection ID**, porque todavía se están eligiendo y cada uno informa el suyo.
- **Cabecera corta** (después del establecimiento): lleva **un solo CID**.

Error en la diapositiva: dice que la cabecera corta lleva solo el **Source** Connection ID. Según la RFC 9000 lleva solo el **Destination** Connection ID, y tiene lógica: el receptor lo necesita para saber a qué conexión pertenece el paquete; el origen ya no hace falta porque ya se conocen.

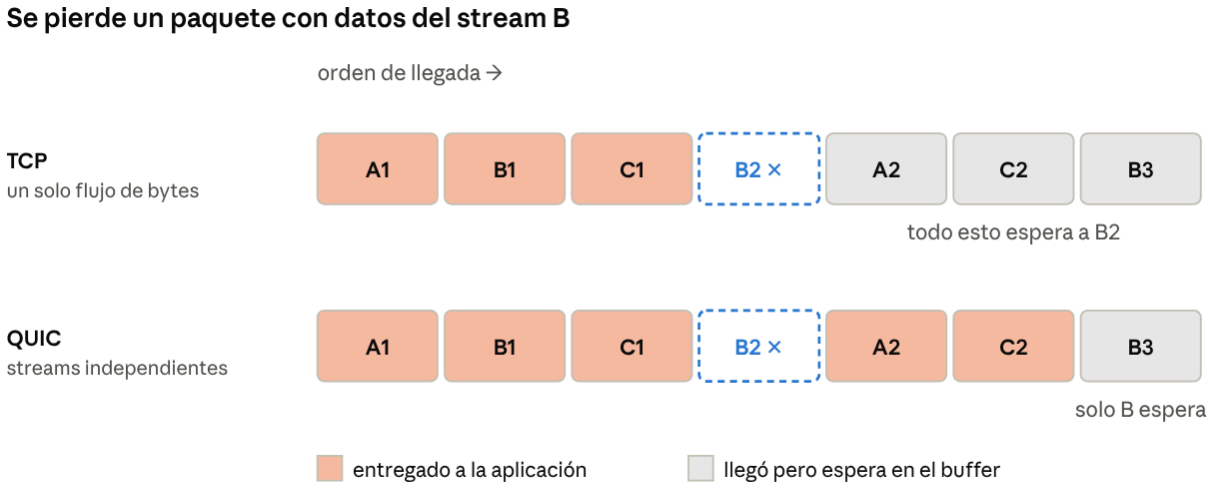
4. Streams y head-of-line blocking

Qué es un stream

Un **stream** es una **abstracción liviana dentro de una conexión QUIC** que representa un **flujo de bytes confiable y ordenado**. Es como tener muchas "mini conexiones TCP" dentro de una sola conexión, sin pagar un handshake por cada una. Lo que dice la clase:

- Pueden ser **unidireccionales o bidireccionales**.
- **Se crean simplemente enviando datos** en ellos (no hay un handshake por stream).
- **Overhead mínimo** para administrarlos (cancelar, finalizar, etc.).
- Un stream puede durar **todo lo que dure la conexión**, o muy poco (uno por pedido HTTP, por ejemplo).
- **Cualquiera de los dos extremos** puede crearlos.
- Puede haber un **número arbitrario** de streams a la vez, y los datos de distintos streams se pueden **enviar intercalados**.

Por qué esto resuelve el head-of-line blocking



A, B y C son tres recursos de una página (por ejemplo tres imágenes) que viajan por la misma conexión

Con TCP, aunque A, B y C sean archivos distintos, viajan como **un único flujo de bytes ordenado**. Si se pierde un pedazo de B, TCP no puede entregarle a la aplicación nada de lo que vino después (aunque sea de A o de C) hasta recuperar B. Eso es el **head-of-line blocking**: el primero de la fila frena a todos.

Con QUIC, **cada stream se ordena por separado**. Una pérdida en B solo frena a B; A y C siguen entregándose. **Ojo con el límite** (la clase lo dice en la parte de frames): si el **paquete** perdido llevaba frames de **varios** streams, se frenan todos esos streams. QUIC reduce el HOL blocking, no lo elimina por completo.

Identificadores de stream

- Cada stream tiene un **stream ID único de 62 bits** (de 0 a $2^{62} - 1$). Un extremo **no puede reusar** un ID.
- Los **dos bits menos significativos** codifican quién lo creó y de qué tipo es:
  - **Bit 0** (el último): quién lo inició. **0 = cliente** (IDs pares), **1 = servidor** (IDs impares).
  - **Bit 1** (el anteúltimo): **0 = bidireccional**, **1 = unidireccional**.

Bits	Iniciado por	Tipo	IDs que usa
0x00	Cliente	Bidireccional	0, 4, 8, 12…
0x01	Servidor	Bidireccional	1, 5, 9, 13…
0x02	Cliente	Unidireccional	2, 6, 10, 14…
0x03	Servidor	Unidireccional	3, 7, 11, 15…

Para saber de qué tipo es un stream, se mira **ID mod 4**. Ejemplo: el stream 9 → 9 mod 4 = 1 → bidireccional abierto por el servidor. Esto evita que los dos extremos elijan el mismo ID al mismo tiempo: cada uno numera en su propio subconjunto.

Cómo viajan los datos de un stream

- Los datos de la aplicación van en **frames STREAM**.
- Cada frame STREAM lleva **stream ID, Offset y Length**: a qué stream pertenece, en qué posición del stream empiezan esos bytes y cuántos son. El **offset** cumple el papel del número de secuencia de TCP, pero **por stream**.
- Los datos **se pasan ordenados a la aplicación**; lo que llega fuera de orden se guarda en **buffers**.
- Los **límites de un frame no se preservan**: al retransmitir o al entregar, los bytes se pueden reagrupar distinto. Igual que en TCP, la aplicación ve un flujo de bytes, no mensajes.
- Todo lo que se envía tiene que respetar los **límites de control de flujo** del otro (sección 8), y **cada extremo limita cuántos streams simultáneos** puede abrir el otro.
- La aplicación **puede indicar prioridades** entre streams; QUIC decide cómo repartir los recursos.

Estados de un stream

Cada stream tiene una **parte emisora** y una **parte receptora**, cada una con sus estados (es un "diagrama de estados TCP" simplificado, uno por sentido):

Parte emisora	Cuándo
Ready	La aplicación abrió el stream
Send	Se mandó el primer frame STREAM; se siguen mandando datos
Data Sent	Se mandó un frame STREAM con el **bit FIN**: no hay más datos
Data Recvd	Llegaron los ACK de todos los datos (terminal, todo bien)
Reset Sent → Reset Recvd	Desde Ready, Send o Data Sent se puede mandar un **RESET_STREAM** (cortar el stream); cuando lo confirman, estado terminal

Parte receptora	Cuándo
Recv	Se crea al recibir el primer STREAM (o RESET_STREAM). Guarda en buffer y entrega a la aplicación; a medida que la aplicación consume, manda **MAX_STREAM_DATA** para que el otro pueda seguir
Size Known	Llegó el frame con FIN: ya se sabe el **tamaño total** del stream (FIN + offset + length)
Data Recvd	Llegó todo
Data Read	Todo se le entregó a la aplicación (terminal)
Reset Recvd	Llegó un RESET_STREAM

(La diapositiva escribe "Size Know"; es *Size Known*.) El FIN de un stream es el análogo al FIN de TCP, pero cierra **solo ese stream**, no la conexión.

5. Paquetes y frames

Las muñecas rusas

Lo más importante de esta parte es entender **qué va adentro de qué**:

Nivel	Contiene	Regla
Datagrama UDP	1 o más **paquetes QUIC**	Varios paquetes pueden ir juntos en un datagrama (por ejemplo, durante el handshake)
Paquete QUIC	Una cabecera y 1 o más **frames**, del mismo o de distintos streams	Tiene un **número de paquete**; es la unidad que se confirma y se cifra
Frame	Un tipo específico de información: datos de un stream, un ACK, datos criptográficos…	**Tiene que caber entero en un paquete**

Un **frame** es una **unidad estructurada de comunicación** para transportar un tipo específico de datos. La **multiplexación de streams** se logra justamente **intercalando frames STREAM de varios streams dentro de uno o más paquetes**. El dibujo "Paquete,

Streams, Frames" de la clase lo muestra: dentro de cada datagrama UDP va un paquete QUIC con su cabecera, y adentro frames de los streams 1, 3, 4… mezclados; todos los datagramas pertenecen a la misma conexión (Connection ID 9).

Dato clave de la clase: la confiabilidad se maneja a nivel de paquete, no de frame. Se confirman paquetes, no frames (sección 7).

Cabecera larga y cabecera corta

	Cabecera larga	Cabecera corta
Cuándo	**Durante el establecimiento** de la conexión	**Después del establecimiento**
Tipos de paquete	Initial, 0-RTT, Handshake, Retry, Version Negotiation	**1-RTT**
Qué lleva	Versión, CID destino y origen, longitud…	Lo mínimo: CID destino y número de paquete. **No tiene campo longitud** (va al final del datagrama, ocupa lo que queda)

La idea es la misma de siempre: mientras se negocia hace falta información extra; una vez establecida la conexión, se ahorran bytes en cada paquete.

Tipos de paquete

Tipo	Lo manda	Para qué
Version Negotiation	Solo el servidor	Cuando **no conoce la versión** de QUIC que pidió el cliente; le lista las que sí soporta
Initial	Ambos	Lleva el **primer frame CRYPTO** de cliente y servidor (el comienzo del intercambio de claves)
0-RTT	Cliente	**Datos de la aplicación antes de completar el handshake**. Requiere una **conexión previa**
Handshake	Ambos	Mensajes criptográficos y ACKs del handshake
Retry	Servidor	Para **validar la IP del cliente** antes de gastar recursos en el intercambio de claves (le manda un token que el cliente tiene que devolver; evita ataques con IP falsificada)

Tipo	Lo manda	Para qué
1-RTT	Ambos	Todo lo que va después de negociar versión y claves: frames **STREAM y ACK**. El payload **siempre va protegido**

Números de paquete y espacios

- Los paquetes se numeran **de 0 a $2^{62} - 1$** y **siempre empiezan en 0** (no hay ISN aleatorio como en TCP: no hace falta, porque todo va cifrado y nadie de afuera puede inyectar paquetes válidos).
- La numeración se divide en **3 espacios independientes**, cada uno con su propia cuenta desde 0:
  - **Initial Space**: todos los paquetes Initial.
  - **Handshake Space**: todos los paquetes Handshake.
  - **Application Data Space**: los paquetes 0-RTT y 1-RTT.
- **Cada tipo de paquete usa claves de protección distintas** (sección 6). Separar los espacios hace que un ACK de un espacio no se confunda con paquetes de otro.
- Algunos paquetes (**Retry, Version Negotiation**) **no llevan frames**, y hay frames **prohibidos** en ciertos espacios (por ejemplo, no se pueden mandar datos STREAM en un Initial).

Tipos de frame (los que nombra la clase)

Cada frame empieza con un campo **Frame Type**.

Frame	Para qué	Su análogo en TCP
CRYPTO	Mensajes criptográficos del handshake (los de TLS)	— (TLS va aparte, encima de TCP)
STREAM	Datos de un stream; **crea el stream implícitamente** si no existía	Los datos de un segmento
ACK	Confirma **paquetes** (no frames)	Campo ACK + SACK
RESET_STREAM	Termina abruptamente la parte emisora de un stream	RST, pero solo de un stream
PING	Verificar que el otro sigue vivo	Keepalive
MAX_DATA	Control de flujo: cuánto se puede mandar en toda la conexión	Campo Window

Frame	Para qué	Su análogo en TCP
CONNECTION_CLOSE	Avisar que se cierra la conexión	FIN / RST

(La diapositiva escribe "STREAMS"; el frame se llama STREAM.) La columna de la derecha muestra la filosofía de QUIC: lo que en TCP son **campos fijos de la cabecera**, en QUIC son **frames** que se mandan solo cuando hacen falta. Para agregar una función nueva, se define un tipo de frame nuevo: es la respuesta a "cabecera de tamaño fijo" de la sección 1.

La limitación que remarca la clase: la pérdida de un paquete bloquea todos los streams para los cuales contiene frames. Por eso a los implementadores les conviene no mezclar demasiados streams en un mismo paquete.

6. Handshake y seguridad: QUIC + TLS 1.3

TLS 1.3 en dos minutos

TLS (Transport Layer Security) es el protocolo que da un **canal seguro** entre dos extremos (es la "S" de HTTPS). Lo que dice la clase:

- Provee **autenticación del servidor** ante el cliente (y opcionalmente del cliente ante el servidor), y **confidencialidad e integridad** de los datos.
- **Requiere un transporte confiable** para funcionar: sus mensajes tienen que llegar completos y en orden. Por eso, en la web clásica, va encima de TCP.
- **QUIC se basa en TLS 1.3 y no soporta otra versión.**
- Tiene dos componentes:
  - **Handshake Protocol**: autenticación, negociación de algoritmos criptográficos e **intercambio de claves**.
  - **Record Protocol**: la transferencia segura de datos con las claves acordadas. Cada mensaje se transporta como una serie de **TLS records**, cada uno protegido individualmente.
- El cliente inicia un **intercambio autenticado de claves** (con **PSK**, una clave precompartida de una sesión anterior, y/o **Diffie-Hellman**, que permite acordar un secreto sin enviarlo). Si sale bien, los dos comparten un **secreto**; el cliente además **autenticó al servidor** con su certificado **X.509**.

El dibujo del handshake de TLS 1.3 tiene esta secuencia (entre llaves, lo que ya viaja cifrado):

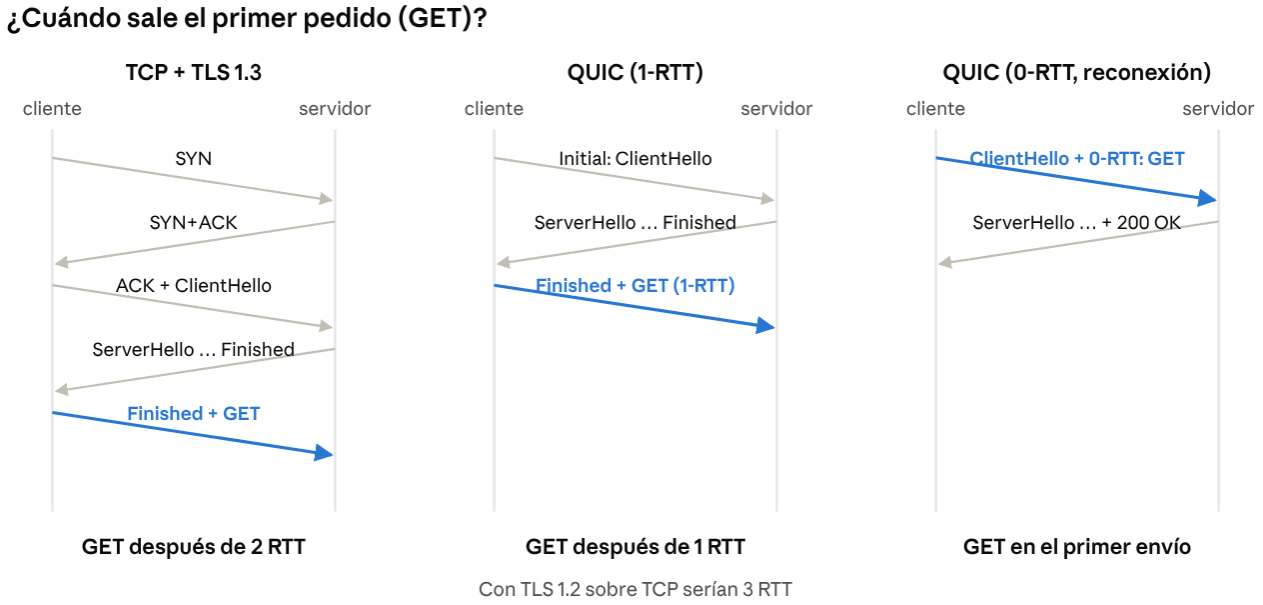
Quién	Mensajes
Cliente	**ClientHello** (versión, número aleatorio, algoritmos que soporta, su parte de Diffie-Hellman)
Servidor	**ServerHello** (algoritmo elegido, su parte de Diffie-Hellman) → desde acá ya hay claves; {EncryptedExtensions}, {CertificateRequest} (opcional), {Certificate}, {CertificateVerify}, {**Finished**}
Cliente	{Certificate} y {CertificateVerify} (solo si se los pidieron), {**Finished**}

El **Finished** es un resumen cifrado de todo el handshake: si alguien hubiera alterado algo en el camino, no coincidiría. **El handshake está completo cuando cada extremo mandó su Finished y verificó el del otro.**

Cómo se conecta QUIC: 1 RTT

QUIC combina el handshake de transporte y el criptográfico en un solo RTT. No hay un "SYN" separado: el primer paquete ya lleva el ClientHello.

1. El cliente manda un **ClientHello** de TLS 1.3 dentro de un **frame CRYPTO**, en un **paquete Initial**.
2. El servidor responde con el **ServerHello** y toda la información de seguridad (certificado, Finished) en varios paquetes: **Initial** (el ServerHello) y **Handshake** (el resto, ya cifrado). **Opcionalmente puede mandar ya datos en paquetes 1-RTT.**
3. El cliente termina el handshake (su **Finished**, en un paquete Handshake) y **en el mismo vuelo** manda su primer pedido en un **paquete 1-RTT** (en el dibujo: "[Stream Frame] GET /").



Las dos columnas de QUIC resumen los dibujos "1-RTT Handshake" y "0-RTT Handshake" de la clase

0-RTT: mandar datos antes de terminar el handshake

- **Permite al cliente mandar datos de la aplicación antes de que el handshake esté completo.**
- Funciona **solo si hubo una conexión previa: reutiliza parámetros negociados** antes, que el cliente recuerda.
- El cliente le da al servidor un **ticket de sesión TLS** (en el dibujo: "Initial [Token + Crypto frames]") para que el servidor pueda **recuperar esos datos** y descifrar lo que viene.
- En el dibujo, el cliente manda ClientHello + **paquete 0-RTT con "GET /"** en el primer envío, y el servidor contesta ServerHello… junto con el **"200 OK"**.

Lo que no dice la diapositiva pero conviene saber: los datos 0-RTT se pueden **repetir** (un atacante que capturó el paquete puede reenviarlo, *replay*), porque todavía no hay un intercambio nuevo que lo impida. Por eso solo se usan para pedidos que no hacen daño si se repiten, como un GET, y no para "pagar".

QUIC y TLS cooperan (no es un modelo de capas estricto)

- **QUIC usa el handshake de TLS** (autenticación, negociación de parámetros de seguridad y de transporte).
- **TLS usa la confiabilidad y el orden de QUIC**: los mensajes de TLS (handshake y alertas) viajan en **frames CRYPTO**, que tienen **offset y longitud** como los STREAM, así que llegan completos y ordenados. Es el "transporte confiable" que TLS necesita.
- Pero **el cifrado de los paquetes lo hace QUIC**, con las claves que le entrega TLS. No se usa el Record Protocol de TLS.

- TLS le avisa a QUIC los cambios: claves nuevas, handshake terminado, certificado del servidor, etc.

Niveles de cifrado

Cada pedazo de datos de TLS va asociado a las claves que TLS está usando en ese momento, y **cada nivel de cifrado corresponde a un espacio de números de paquete**:

Tipo de paquete	Claves	Espacio de números
Initial	Initial secrets	Initial
0-RTT Protected	0-RTT	Application Data
Handshake	Handshake	Handshake
Retry	Retry	—
Version Negotiation	— (sin cifrar)	—
Cabecera corta (1-RTT)	1-RTT	Application Data

Cómo evolucionan las claves:

1. **Initial**: todavía no se negoció nada, así que se cifran con una clave **derivada del Destination Connection ID** (que viaja en el mismo paquete). Ojo: cualquiera que vea el paquete puede calcularla; sirve para que los equipos del camino no lo manipulen, **no para que sea secreto**.
2. **0-RTT**: reutiliza claves y parámetros de la conexión anterior.
3. **Handshake**: en cuanto se intercambiaron ClientHello y ServerHello, ya se pueden derivar claves (gracias a Diffie-Hellman) para el resto del handshake. Desde acá es secreto de verdad.
4. **1-RTT**: al completar el handshake se derivan las claves para **toda la comunicación futura**.

Cómo se arma un paquete protegido

1. Un paquete tiene **cabecera y payload**.
2. El **payload se cifra** directamente con las claves del nivel actual.
3. Después se aplica **Header Protection**: se ocultan algunas partes de la cabecera, como **el número de paquete**, para que un observador no pueda seguir la conexión ni deducir pérdidas.
4. Cabecera protegida + payload cifrado = paquete QUIC listo para meter en un datagrama UDP.

7. ACKs, retransmisión y cierre de la conexión

La gran diferencia con TCP: se confirman paquetes, no bytes

En TCP, el número de secuencia cumple **dos funciones a la vez**: dice **en qué posición del flujo** van los datos y es **lo que se confirma** con el ACK. Si retransmitís el segmento 5000, sale otra vez con SN = 5000, y ahí aparece la **ambigüedad** de Karn (clase 7): el ACK, ¿es del original o de la copia?

QUIC **separa las dos funciones**:

- **La posición de los datos** la da el **offset** del frame STREAM, dentro de su stream.
- **Lo que se confirma** es el **número de paquete**, que **solo crece**.

La clase lo dice así: "QUIC separa el orden de transmisión del orden de envío". (Lo que quiere decir es **orden de transmisión** frente a **orden de entrega** de los datos; "envío" está mal puesto.)

Consecuencias, una por una como en la clase:

- Las confirmaciones viajan en **frames ACK**.
- **Cada paquete tiene un número que el otro extremo tiene que confirmar**, y ese reconocimiento **incluye todos los frames** que iban en el paquete.
- **Cada paquete QUIC de un datagrama UDP se confirma individualmente** (aunque viajen juntos).
- **Si se pierde un paquete, todos los streams con datos en él se bloquean** esperando la retransmisión.
- **Un paquete perdido nunca se retransmite tal cual: se genera otro paquete con un número nuevo**, que lleva los frames que hagan falta (los datos de STREAM con el mismo offset). Como el número es nuevo, **cuando llega el ACK se sabe exactamente a qué envío corresponde**: el RTT se mide sin ambigüedad, sin necesidad del algoritmo de Karn.

El frame ACK

- Confirma **el último paquete recibido** e indica los **huecos** (*gaps*): es como si TCP mandara siempre SACK. Con eso el emisor sabe exactamente qué se perdió. (La diapositiva dice "gaps en el stream": son huecos en la **numeración de paquetes**.)
- Los paquetes **Initial y Handshake se confirman inmediatamente** (si ya se tienen las claves para leerlos): el handshake tiene que avanzar rápido.
- Los paquetes **0-RTT y 1-RTT se confirman dentro del tiempo *max_ack_delay*** que anunció el otro. Se permite demorar un poco para juntar varios ACK en uno (como el *delayed ACK* de TCP).

- **Un paquete no se confirma hasta haberlo descifrado y procesado todos sus frames.** Confirmar es decir "lo tengo y es válido".
- **Igual que en TCP, un ACK no significa que la aplicación leyó los datos**: solo que QUIC los recibió.

Cierre de la conexión: tres formas

Forma	Cómo funciona	Cuándo
Tiempo de inactividad (*idle timeout*)	Cliente y servidor anuncian su tiempo máximo de inactividad y se usa **el menor**. Si pasa ese tiempo sin recibir nada, la conexión **se cierra en silencio**, sin mandar nada. Cada paquete recibido y procesado bien **reinicia** el contador	Conexiones abandonadas. Para mantenerla viva se pueden mandar PING
Cierre inmediato	Un extremo manda un frame **CONNECTION_CLOSE** y **todos los streams se cierran en el acto**	La aplicación quiere cerrar, o hubo una **violación del protocolo**
Reset sin estado (*stateless reset*)	Último recurso: un extremo recibe paquetes que **no puede asociar a ninguna conexión** (por ejemplo, porque se reinició y perdió el estado) y responde con un paquete especial que le indica al otro que la conexión murió	Equivale al RST de TCP ante una half-open connection

Lo que remarca la clase: QUIC no tiene un cierre "elegante" de la conexión como el FIN de TCP con half-close. El cierre ordenado existe, pero **por stream** (bit FIN en el frame STREAM, sección 4); la conexión entera se termina de golpe con CONNECTION_CLOSE o por inactividad. La aplicación es la que tiene que asegurarse de terminar sus streams antes de cerrar.

8. Control de flujo (y de congestión)

Por qué dos niveles

Como en TCP, el receptor necesita **limitar cuántos datos puede recibir y guardar en su buffer**. Pero ahora hay muchos streams en una conexión, y aparecen dos riesgos distintos:

Nivel	Qué previene	Frame que lo amplía
Stream	Que **un solo stream** se coma todo el buffer de la conexión y deje sin lugar a los demás	**MAX_STREAM_DATA**
Conexión	Que la **suma de todos los streams** supere la capacidad del buffer del receptor	**MAX_DATA**

El emisor no puede pasarse de ninguno de los dos límites. Ejemplo: límite de conexión 100 KB y de stream 40 KB. Con tres streams, cada uno puede mandar hasta 40 KB, pero entre los tres no más de 100 KB.

Cómo funciona

- Los **límites iniciales** para todos los streams se fijan **durante el handshake** (como parámetros de transporte).
- A medida que la aplicación consume datos, el receptor manda **MAX_DATA** o **MAX_STREAM_DATA** para **anunciar límites más grandes**.
- Los límites son **absolutos, no relativos**: no dicen "me quedan 10 KB libres" (como la ventana de TCP, que es relativa al ACK), sino "podés mandar hasta el byte 150.000 de este stream". Para la conexión, el límite es el **máximo de la suma de los offsets de todos los streams**, y el receptor lleva la **suma acumulada** de bytes recibidos en todos los streams.
- **Si el receptor anuncia un límite menor que el anterior, no es un error, pero no tiene efecto**: un límite absoluto solo puede crecer. El emisor **ignora todo frame que no aumente los límites**. (En TCP, en cambio, la ventana sí se puede achicar.)
- **Si el emisor viola los límites** (de la conexión o de cualquier stream), el receptor **cierra la conexión con un error FLOW_CONTROL_ERROR**.

Emisor bloqueado

- Si el emisor ya mandó datos **hasta el límite**, **no puede mandar más y se considera bloqueado**.
- Debería avisarle al receptor con un frame **STREAM_DATA_BLOCKED** (bloqueado en un stream) o **DATA_BLOCKED** (bloqueado en la conexión): "tengo datos para mandar pero tus límites no me dejan".
- Debería mandarlos **periódicamente**, para que la conexión **no se cierre por inactividad** mientras espera.

Es la misma situación que la ventana 0 de TCP, resuelta de otra forma: en vez del **persist timer** (el emisor pregunta mandando 1 byte de prueba), el emisor **avisa explícitamente** que está bloqueado.

¿Y el control de congestión?

Las diapositivas no lo desarrollan, pero QUIC **sí tiene control de congestión**, y usa las mismas ideas de la clase 9 (ventana de congestión, slow start, reducción ante pérdidas; el estándar es la RFC 9002, con un algoritmo tipo Reno/NewReno, y las implementaciones suelen usar CUBIC o BBR). Dos ventajas frente a TCP:

- Mide el **RTT sin ambigüedad**, porque las retransmisiones van en paquetes con número nuevo (sección 7).
- Como corre en la aplicación, **se puede cambiar el algoritmo sin tocar el sistema operativo**.

9. QUIC frente a TCP, y preguntas para autoevaluarte

La tabla para repasar

	TCP (+ TLS)	QUIC
Va sobre	IP	UDP
Dónde se implementa	Kernel del SO	Espacio de usuario (la aplicación)
Cifrado	Opcional, TLS aparte	Obligatorio, TLS 1.3 integrado; cifra hasta parte de la cabecera
Establecimiento	1 RTT (TCP) + 1 RTT (TLS 1.3)	1 RTT; 0-RTT si hubo conexión previa
Identificación de la conexión	Quíntupla (IP y puertos)	Connection IDs: sobrevive a cambios de IP
Flujos	Uno solo, ordenado	Muchos streams independientes
Head-of-line blocking	Una pérdida frena todo	Solo frena a los streams del paquete perdido
Qué se numera / confirma	Bytes (SN y ACK sobre la misma numeración)	Paquetes (número que solo crece); los datos se ubican por offset
Retransmisión	Mismo SN → ambigüedad (Karn)	Paquete nuevo con número nuevo → sin ambigüedad

	TCP (+ TLS)	QUIC
ACK selectivo	Opción SACK (máx. 4 bloques)	Siempre: el frame ACK informa los huecos
Control de flujo	Ventana relativa (campo Window)	Límites absolutos por stream y por conexión (MAX_STREAM_DATA, MAX_DATA)
Extensibilidad	Opciones, máx. 40 bytes	Tipos de frame nuevos
Cierre	FIN (ordenado, half-close) o RST	Por inactividad, CONNECTION_CLOSE o stateless reset; el FIN es por stream

Preguntas

1. Nombrá los cuatro problemas de TCP que motivan QUIC y cómo los resuelve cada uno.
2. ¿Por qué QUIC va sobre UDP en lugar de ser un protocolo nuevo sobre IP (con su propio número de protocolo)?
3. Estás descargando un archivo por QUIC y pasás del Wi-Fi a datos móviles. ¿Qué pasa con la conexión? ¿Y si fuera TCP?
4. ¿De qué tipo es el stream con ID 6? ¿Y el 13?
5. Ordena de más grande a más chico: frame, datagrama UDP, paquete QUIC.
6. ¿Por qué QUIC no elimina por completo el head-of-line blocking?
7. ¿Cuándo se usa la cabecera larga y cuándo la corta? ¿Qué tipo de paquete usa la corta?
8. ¿Para qué sirven los paquetes Retry y Version Negotiation?
9. ¿Cuántos RTT pasan antes de mandar el primer GET con TCP + TLS 1.3, con QUIC y con QUIC 0-RTT?
10. ¿Qué necesita el cliente para poder usar 0-RTT, y qué riesgo tiene?
11. ¿Por qué QUIC no necesita el algoritmo de Karn para medir el RTT?
12. Los paquetes Initial van cifrados. ¿Eso los hace secretos? ¿Por qué?
13. El receptor anuncia MAX_STREAM_DATA = 50.000 y después MAX_STREAM_DATA = 30.000. ¿Qué pasa?
14. ¿Qué hace un emisor que llegó al límite de control de flujo? ¿Cuál es el equivalente en TCP?
15. ¿De qué tres maneras puede terminar una conexión QUIC?

Respuestas rápidas

1. HOL blocking → streams independientes; retardo de establecimiento → handshake combinado de 1 RTT y 0-RTT; cabecera fija → frames; conexión atada a IP y puerto → Connection IDs.
2. Porque routers, firewalls y NAT dejan pasar UDP, y un protocolo nuevo sobre IP sería bloqueado o tardaría décadas en desplegarse; además, sobre UDP se puede implementar en la aplicación.
3. Con QUIC la conexión sigue, porque el servidor la identifica por el CID y no por la IP. Con TCP se corta (cambió la quíntupla) y hay que abrir otra.
4. 6 mod 4 = 2 → unidireccional, abierto por el cliente. 13 mod 4 = 1 → bidireccional, abierto por el servidor.
5. Datagrama UDP ⊃ paquete QUIC ⊃ frame.
6. Porque si un paquete perdido llevaba frames de varios streams, todos esos streams quedan esperando la retransmisión.
7. Larga durante el establecimiento (Initial, 0-RTT, Handshake, Retry, Version Negotiation); corta después, en los paquetes 1-RTT.
8. Retry: el servidor valida la IP del cliente antes de gastar recursos. Version Negotiation: el servidor no soporta la versión pedida y le informa las que sí.
9. 2, 1 y 0 (el GET va en el primer envío).
10. Haber tenido una conexión previa con ese servidor (ticket de sesión y parámetros guardados). Riesgo: los datos 0-RTT se pueden repetir (replay).
11. Porque cada retransmisión viaja en un paquete con número nuevo: el ACK identifica exactamente qué envío llegó.
12. No: la clave se deriva del Destination Connection ID, que viaja en el paquete. Cualquiera puede descifrarlos; sirve para evitar manipulaciones, no para ocultar.
13. Nada: anunciar un límite menor no es error pero no tiene efecto; el límite sigue en 50.000.
14. Queda bloqueado y manda STREAM_DATA_BLOCKED o DATA_BLOCKED periódicamente. En TCP es la ventana 0, que se resuelve con el persist timer.
15. Por tiempo de inactividad (silencioso), por cierre inmediato con CONNECTION_CLOSE, o con un stateless reset cuando un extremo perdió el estado.